Tarea 2: Path Tracing con OpenMP y Numba

Declaracion de uso de IA

Se uso un asistente de IA para revisar la pauta y clases, implementar las versiones OpenMP y Numba, automatizar experimentos, revisar cobertura de resultados y generar este informe. Los resultados numericos provienen de las tres corridas siguientes:

- `results/final/local/results.csv` (computador Local del grupo).
- `results/final/cluster/results.csv` (cluster IALab).
- `results/final/grupo/results.csv` (computador del integrante adicional del grupo). La carpeta `results/final/grupo/` espeja la corrida original `results/full_grupo_jeanf_20260514_105212/`.

Entorno de ejecucion

Maquina	Procesador	Cores fisicos	Threads logicos	Threads/core	Max MHz
Local	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz	6	12	2	No informado
Cluster	Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz	20	40	2	3200.0000
Computador grupo	Intel(R) Core(TM) i5-10600K CPU @ 4.10GHz	6	12	2	4100 (base)

Cobertura de resultados

Las corridas Local y Cluster corresponden a la matriz completa con `--full --reps 1`: dos escenas, tres configuraciones y p en {1, 2, 4, 8}.

Maquina	Filas	Serial	Scheduling	OpenMP cmp.	Numba cmp.
Local	148	12	88	24	24
Cluster	148	12	88	24	24
Computador grupo	148	12	88	24	24

Configuraciones

Config.	Resolucion	S ray tracing	N path tracing
small	400 x 300	2	32
medium	800 x 600	4	128
large	1600 x 1200	4	256

Implementacion

`omp_pt.cpp` paraleliza el loop de pixeles con `#pragma omp parallel for`. `omp_pt_sched.cpp` usa `schedule(runtime)` y `omp_set_schedule` para probar `static`, `dynamic` y `guided` con distintos chunks. Cada pixel es independiente y escribe una posicion unica en la imagen, por lo que la paralelizacion sigue el patron map.

`numba_pt.py` implementa la misma logica con `@njit(parallel=True)` y `prange`. El script realiza un warm-up minimo antes de medir para excluir el costo de compilacion JIT inicial de Numba.

Resultados seriales

Comparacion serial_pt entre computadores

El cociente es `Cluster/Local`; valores mayores a 1 indican que el cluster demoro mas que el equipo local.

Escena	Config.	Local Ts	Cluster Ts	Grupo Ts	Cluster/Local	Grupo/Local
scene.txt	small	4.075	6.443	5.025	1.58x	1.23x
scene.txt	medium	64.690	100.985	73.510	1.56x	1.14x
scene.txt	large	533.884	811.762	621.377	1.52x	1.16x
scene_many.txt	small	6.177	11.424	7.216	1.85x	1.17x
scene_many.txt	medium	98.010	175.809	121.263	1.79x	1.24x
scene_many.txt	large	782.014	1406.140	926.600	1.80x	1.18x

Local: serial.cpp vs serial_pt.cpp

Escena	Config.	serial.cpp	serial_pt.cpp
scene.txt	small	0.083	4.075
scene.txt	medium	1.379	64.690
scene.txt	large	5.658	533.884
scene_many.txt	small	0.220	6.177
scene_many.txt	medium	3.484	98.010
scene_many.txt	large	13.775	782.014

Crecimiento Local

Escena	small -> medium	medium -> large
scene.txt	15.87x	8.25x
scene_many.txt	15.87x	7.98x

Cluster: serial.cpp vs serial_pt.cpp

Escena	Config.	serial.cpp	serial_pt.cpp
scene.txt	small	0.129	6.443
scene.txt	medium	1.792	100.985
scene.txt	large	7.550	811.762
scene_many.txt	small	0.376	11.424
scene_many.txt	medium	6.262	175.809
scene_many.txt	large	25.113	1406.140

Crecimiento Cluster

Escena	small -> medium	medium -> large
scene.txt	15.67x	8.04x
scene_many.txt	15.39x	8.00x

El crecimiento observado es consistente con el costo esperado O(WHNDK). De `small` a `medium` se cuadruplica la resolucion y N pasa de 32 a 128, por lo que se espera cerca de 16x. De `medium` a `large` la resolucion vuelve a crecer 4x y N se duplica, por lo que se espera cerca de 8x.

Respuesta directa a las preguntas de la parte (a):

- *¿En cuanto crece el tiempo al duplicar la resolucion?* Cerca de **4x**. La resolucion entra como `W*H`: si se duplican ambas dimensiones, el numero total de pixeles se multiplica por 4 y el costo de path tracing es lineal en el numero de pixeles, porque cada pixel es un trabajo independiente. Se verifica en las mediciones: la transicion `small -> medium` cuadruplica resolucion *y* cuadruplica N (32 -> 128), dando ~16x = 4 (resolucion) x 4 (N).
- *¿Y al duplicar N?* Cerca de **2x**. N es el numero de caminos por pixel y entra linealmente en el costo (`O(W*H*N*D*K)`): duplicar N lanza el doble de caminos por pixel, por lo que el tiempo se duplica. Se verifica en `medium -> large`: la resolucion se cuadruplica y N se duplica (128 -> 256), dando ~8x = 4 x 2.
- *Justificacion:* el path tracer pasa la mayor parte del tiempo trazando rayos. Cada rayo cuesta `O(D*K)` por las intersecciones con K objetos y D rebotes maximos, y el numero total de rayos es `W*H*N`. La terminacion anticipada por `throughput < 0.01` y por cache hacen que los factores sean ligeramente menores a los predichos (15.87x en vez de 16x; 8.04x en vez de 8x), pero los ordenes coinciden.

Barrido de scheduling OpenMP

Local

Escena	Mejor promedio	Tiempo prom.
scene.txt	guided(8)	34.511
scene_many.txt	dynamic(128)	53.780

Escena	p=1	p=2	p=4	p=8
scene.txt	static(32), 64.336s	guided(8), 37.323s	static(1), 21.527s	dynamic(1), 14.067s
scene_many.txt	dynamic(128), 99.200s	guided(32), 57.949s	static(1), 34.195s	dynamic(8), 22.673s

Cluster

Escena	Mejor promedio	Tiempo prom.
scene.txt	dynamic(128)	50.311
scene_many.txt	dynamic(1)	86.811

Escena	p=1	p=2	p=4	p=8
scene.txt	guided(1), 101.302s	dynamic(128), 52.347s	guided(32), 27.533s	dynamic(1), 19.556s
scene_many.txt	dynamic(1), 169.603s	guided(1), 89.833s	guided(8), 47.791s	dynamic(1), 37.343s

No aparece una politica unica para todos los casos. La carga por pixel es irregular porque algunos rayos terminan en el fondo y otros hacen mas rebotes, sombras e intersecciones; por eso `dynamic` o `guided` pueden mejorar el balance, aunque chunks demasiado pequenos agregan overhead. En la escena con 40 esferas aumenta el costo de interseccion y el balance de carga pesa mas.

OpenMP vs Numba

Para cada punto se usa como referencia el tiempo `serial_pt` de la misma escena y configuracion. Se calcula `S(p)=Ts/T(p)` y `E(p)=S(p)/p`. Las siguientes tablas incluyen todos los puntos p medidos para Local y Cluster.

Resultados paralelos completos - Local

Local: scene.txt - small

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	4.181	0.97	0.97	3.579	1.14	1.14	Numba
2	2.406	1.69	0.85	1.994	2.04	1.02	Numba
4	1.346	3.03	0.76	1.142	3.57	0.89	Numba
8	0.765	5.32	0.67	0.659	6.18	0.77	Numba

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor

Local: scene.txt - medium

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	70.217	0.92	0.92	51.965	1.24	1.24	Numba
2	39.678	1.63	0.82	32.416	2.00	1.00	Numba
4	23.280	2.78	0.69	18.847	3.43	0.86	Numba
8	15.464	4.18	0.52	12.602	5.13	0.64	Numba

Local: scene.txt - large

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	581.278	0.92	0.92	399.574	1.34	1.34	Numba
2	302.865	1.76	0.88	261.006	2.05	1.02	Numba
4	175.561	3.04	0.76	159.330	3.35	0.84	Numba
8	112.857	4.73	0.59	102.854	5.19	0.65	Numba

Local: scene_many.txt - small

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	6.308	0.98	0.98	5.488	1.13	1.13	Numba
2	3.659	1.69	0.84	3.482	1.77	0.89	Numba
4	2.227	2.77	0.69	2.049	3.02	0.75	Numba
8	1.251	4.94	0.62	1.298	4.76	0.59	OpenMP

Local: scene_many.txt - medium

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	101.456	0.97	0.97	87.361	1.12	1.12	Numba
2	58.990	1.66	0.83	56.068	1.75	0.87	Numba
4	34.330	2.85	0.71	34.581	2.83	0.71	OpenMP
8	22.855	4.29	0.54	23.217	4.22	0.53	OpenMP

Local: scene_many.txt - large

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	796.495	0.98	0.98	696.564	1.12	1.12	Numba
2	468.221	1.67	0.84	452.235	1.73	0.86	Numba
4	279.796	2.79	0.70	284.197	2.75	0.69	OpenMP
8	182.725	4.28	0.53	185.490	4.22	0.53	OpenMP

En Local, Numba fue mas rapido que OpenMP en 19 de 24 puntos comparables. El cociente promedio `T_OpenMP/T_Numba` fue 1.13x. La mayor ventaja de Numba fue 1.45x en scene.txt `large` con p=1. La menor razon fue 0.96x en scene_many.txt `small` con p=8.

Resumen a 8 threads - Local

Escena	Config.	Ts	OMP T8	OMP S8	OMP E8	Numba T8	Numba S8	Numba E8	Mejor
scene.txt	small	4.075	0.765	5.32	0.67	0.659	6.18	0.77	Numba
scene.txt	medium	64.690	15.464	4.18	0.52	12.602	5.13	0.64	Numba
scene.txt	large	533.884	112.857	4.73	0.59	102.854	5.19	0.65	Numba
scene_many.txt	small	6.177	1.251	4.94	0.62	1.298	4.76	0.59	OpenMP
scene_many.txt	medium	98.010	22.855	4.29	0.54	23.217	4.22	0.53	OpenMP
scene_many.txt	large	782.014	182.725	4.28	0.53	185.490	4.22	0.53	OpenMP

Resultados paralelos completos - Cluster

Cluster: scene.txt - small

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	6.617	0.97	0.97	4.886	1.32	1.32	Numba
2	3.416	1.89	0.94	2.744	2.35	1.17	Numba
4	1.778	3.62	0.91	1.484	4.34	1.09	Numba
8	1.269	5.08	0.63	1.084	5.94	0.74	Numba

Cluster: scene.txt - medium

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	103.303	0.98	0.98	73.706	1.37	1.37	Numba
2	52.479	1.92	0.96	44.085	2.29	1.15	Numba
4	27.837	3.63	0.91	23.239	4.35	1.09	Numba
8	19.699	5.13	0.64	16.977	5.95	0.74	Numba

Cluster: scene.txt - large

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	822.579	0.99	0.99	581.332	1.40	1.40	Numba
2	421.544	1.93	0.96	352.311	2.30	1.15	Numba
4	219.374	3.70	0.93	185.739	4.37	1.09	Numba
8	157.145	5.17	0.65	135.203	6.00	0.75	Numba

Cluster: scene_many.txt - small

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	11.421	1.00	1.00	8.166	1.40	1.40	Numba
2	5.732	1.99	1.00	4.711	2.43	1.21	Numba
4	3.062	3.73	0.93	2.596	4.40	1.10	Numba
8	2.380	4.80	0.60	2.013	5.68	0.71	Numba

Cluster: scene_many.txt - medium

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	177.196	0.99	0.99	128.415	1.37	1.37	Numba
2	91.865	1.91	0.96	75.439	2.33	1.17	Numba
4	47.580	3.70	0.92	41.411	4.25	1.06	Numba
8	37.521	4.69	0.59	31.523	5.58	0.70	Numba

Cluster: scene_many.txt - large

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	1434.800	0.98	0.98	1043.959	1.35	1.35	Numba
2	732.822	1.92	0.96	613.289	2.29	1.15	Numba
4	381.005	3.69	0.92	332.149	4.23	1.06	Numba
8	298.016	4.72	0.59	252.487	5.57	0.70	Numba

En Cluster, Numba fue mas rapido que OpenMP en 24 de 24 puntos comparables. El cociente promedio `T_OpenMP/T_Numba` fue 1.24x. La mayor ventaja de Numba fue 1.41x en scene.txt `large` con p=1. La menor razon fue 1.15x en scene_many.txt `large` con p=4.

Resumen a 8 threads - Cluster

Escena	Config.	Ts	OMP T8	OMP S8	OMP E8	Numba T8	Numba S8	Numba E8	Mejor
scene.txt	small	6.443	1.269	5.08	0.63	1.084	5.94	0.74	Numba
scene.txt	medium	100.985	19.699	5.13	0.64	16.977	5.95	0.74	Numba
scene.txt	large	811.762	157.145	5.17	0.65	135.203	6.00	0.75	Numba
scene_many.txt	small	11.424	2.380	4.80	0.60	2.013	5.68	0.71	Numba
scene_many.txt	medium	175.809	37.521	4.69	0.59	31.523	5.58	0.70	Numba
scene_many.txt	large	1406.140	298.016	4.72	0.59	252.487	5.57	0.70	Numba

La eficiencia cae al aumentar p por overhead de planificacion, barreras implicitas, sincronizacion, presion de cache y saturacion de recursos compartidos. Aunque el cluster tiene mas cores fisicos, estas corridas solo usan hasta 8 threads, por lo que no aprovechan los 20 cores fisicos completos del nodo.

Respuesta directa a las preguntas de la parte (e):

- *¿Cual escala mejor?* **Numba escala ligeramente mejor que OpenMP**: en Local Numba gana en 19/24 puntos comparables y en Cluster en 24/24, con cociente promedio `T_OpenMP/T_Numba` de 1.13x (Local) y 1.24x (Cluster). Esto se debe principalmente a que Numba aplica LLVM con vectorizacion agresiva y al binding propio de hilos que mantiene mejor afinidad de cache; aun asi, ambas implementaciones tienen curvas de speedup paralelas, dado que el algoritmo es el mismo.
- *¿A partir de que numero de threads decae la eficiencia?* La eficiencia ya cae desde **p=2** (E suele bajar de 1.0 a 0.85-0.95 al pasar de p=1 a p=2). El **quiebre marcado** ocurre en **p=8** en los equipos Local y Grupo (E pasa de ~0.92 en p=4 a ~0.50-0.65 en p=8) porque ambos tienen **6 cores fisicos**: los threads 7 y 8 se ejecutan en cores con Hyper-Threading que comparten unidades funcionales con los primeros 6 threads, asi que aportan menos trabajo paralelo real. En el Cluster (20 cores fisicos) el quiebre no se observa hasta p=8 tampoco, pero esta a otro nivel: alli la caida se explica por mayor latencia de memoria NUMA (dos sockets) y contencion del nodo compartido.
- *¿A que se debe la caida?* Combinacion de: (1) **SMT/Hyper-Threading**: dos threads logicos por core fisico comparten ALU/FPU/cache L1-L2, asi que el segundo thread por core no duplica el throughput; (2) **fork-join overhead**: cada region `parallel for` paga sincronizacion al inicio y al final; (3) **contencion de cache L3**: mas threads compitiendo por el mismo cache; (4) en `large`, **saturacion del ancho de banda de memoria**, porque el array `pixels` no cabe en cache y hay accesos coalescidos en lectura/escritura.

Comparacion entre computadores

La comparacion usa p=8 para las implementaciones paralelas. El cociente es `Cluster/Local`; valores mayores a 1 indican que el cluster demoro mas que el equipo local. La columna del computador de grupo queda preparada para completar despues.

OpenMP p=8

Escena	Config.	Local T8	Cluster T8	Grupo T8	Cluster/Local	Grupo/Local	Mas rapido
scene.txt	small	0.765	1.269	0.720	1.66x	0.94x	Grupo
scene.txt	medium	15.464	19.699	13.320	1.27x	0.86x	Grupo
scene.txt	large	112.857	157.145	158.678	1.39x	1.41x	Local

Escena	Config.	Local T8	Cluster T8	Grupo T8	Cluster/Local	Grupo/Local	Mas rapido
scene_many.txt	small	1.251	2.380	3.341	1.90x	2.67x	Local
scene_many.txt	medium	22.855	37.521	25.485	1.64x	1.12x	Local
scene_many.txt	large	182.725	298.016	215.079	1.63x	1.18x	Local

Numba p=8

Escena	Config.	Local T8	Cluster T8	Grupo T8	Cluster/Local	Grupo/Local	Mas rapido
scene.txt	small	0.659	1.084	0.867	1.65x	1.32x	Local
scene.txt	medium	12.602	16.977	11.929	1.35x	0.95x	Grupo
scene.txt	large	102.854	135.203	169.868	1.31x	1.65x	Local
scene_many.txt	small	1.298	2.013	3.202	1.55x	2.47x	Local
scene_many.txt	medium	23.217	31.523	25.598	1.36x	1.10x	Local
scene_many.txt	large	185.490	252.487	183.132	1.36x	0.99x	Grupo

El equipo Local (i7-9750H movil) y el equipo Grupo (i5-10600K desktop) tienen ambos 6 cores fisicos y 12 threads logicos, pero las medidas no muestran una ventaja consistente de uno sobre otro. El Grupo gana en configs pequenas/medias de `scene.txt` (mayor frecuencia base, 4.10 GHz) pero pierde en `scene.txt large` y `scene_many.txt small`. El Cluster Xeon Silver 4210R queda detras en p=8 porque la comparacion limita el paralelismo a 8 (su ventaja real esta en aprovechar sus 20 cores fisicos), tiene menor frecuencia base y sufre contencion compartida con otros usuarios del nodo.

Computador adicional del grupo

Los datos siguientes provienen de la corrida `--full --reps 1` ejecutada por jean-Philipe Fuentes en su equipo desktop. Carpeta original con CSV, hardware, plots y report: `results/full_grupo_jeanf_20260514_105212/`.

Hardware

Dato	Valor
Integrante	jean-Philipe Fuentes (jhfuentes@uc.cl)
Procesador	Intel(R) Core(TM) i5-10600K CPU @ 4.10GHz
Cores fisicos	6
Threads logicos	12
Threads por core	2
Frecuencia / max MHz	4100 MHz base (max turbo 4800 MHz, no reportado por WSL)
Sistema operativo	Ubuntu 24.04.2 LTS sobre WSL2 (kernel 5.15.167.4)

Resultados seriales (Grupo)

Escena	Config.	serial.cpp	serial_pt.cpp
scene.txt	small	0.090	5.025
scene.txt	medium	1.775	73.510
scene.txt	large	6.486	621.377
scene_many.txt	small	0.357	7.216
scene_many.txt	medium	4.204	121.263
scene_many.txt	large	17.734	926.600

Crecimientos serial_pt observados en el equipo Grupo:

Escena	small -> medium	medium -> large
scene.txt	14.63x	8.45x
scene_many.txt	16.81x	7.64x

Coincide con lo esperado: ~16x cuando se cuadruplica resolucion y N pasa de 32 a 128 (small->medium); ~8x cuando solo se duplica N pero la resolucion vuelve a crecer 4x (medium->large). Los factores ligeramente bajo 16x se explican por terminacion anticipada del path tracer y efectos de cache.

OpenMP y Numba p=8 (Grupo)

Schedule elegido por el script para la fase compare: `dynamic:8` para `scene.txt`, `dynamic:32` para `scene_many.txt` (mismas elecciones que el equipo Local).

Escena	Config.	OMP T8	OMP S8	OMP E8	Numba T8	Numba S8	Numba E8	Mejor
scene.txt	small	0.72	6.98	0.87	0.87	5.80	0.72	OpenMP
scene.txt	medium	13.32	5.52	0.69	11.93	6.16	0.77	Numba
scene.txt	large	158.68	3.92	0.49	169.87	3.66	0.46	OpenMP
scene_many.txt	small	3.34	2.16	0.27	3.20	2.25	0.28	Numba
scene_many.txt	medium	25.49	4.76	0.59	25.60	4.74	0.59	OpenMP
scene_many.txt	large	215.08	4.31	0.54	183.13	5.06	0.63	Numba

OpenMP y Numba tienen rendimiento muy parejo: el ganador alterna entre configs. Esto era esperable porque ambos comparten kernel C-equivalente (Numba via LLVM, OpenMP via g++ con `-O2`), comparten el mismo patron de paralelizacion (loop sobre pixeles) y semillas locales por pixel. La eficiencia E8 cae notoriamente en `small` (donde el overhead de fork/join domina sobre el trabajo) y en `large` (donde el 7mo y 8vo thread caen en cores hyperthreaded del i5-10600K, que solo tiene 6 cores fisicos: SMT no duplica throughput porque ambos hilos comparten unidades funcionales).

Comparacion con Local y Cluster

- **Diferencias mas notables**:
  - En la fase serial, el equipo Grupo (i5-10600K @ 4.10 GHz desktop) fue ~14-24% mas lento que el equipo Local (i7-9750H @ 2.60 GHz movil), pese a tener mayor frecuencia base y misma cantidad de cores fisicos. Era el resultado contrario al esperado por especificaciones.
  - El Cluster (Xeon Silver 4210R @ 2.40 GHz) es ~52-85% mas lento que el equipo Local en serial: mayor latencia por core, contencion del nodo y menor turbo.
  - En p=8 los tres computadores estan mas cerca, pero el Grupo sigue siendo competitivo solo en configs medianas; en `large` y en `scene_many small` queda detras del Local (hasta 2.67x mas lento en `scene_many small` OpenMP).
- **Posibles causas**:
  - Ejecucion bajo WSL2 introduce overhead de virtualizacion (capa Hyper-V) que penaliza el rendimiento serial.
  - Durante la corrida habia procesos concurrentes (Claude Code, IDE, otros agentes) consumiendo CPU: la columna "CPU max MHz" no fue reportada por `lscpu` dentro de WSL, lo que sugiere que las politicas de frecuencia no son visibles desde el guest y el turbo puede estar limitado.
  - El i7-9750H del equipo Local probablemente corrio bajo Linux nativo o WSL con menos contencion, aprovechando mejor su turbo de hasta 4.50 GHz.
- **Efecto de cores/threads/frecuencia/cache**:
  - El Cluster tiene 20 cores fisicos / 40 threads, pero limitar p=8 desperdicia su capacidad: a p=20 deberia escalar mucho mejor.
  - El i5-10600K y el i7-9750H tienen ambos 6 cores fisicos, por lo que la curva de speedup ideal coincide; el techo real es ~6x porque el SMT del 7mo-12vo thread no agrega trabajo paralelo real.
  - La cache L3 del i5-10600K es 12 MiB, igual que el i7-9750H; el Cluster tiene 27.5 MiB de L3 pero compartidos entre 20 cores: por core efectivo es menor.
  - La escena `scene_many.txt` con 40 esferas amplifica las diferencias de cache y de frecuencia por core porque cada rayo evalua mas intersecciones; en el Cluster esto se traduce en 1.80x mas tiempo serial que el Local.

Graficos

Los graficos siguientes fueron generados por el mismo script desde cada `results.csv`.

Local

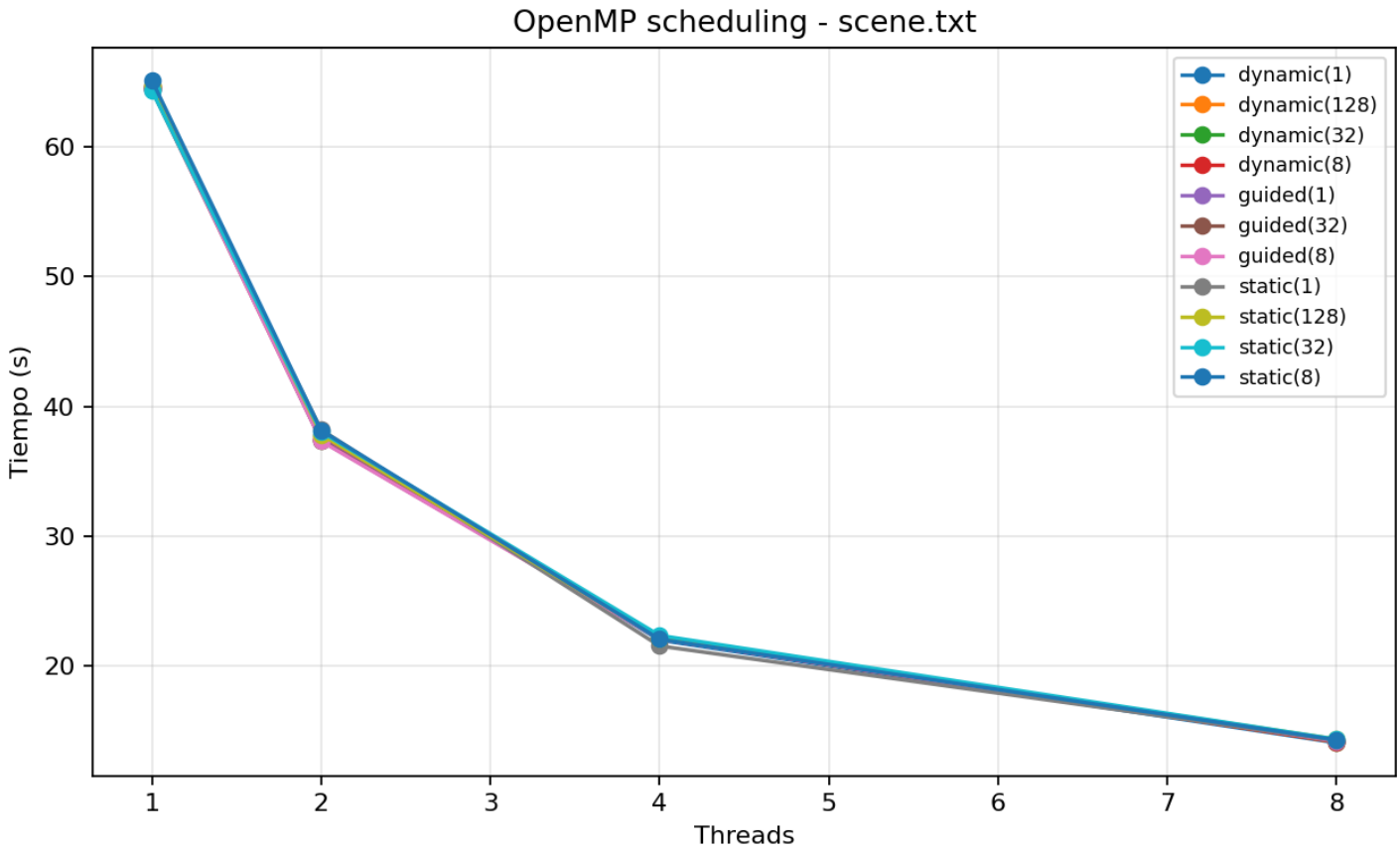

Figure 1: Local: Scheduling scene.txt

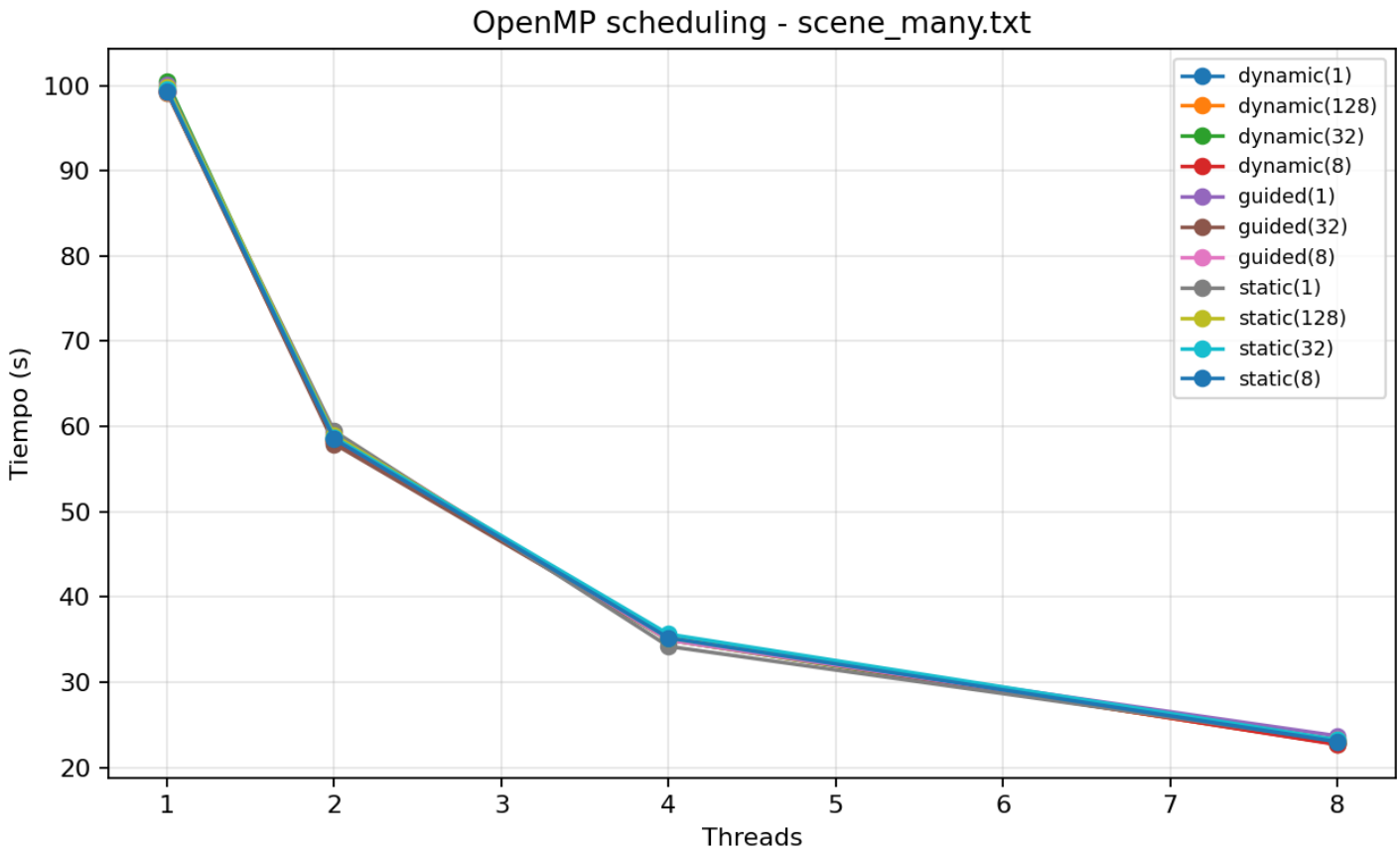

Figure 2: Local: Scheduling scene_many.txt

Local - scene.txt small

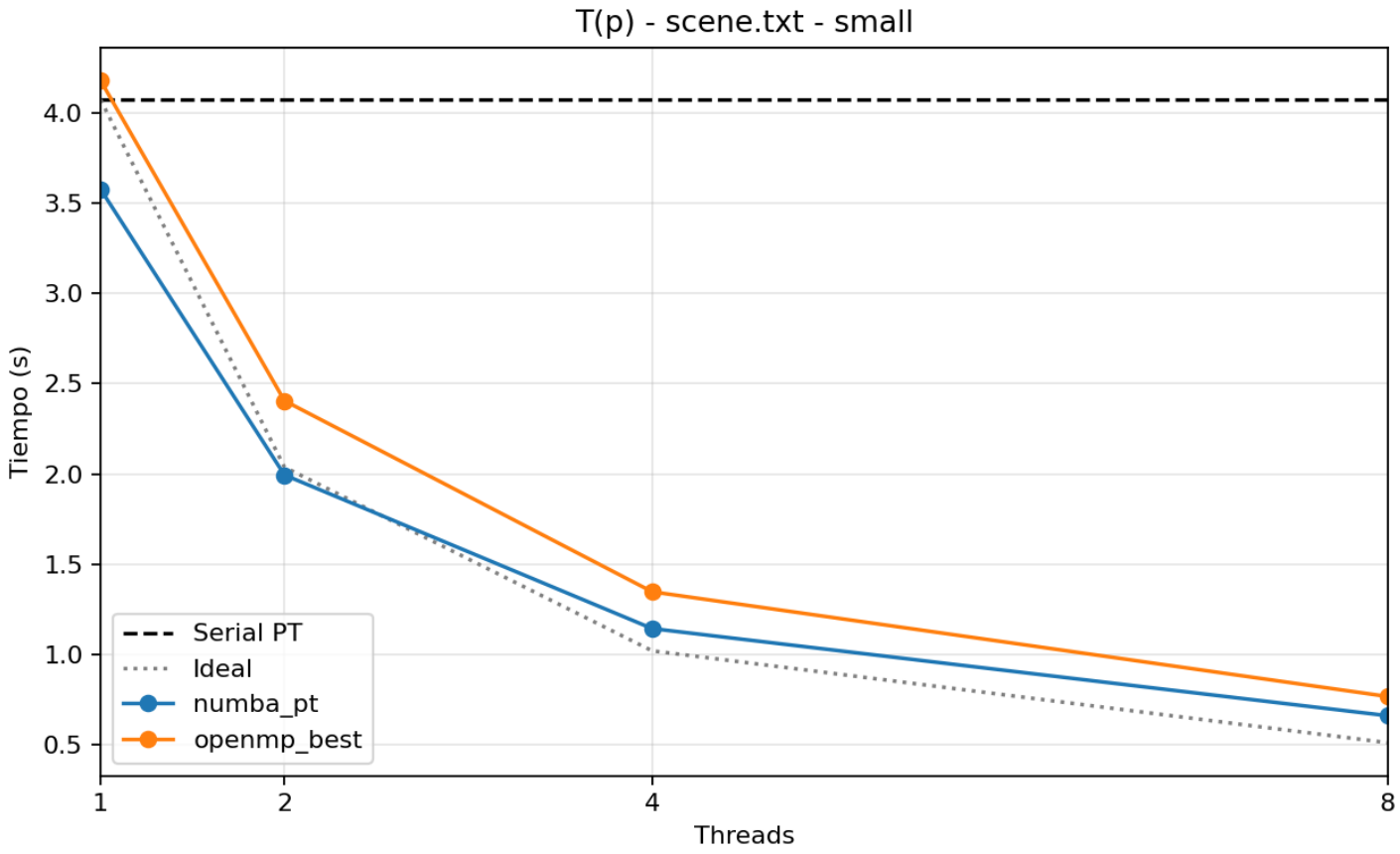

Figure 3: Local: Tiempo scene.txt small

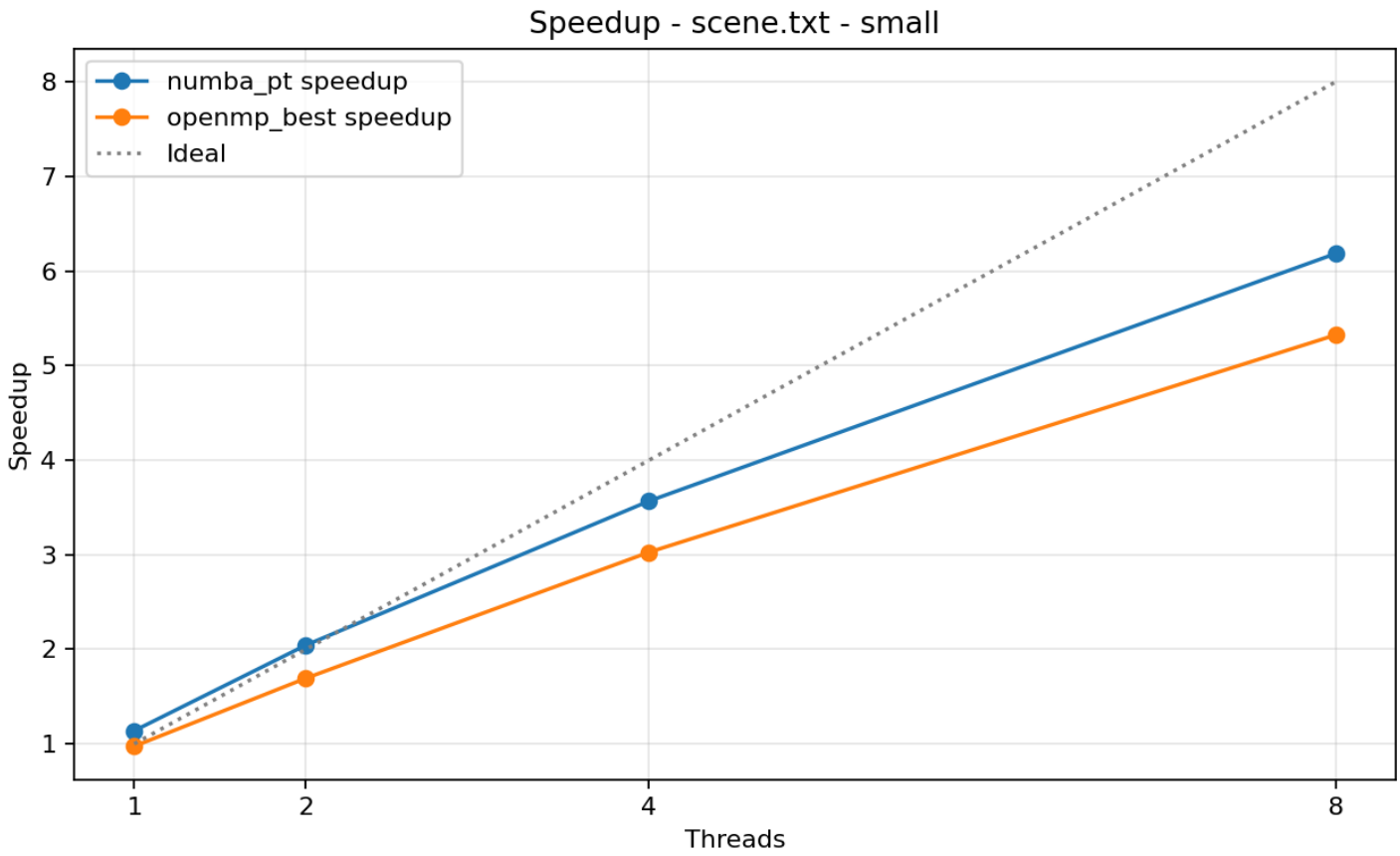

Figure 4: Local: Speedup scene.txt small

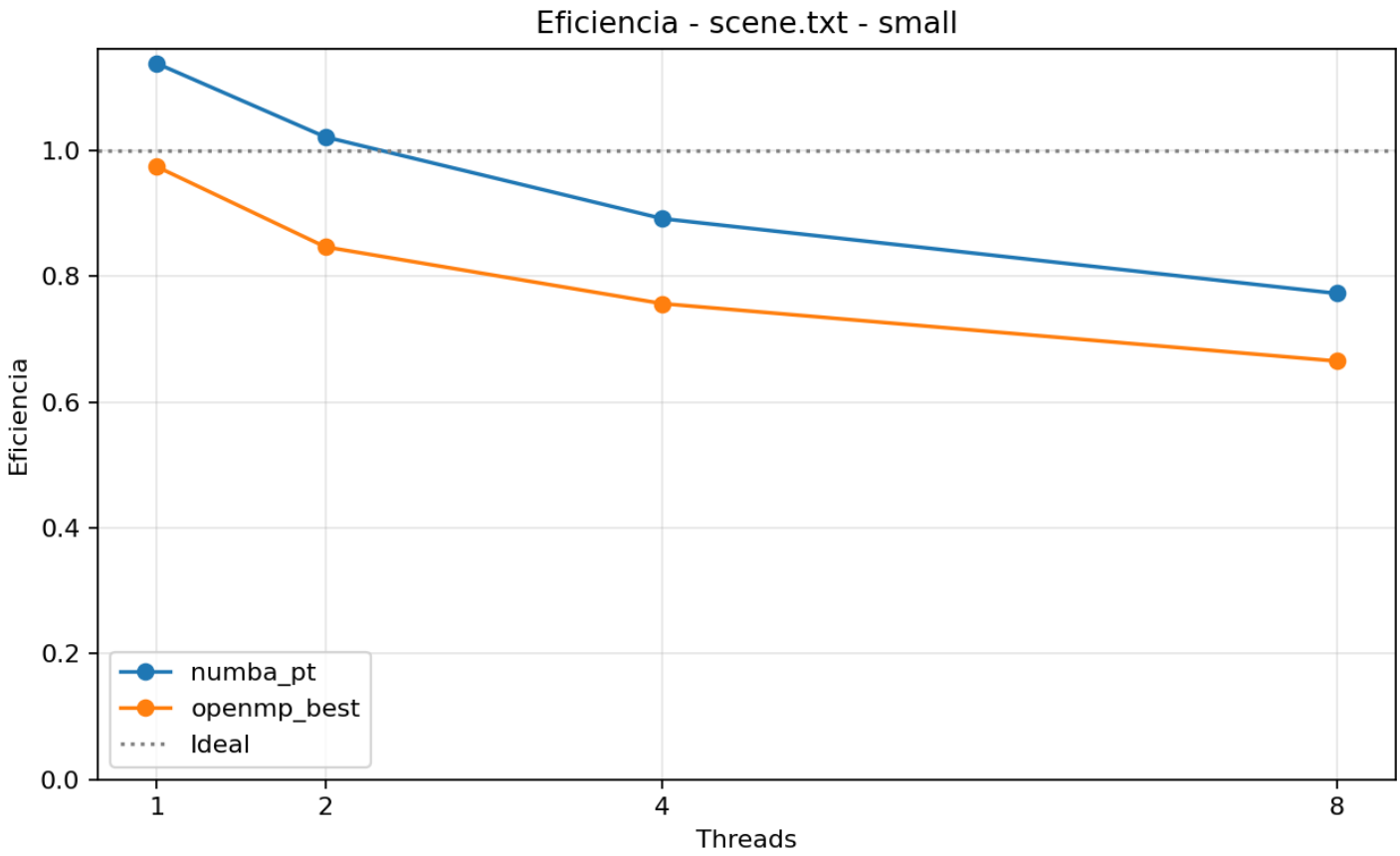

Figure 5: Local: Eficiencia scene.txt small

Local - scene.txt medium

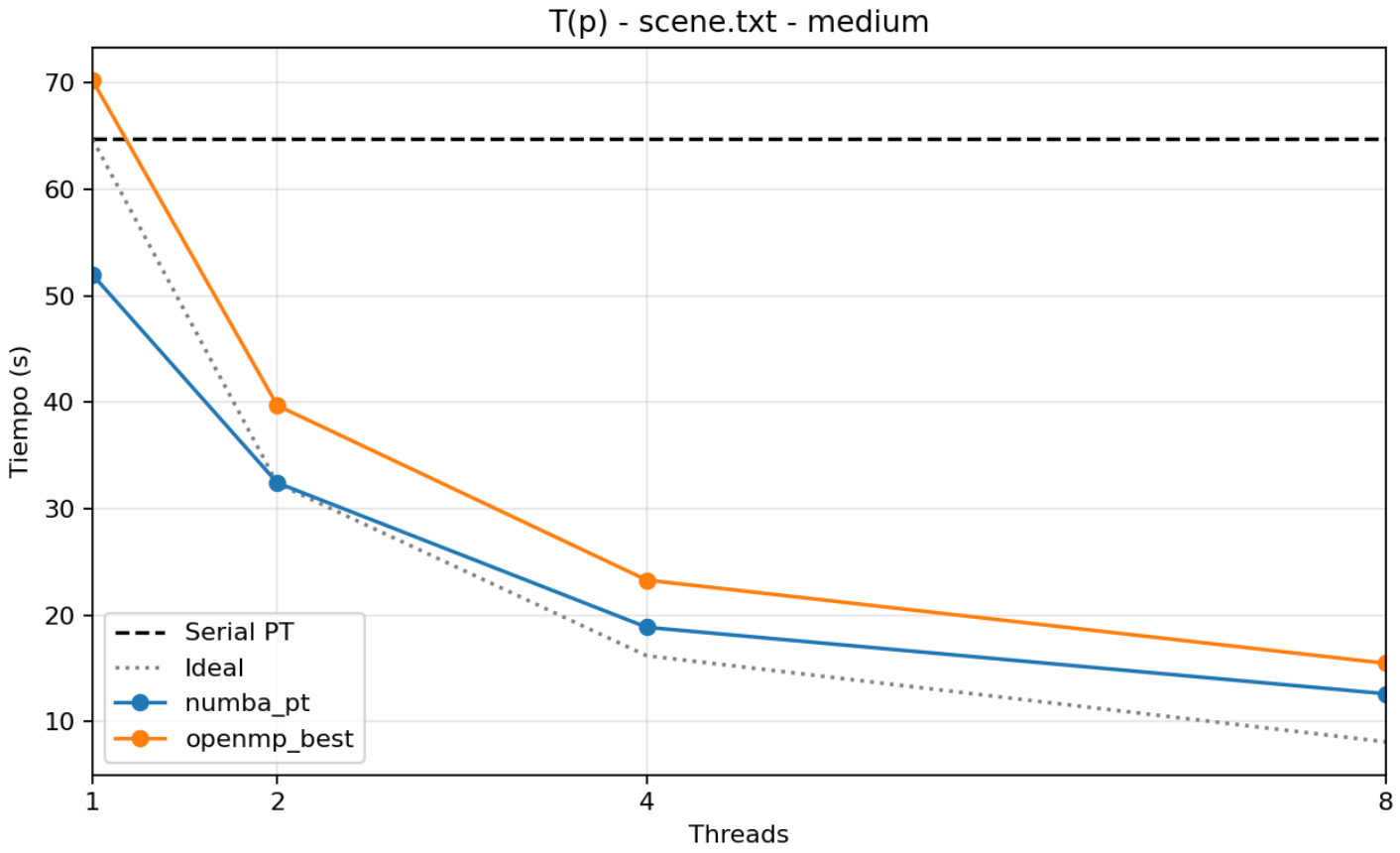

Figure 6: Local: Tiempo scene.txt medium

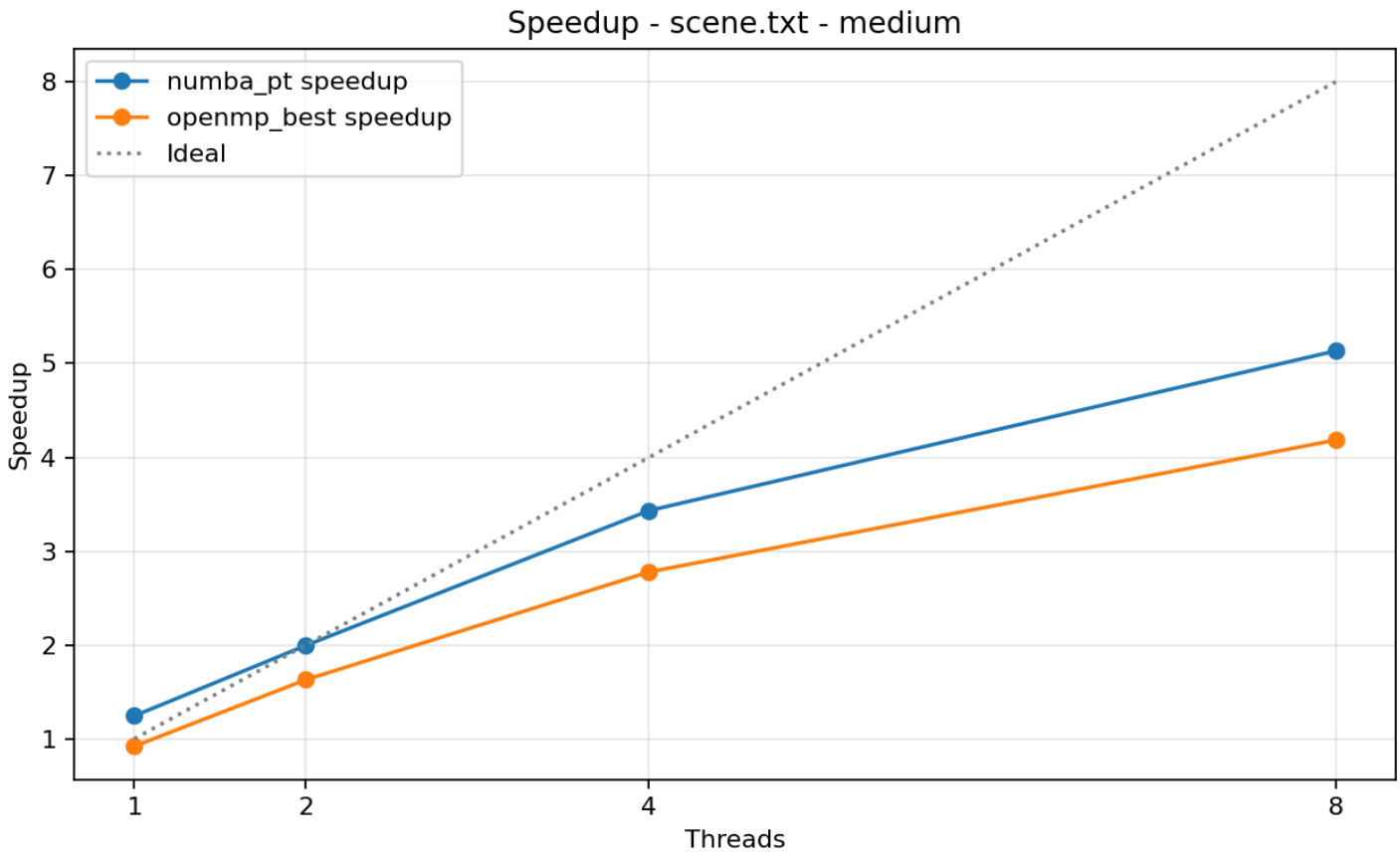

Figure 7: Local: Speedup scene.txt medium

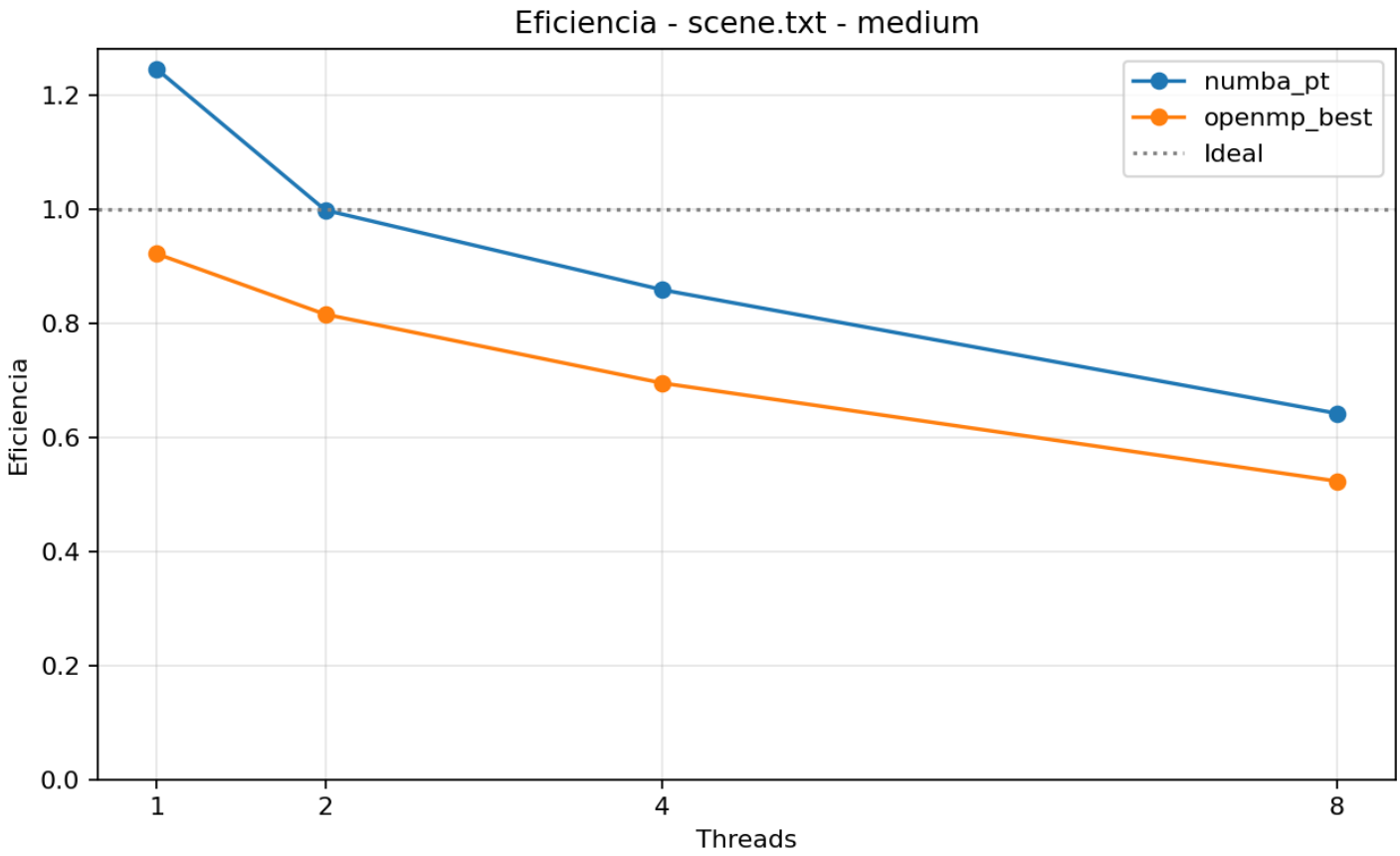

Figure 8: Local: Eficiencia scene.txt medium

Local - scene.txt large

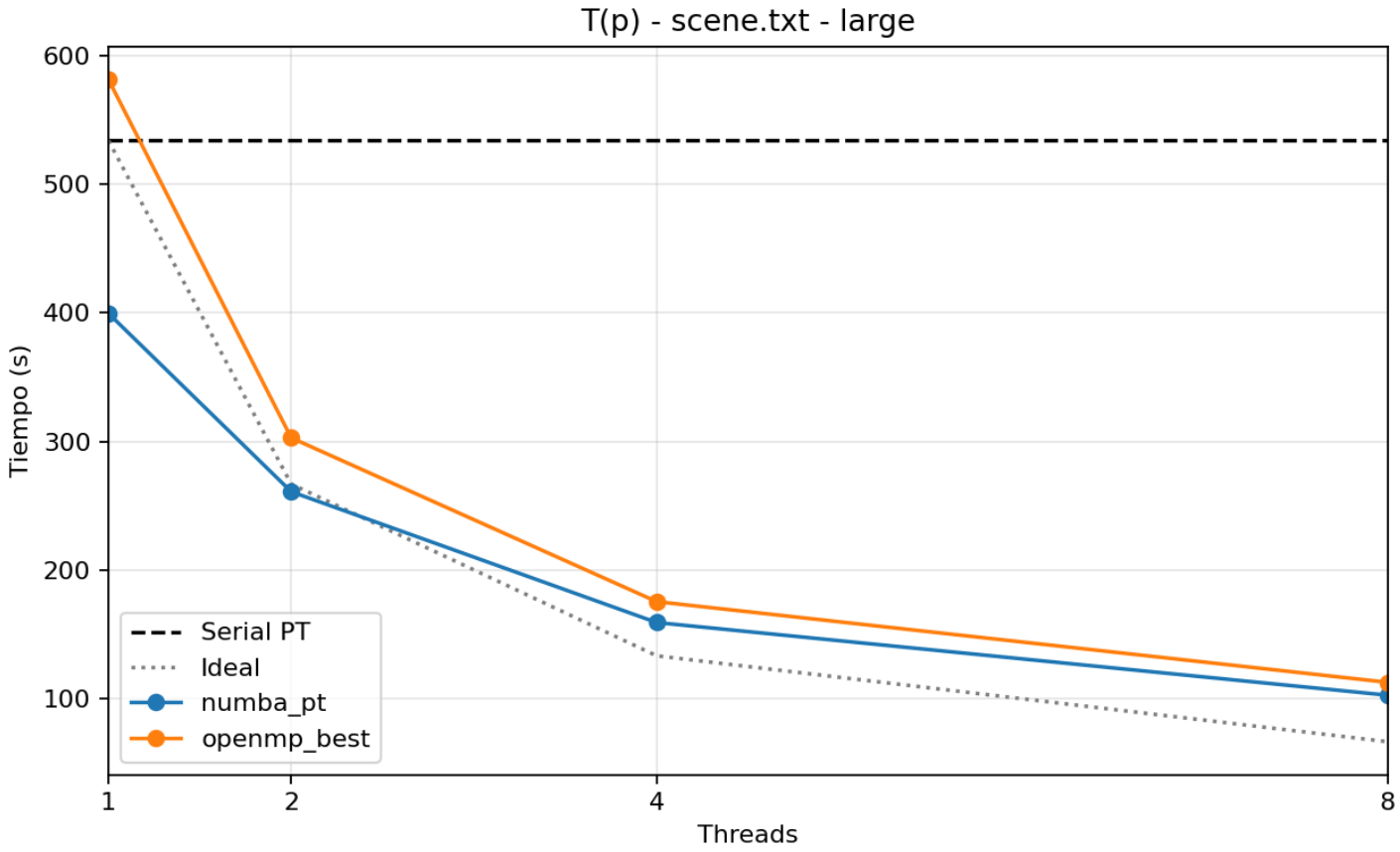

Figure 9: Local: Tiempo scene.txt large

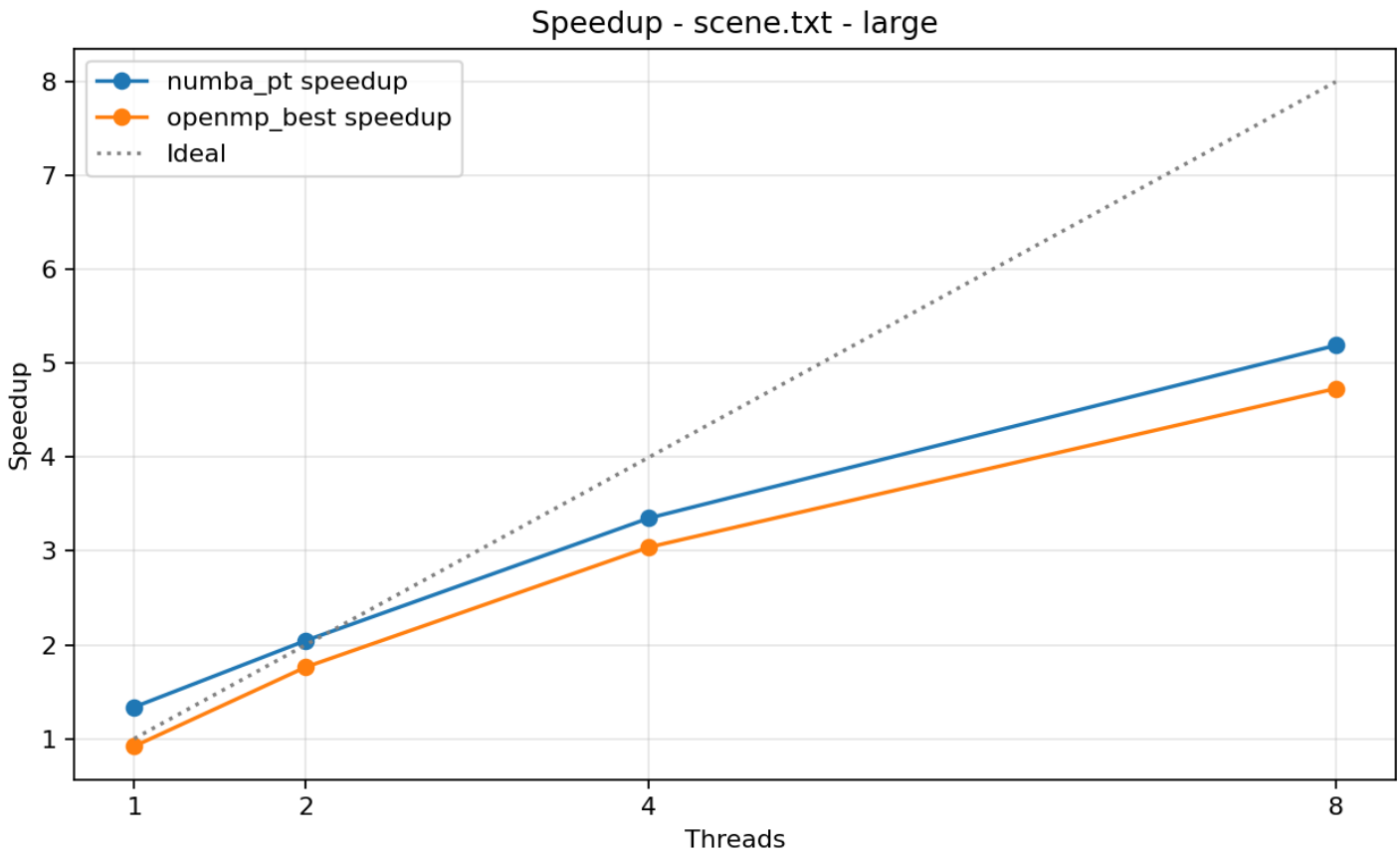

Figure 10: Local: Speedup scene.txt large

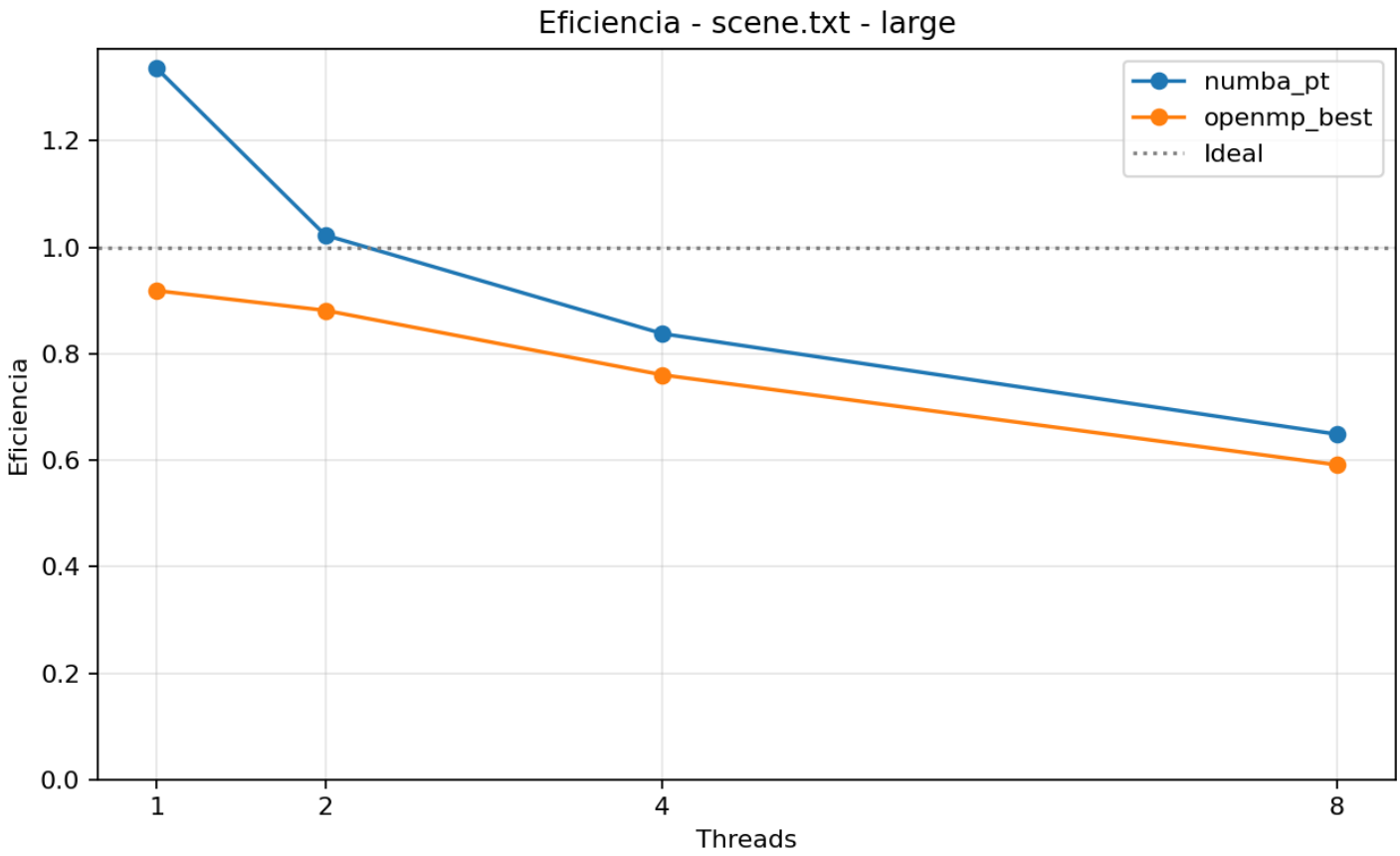

Figure 11: Local: Eficiencia scene.txt large

Local - scene_many.txt small

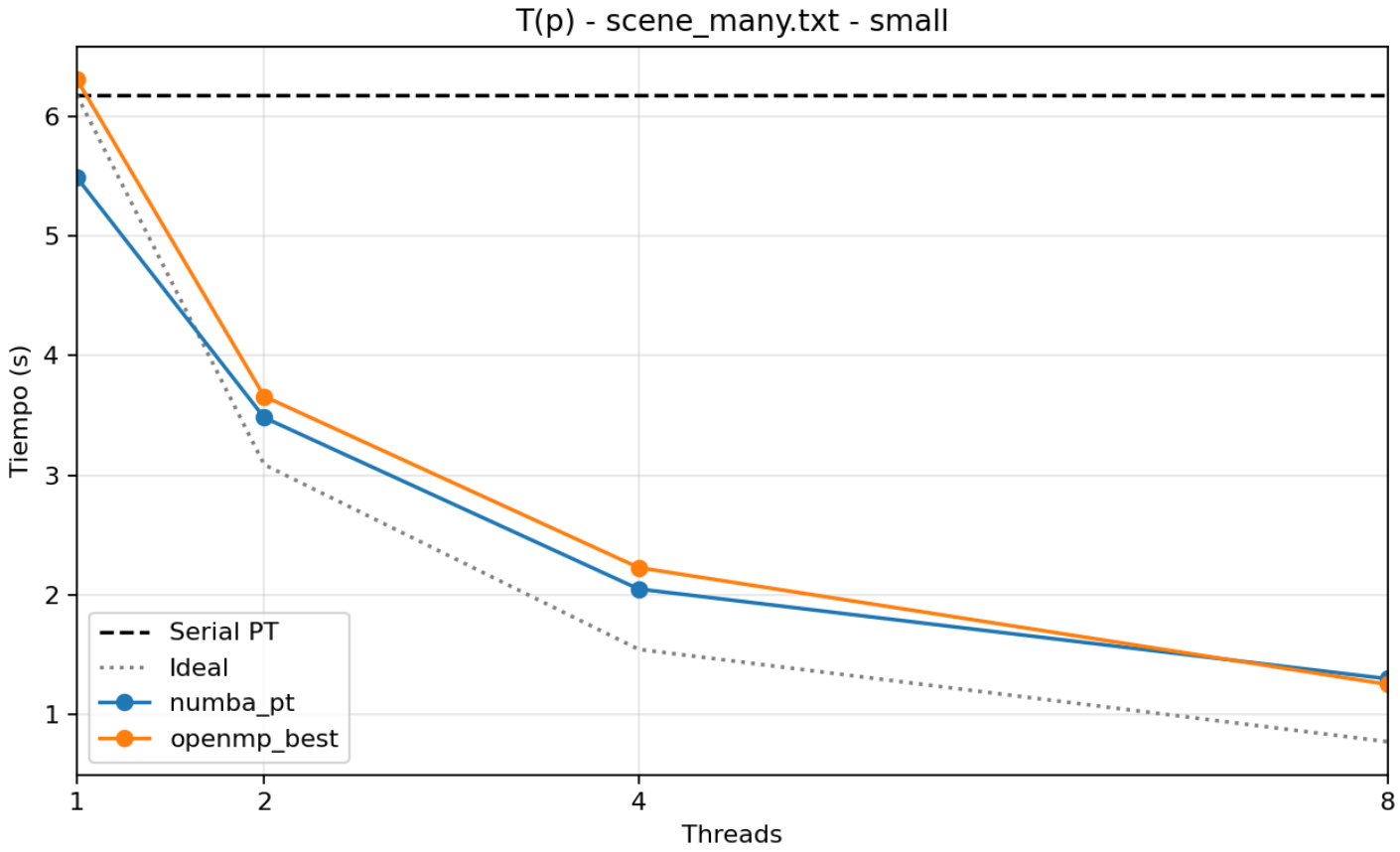

Figure 12: Local: Tiempo scene_many.txt small

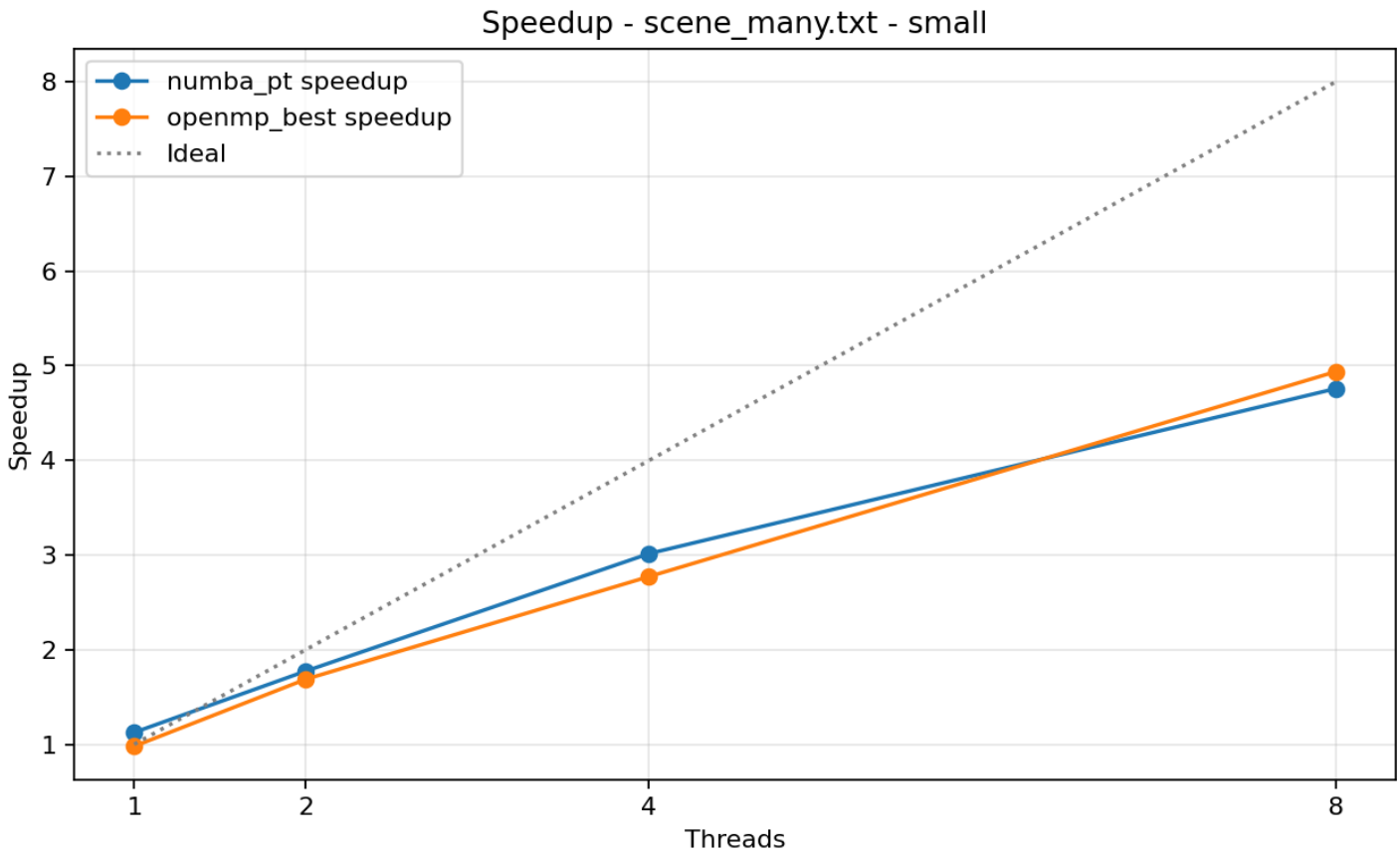

Figure 13: Local: Speedup scene_many.txt small

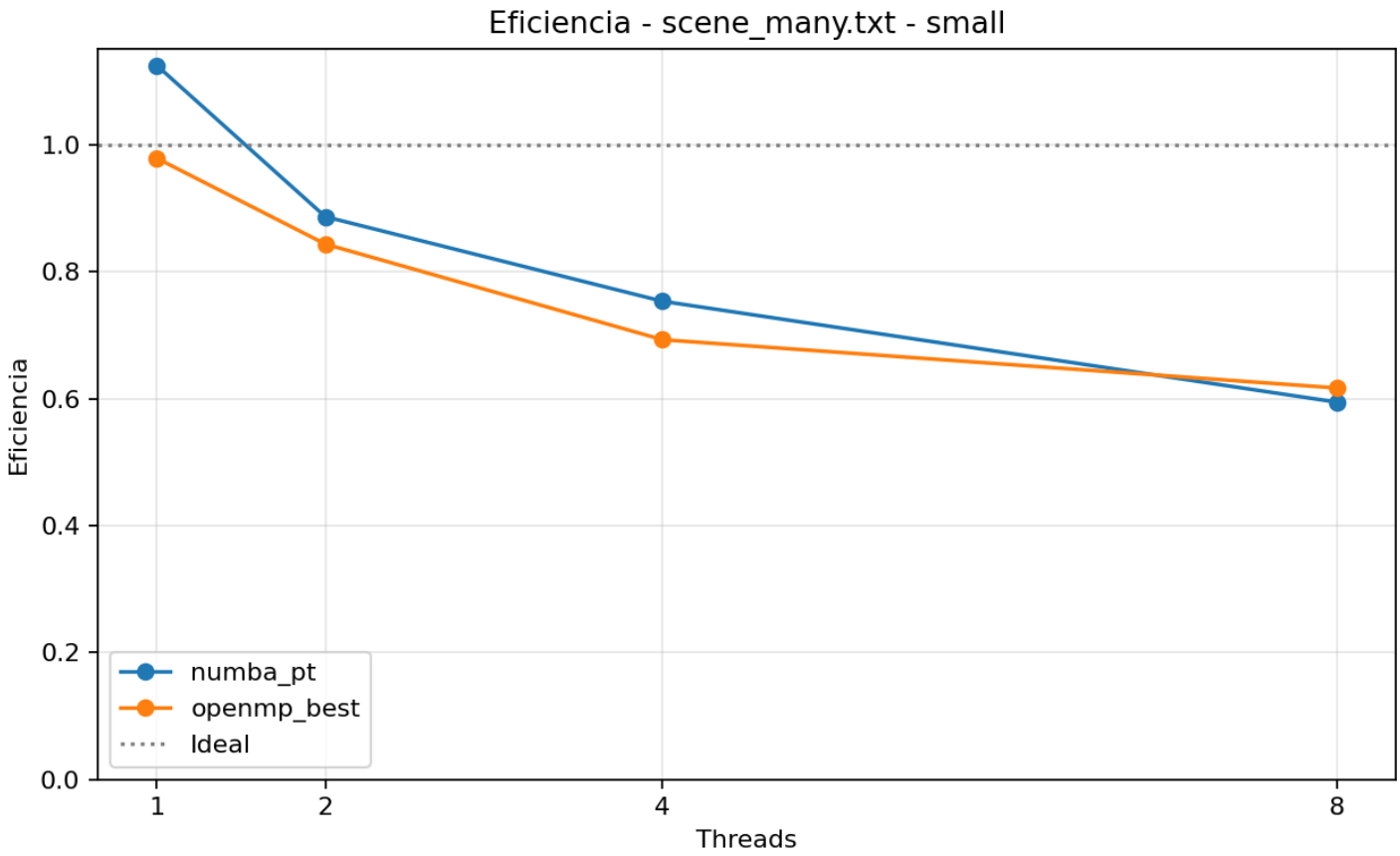

Figure 14: Local: Eficiencia scene_many.txt small

Local - scene_many.txt medium

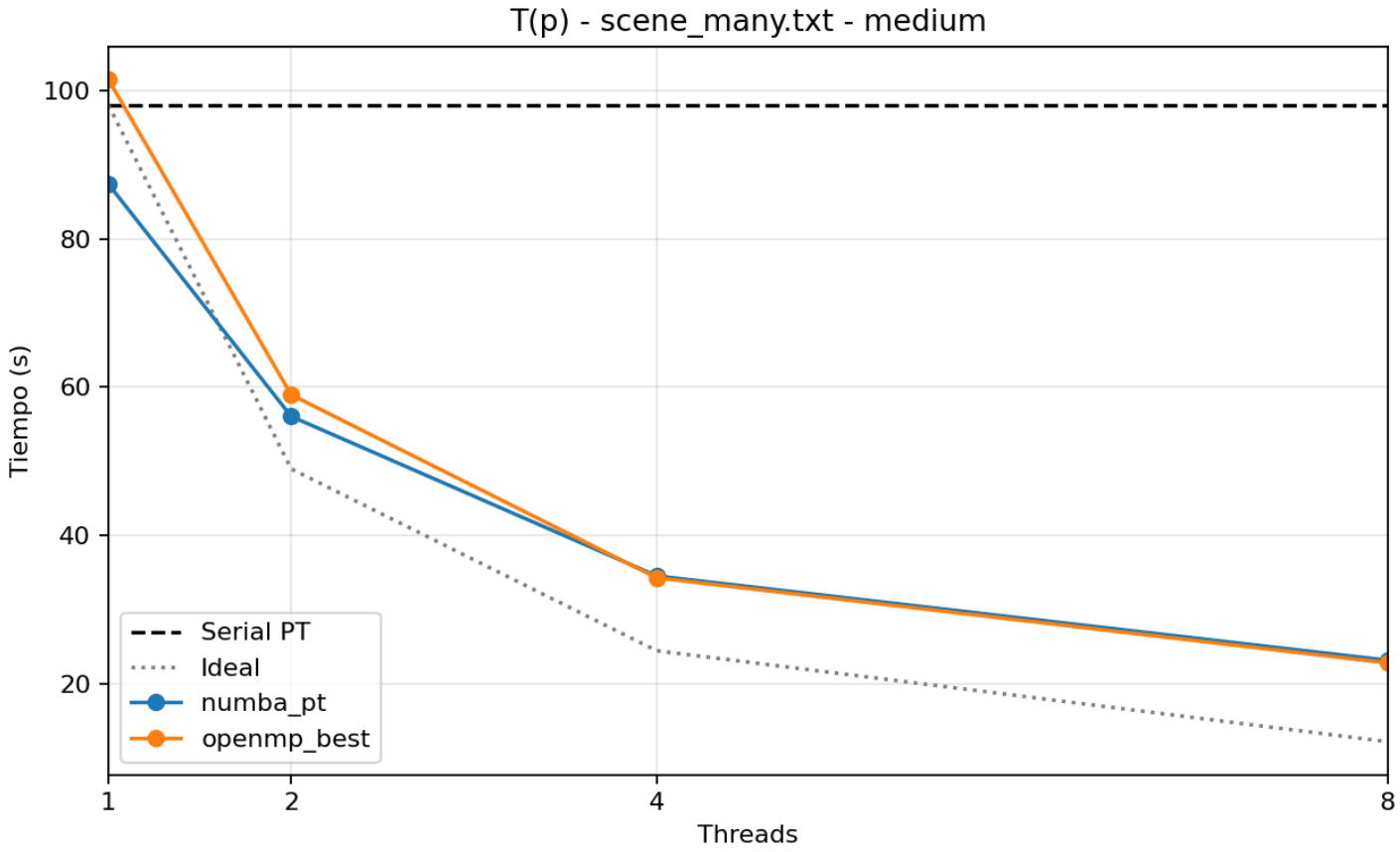

Figure 15: Local: Tiempo scene_many.txt medium

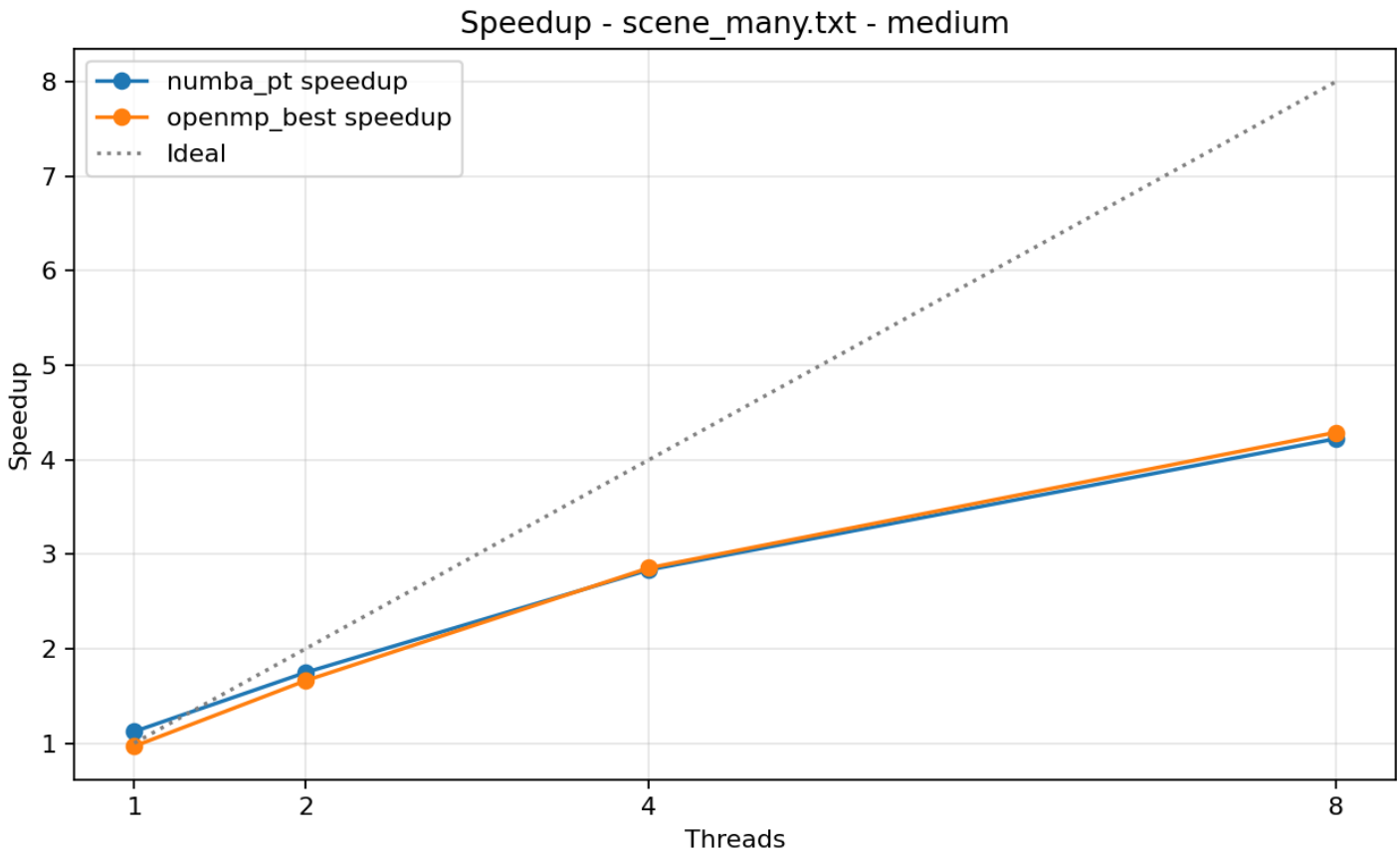

Figure 16: Local: Speedup scene_many.txt medium

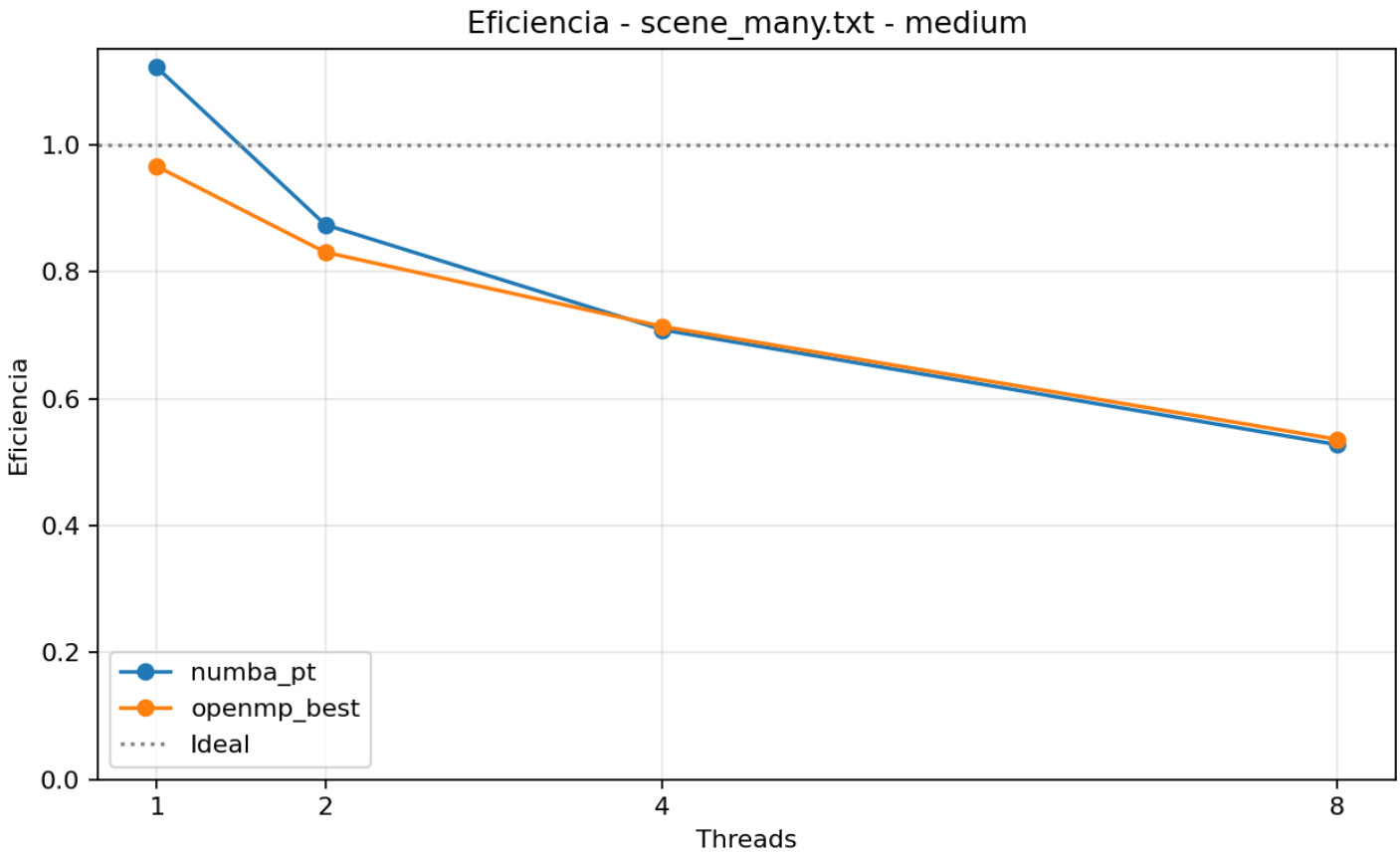

Figure 17: Local: Eficiencia scene_many.txt medium

Local - scene_many.txt large

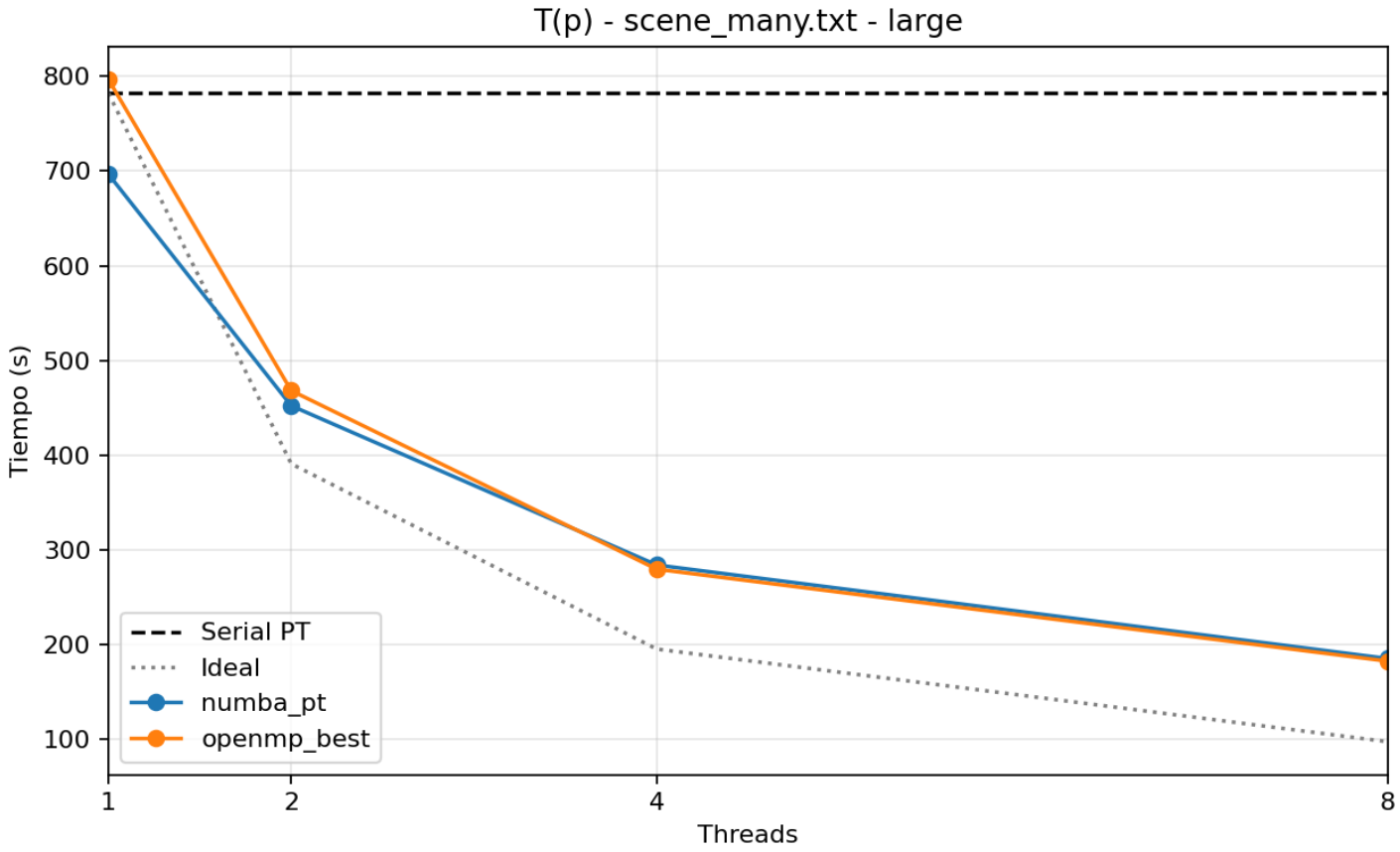

Figure 18: Local: Tiempo scene_many.txt large

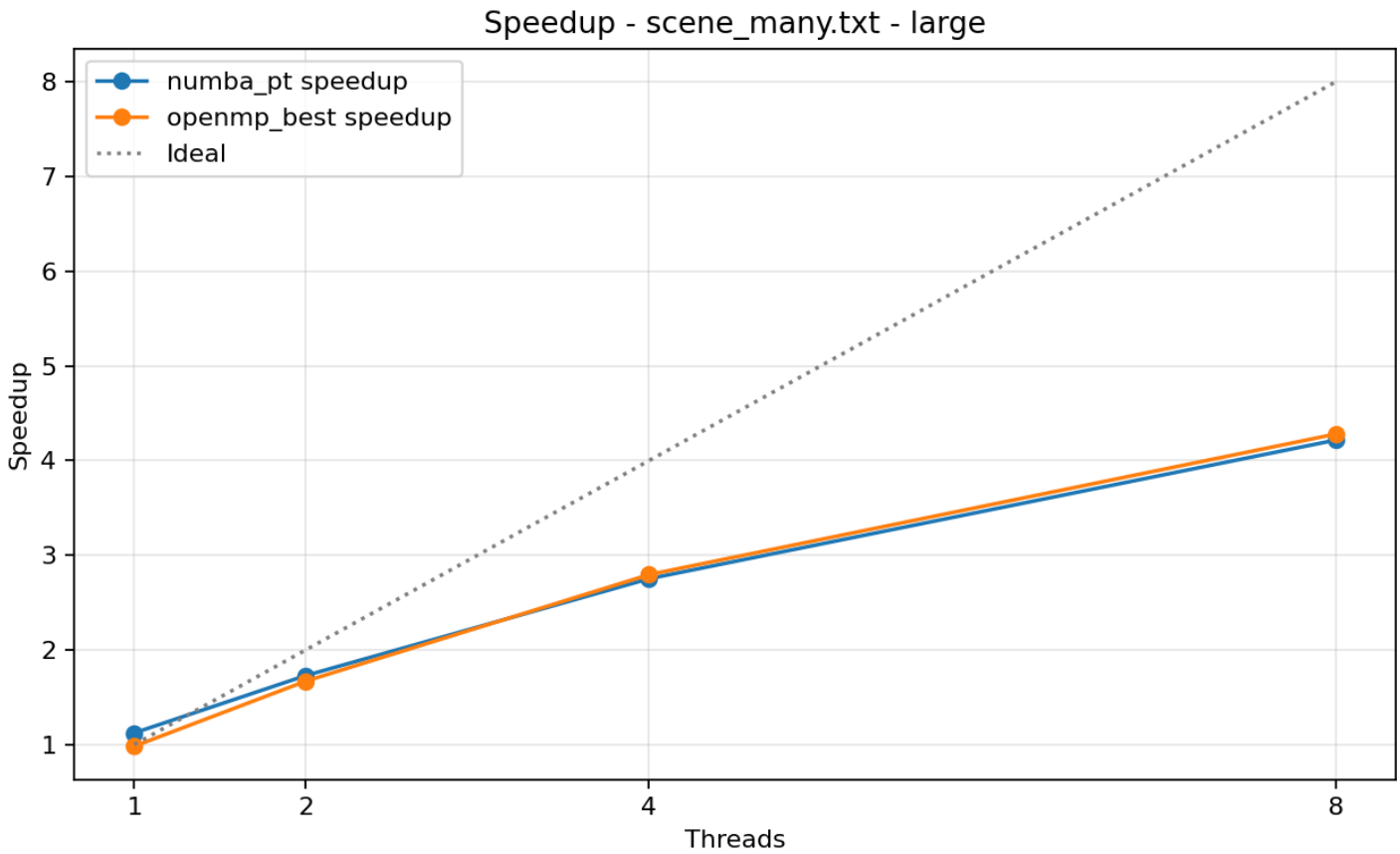

Figure 19: Local: Speedup scene_many.txt large

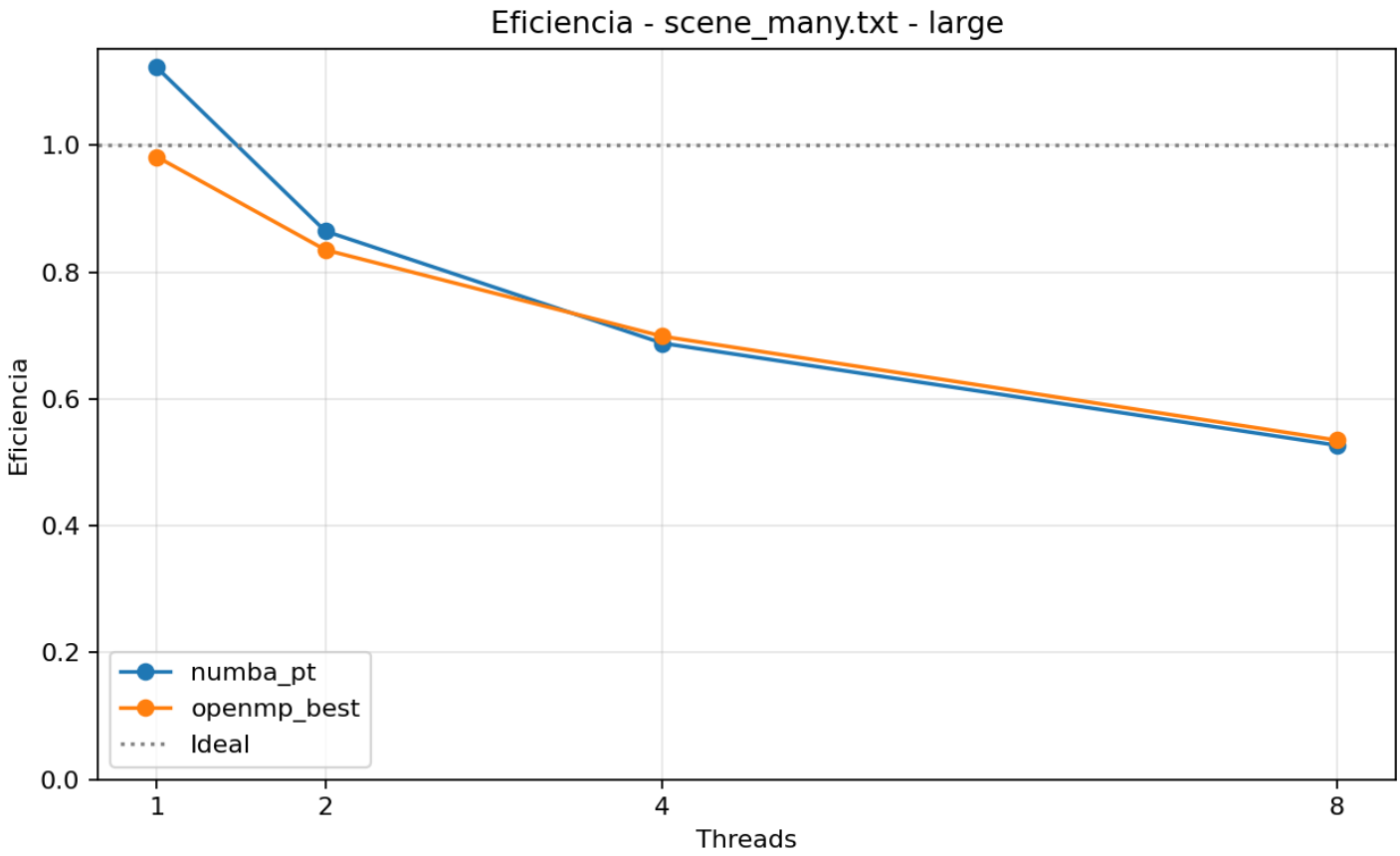

Figure 20: Local: Eficiencia scene_many.txt large

Cluster

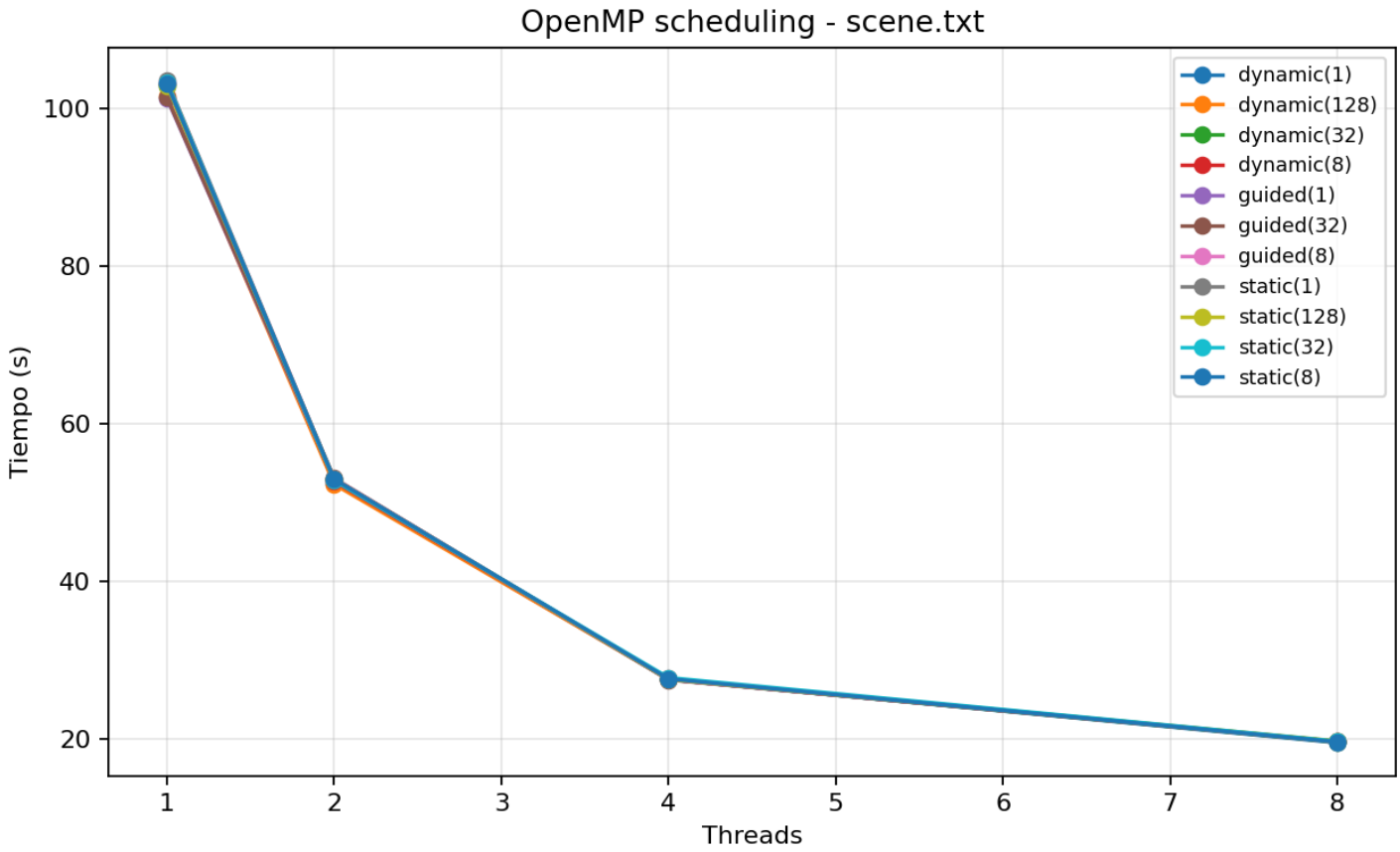

Figure 21: Cluster: Scheduling scene.txt

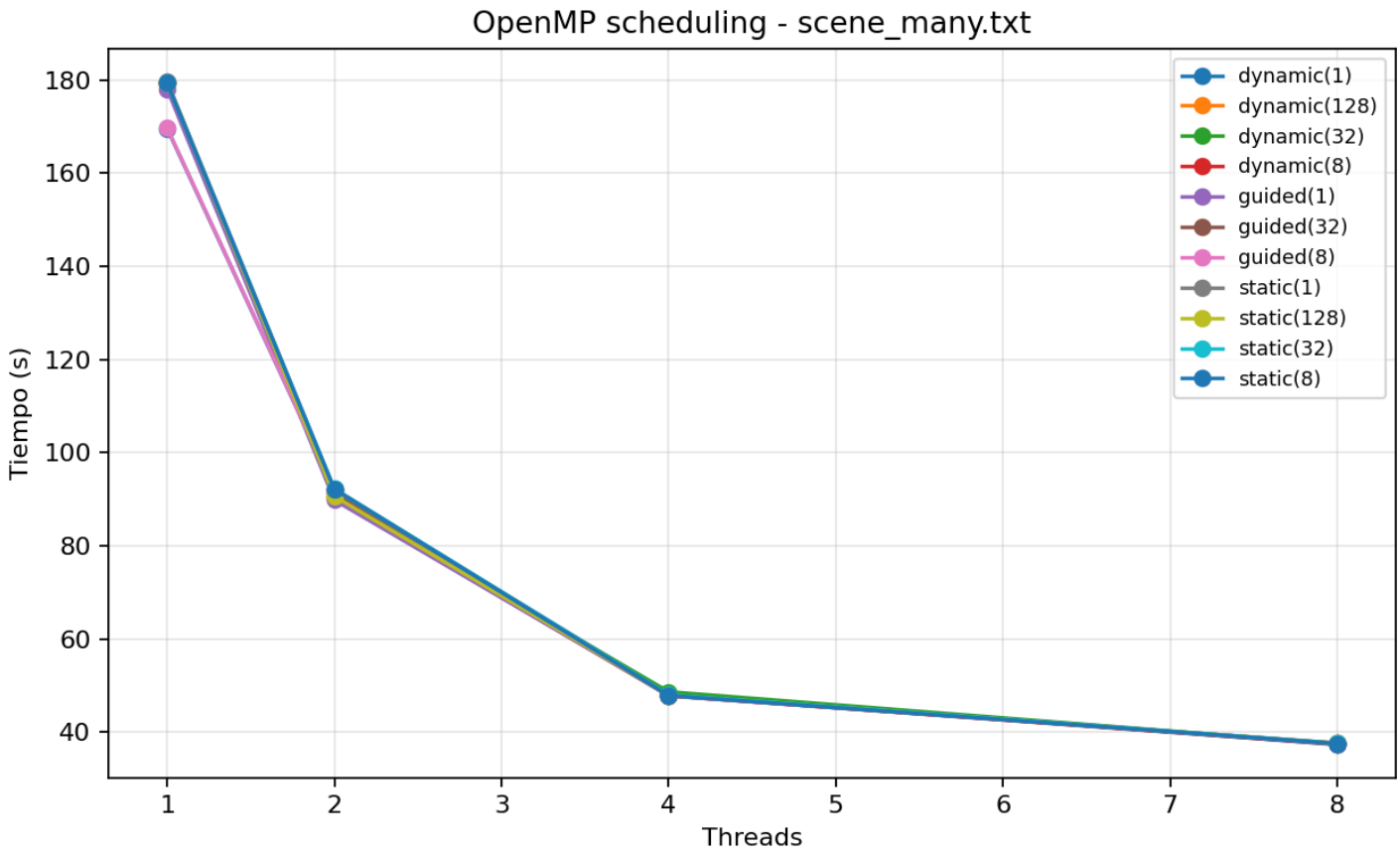

Figure 22: Cluster: Scheduling scene_many.txt

Cluster - scene.txt small

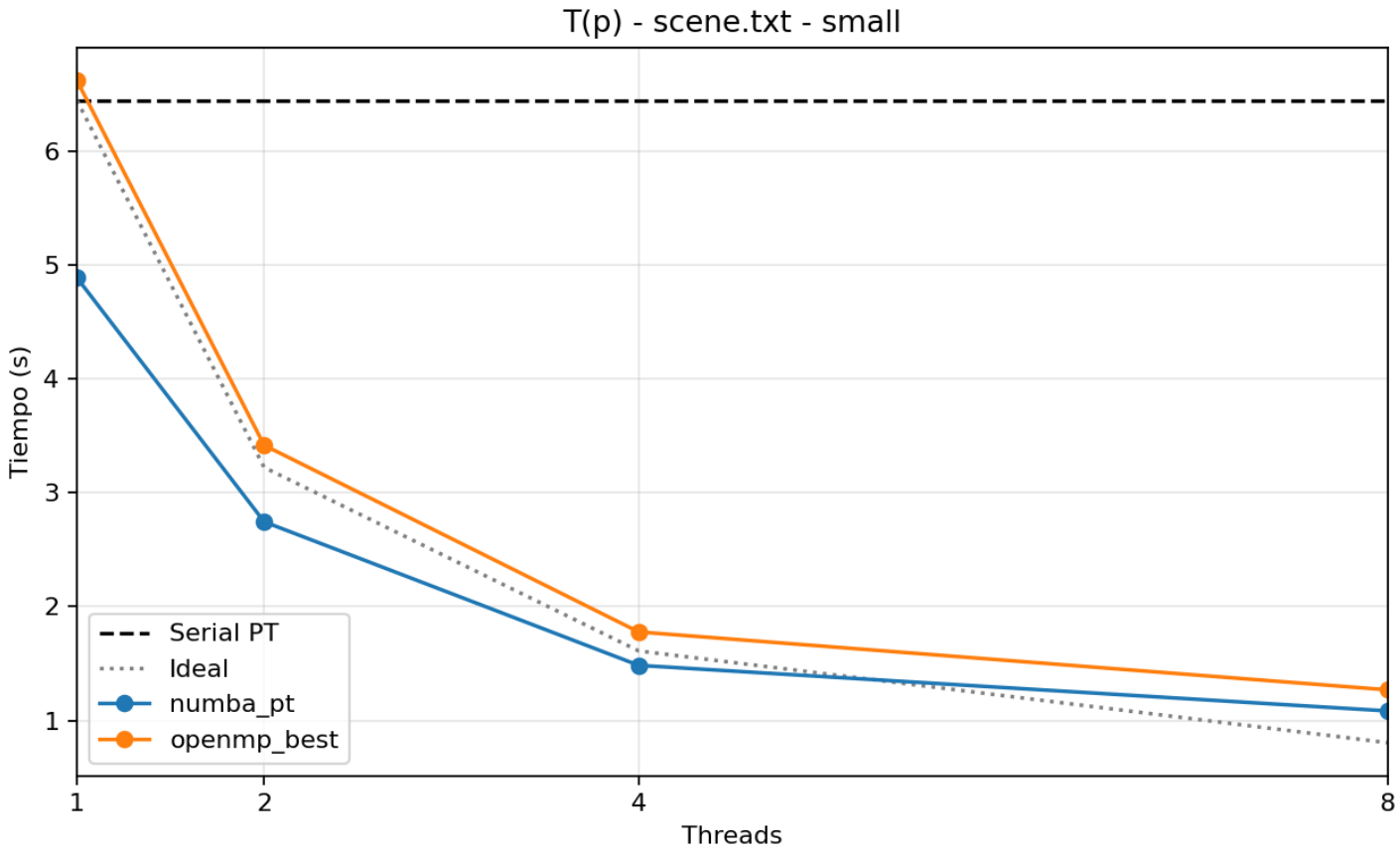

Figure 23: Cluster: Tiempo scene.txt small

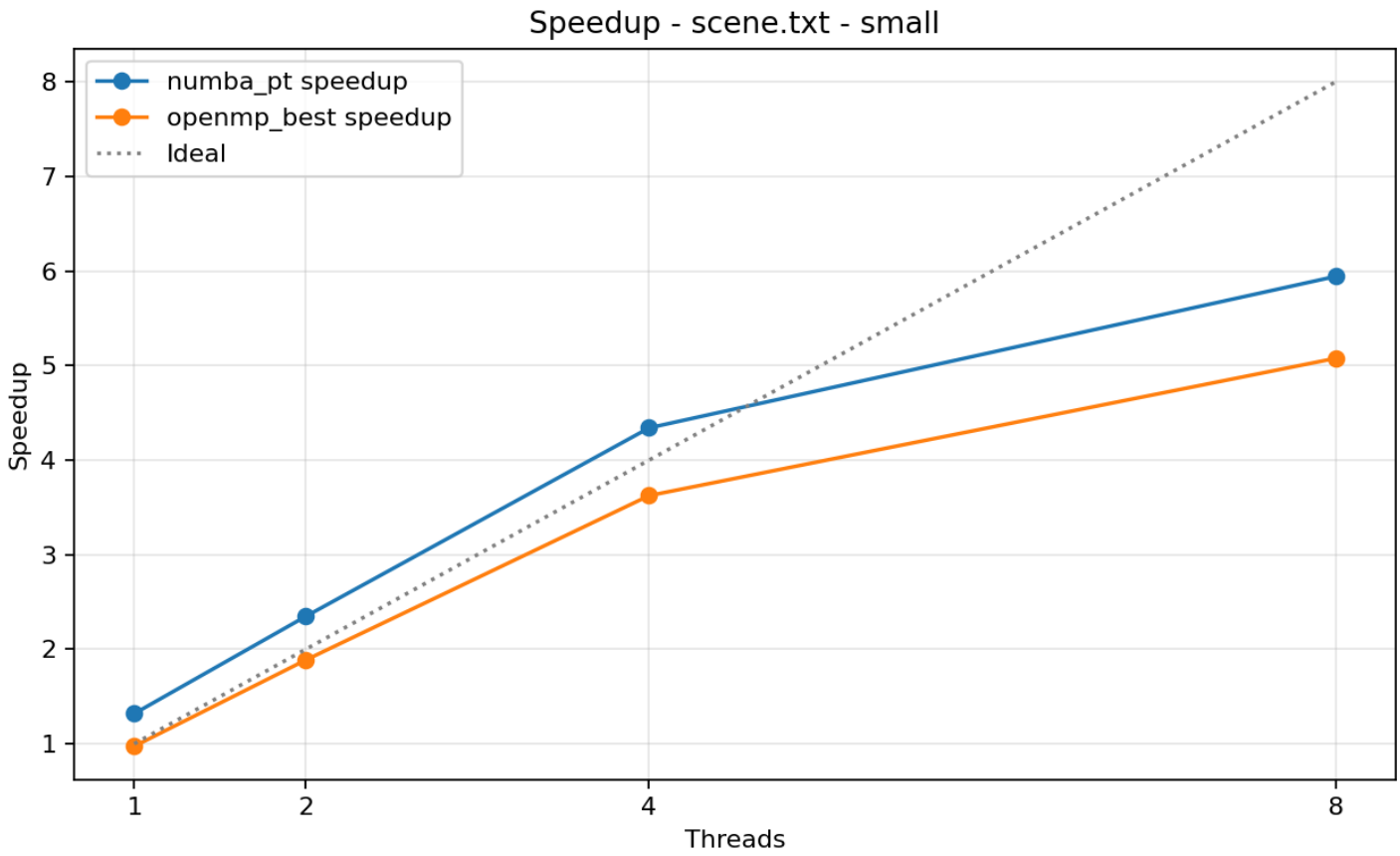

Figure 24: Cluster: Speedup scene.txt small

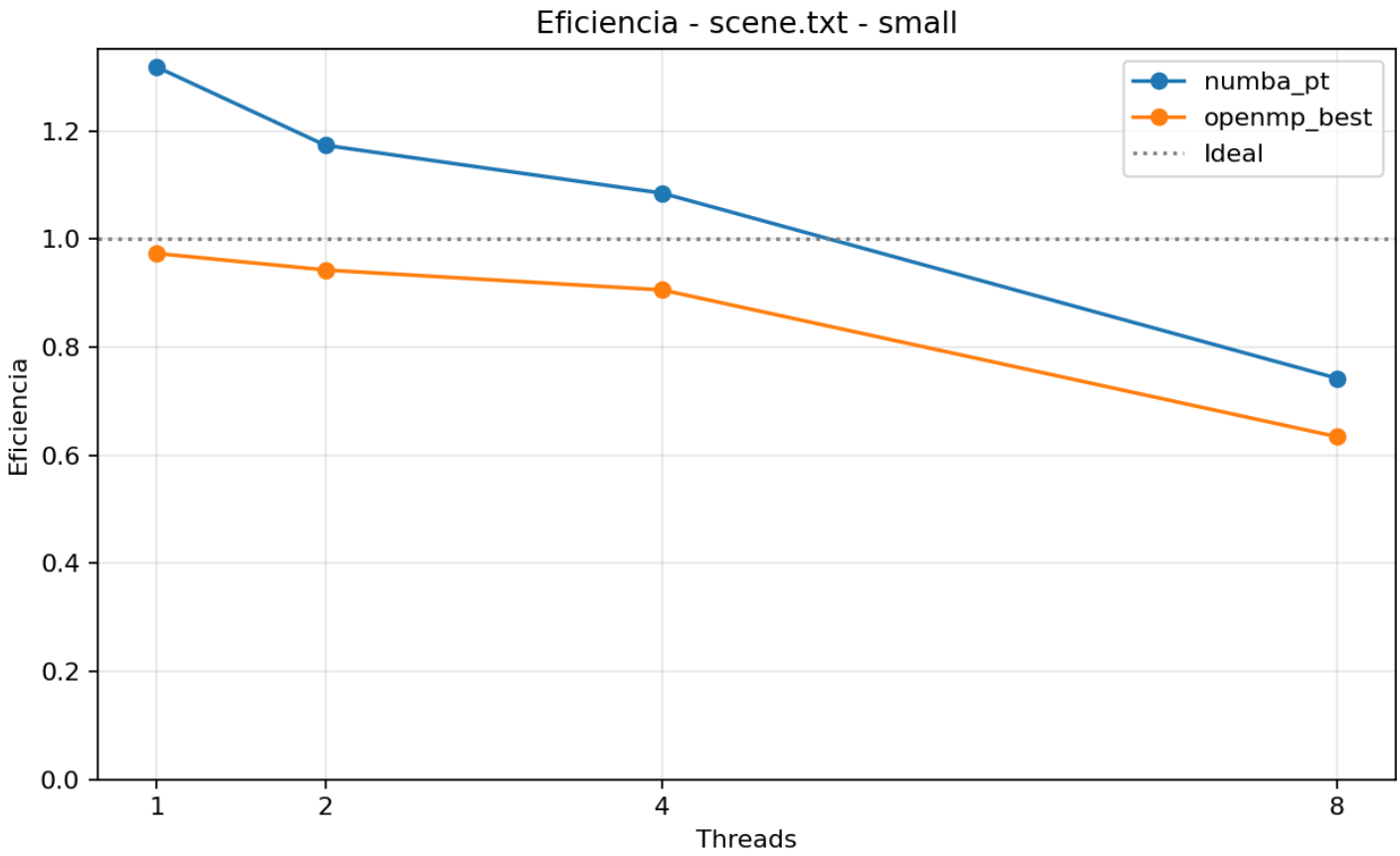

Figure 25: Cluster: Eficiencia scene.txt small

Cluster - scene.txt medium

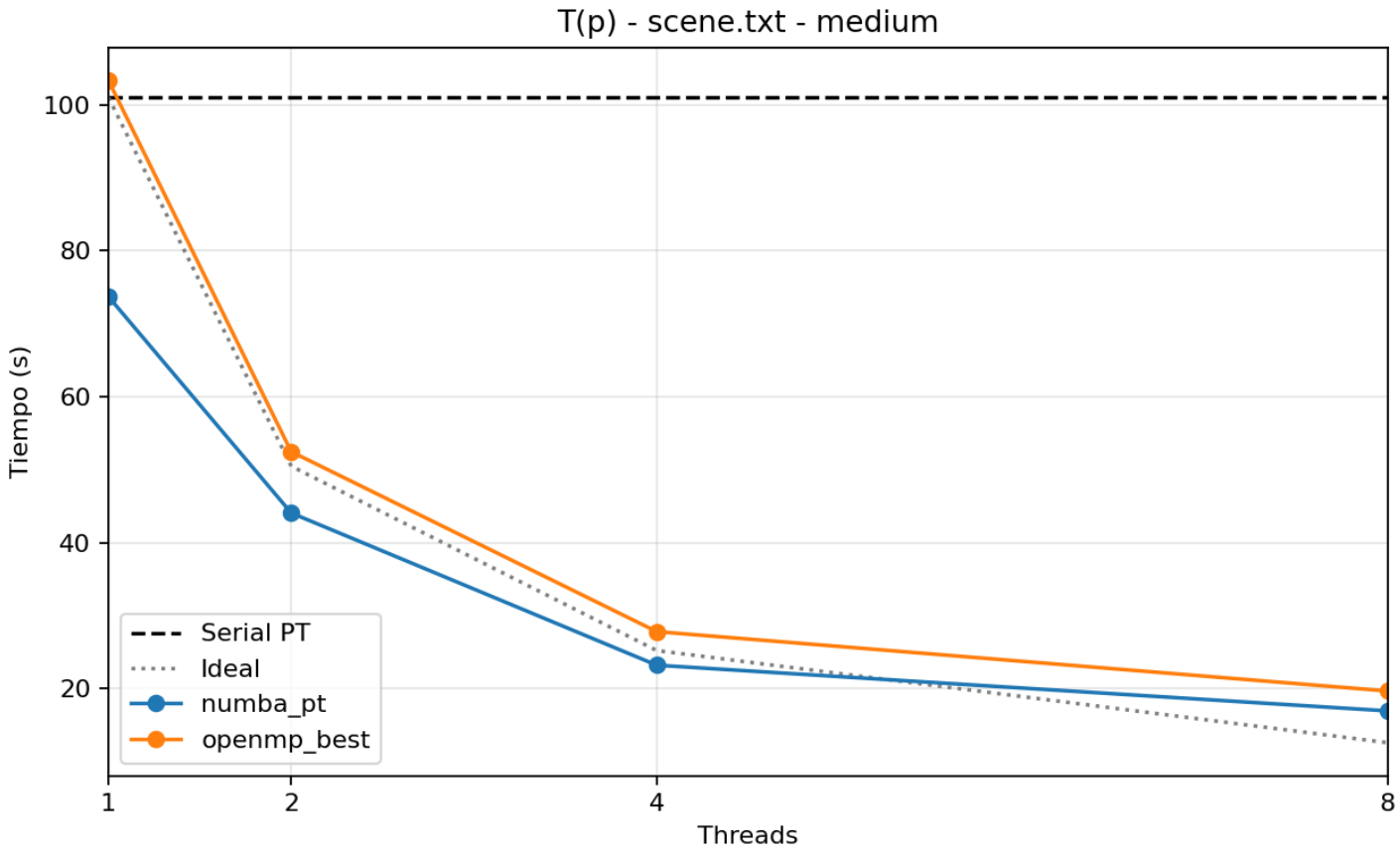

Figure 26: Cluster: Tiempo scene.txt medium

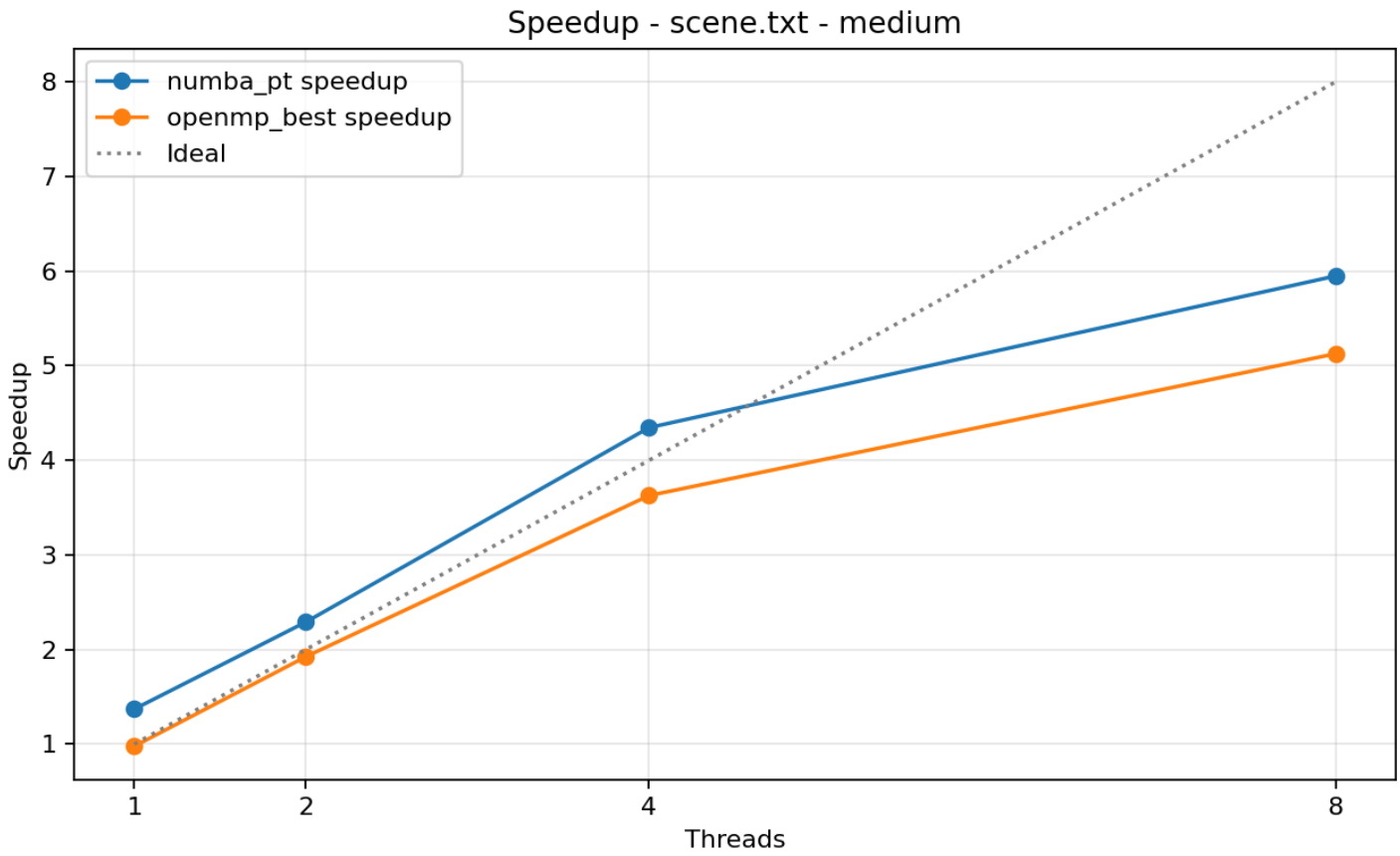

Figure 27: Cluster: Speedup scene.txt medium

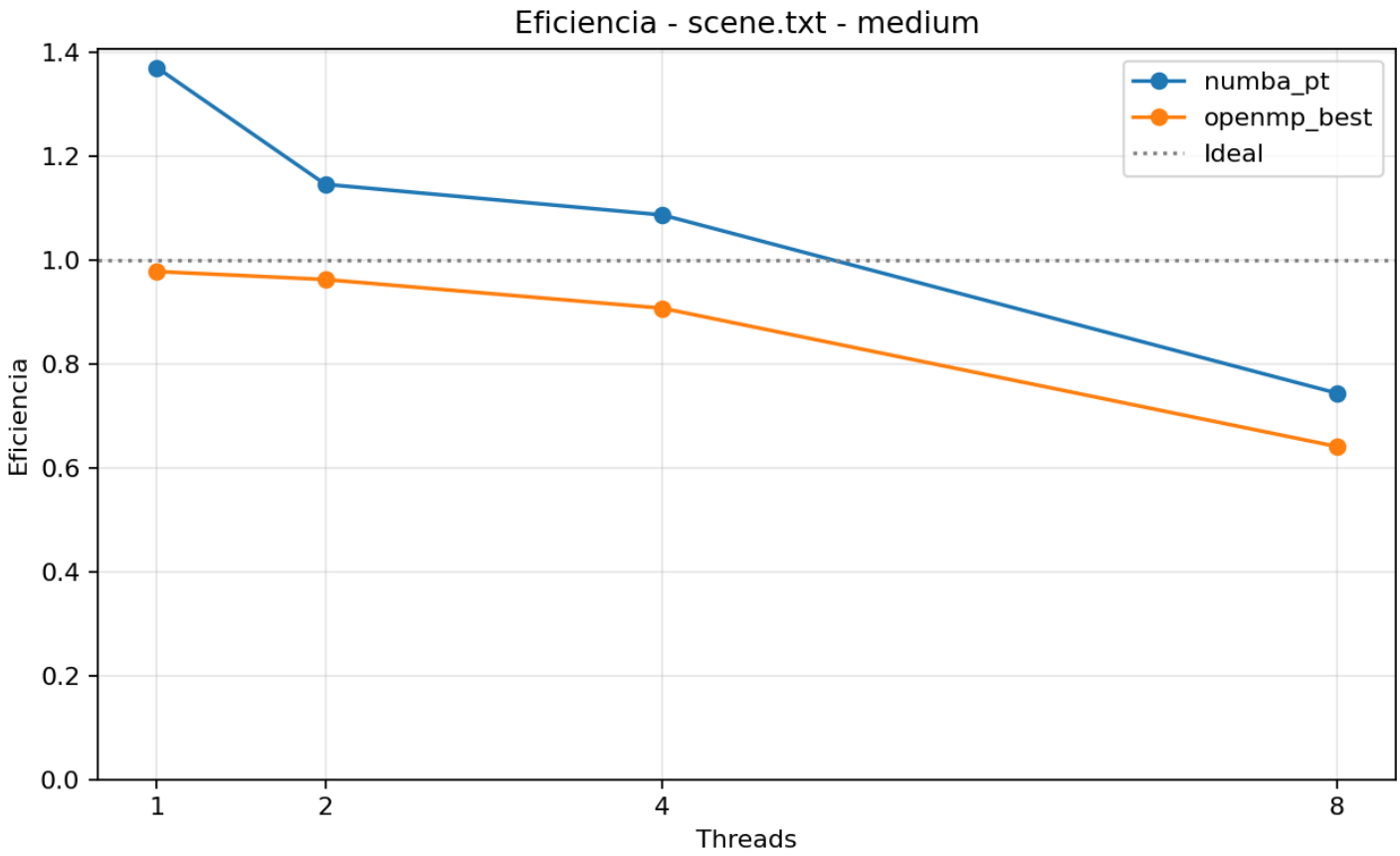

Figure 28: Cluster: Eficiencia scene.txt medium

Cluster - scene.txt large

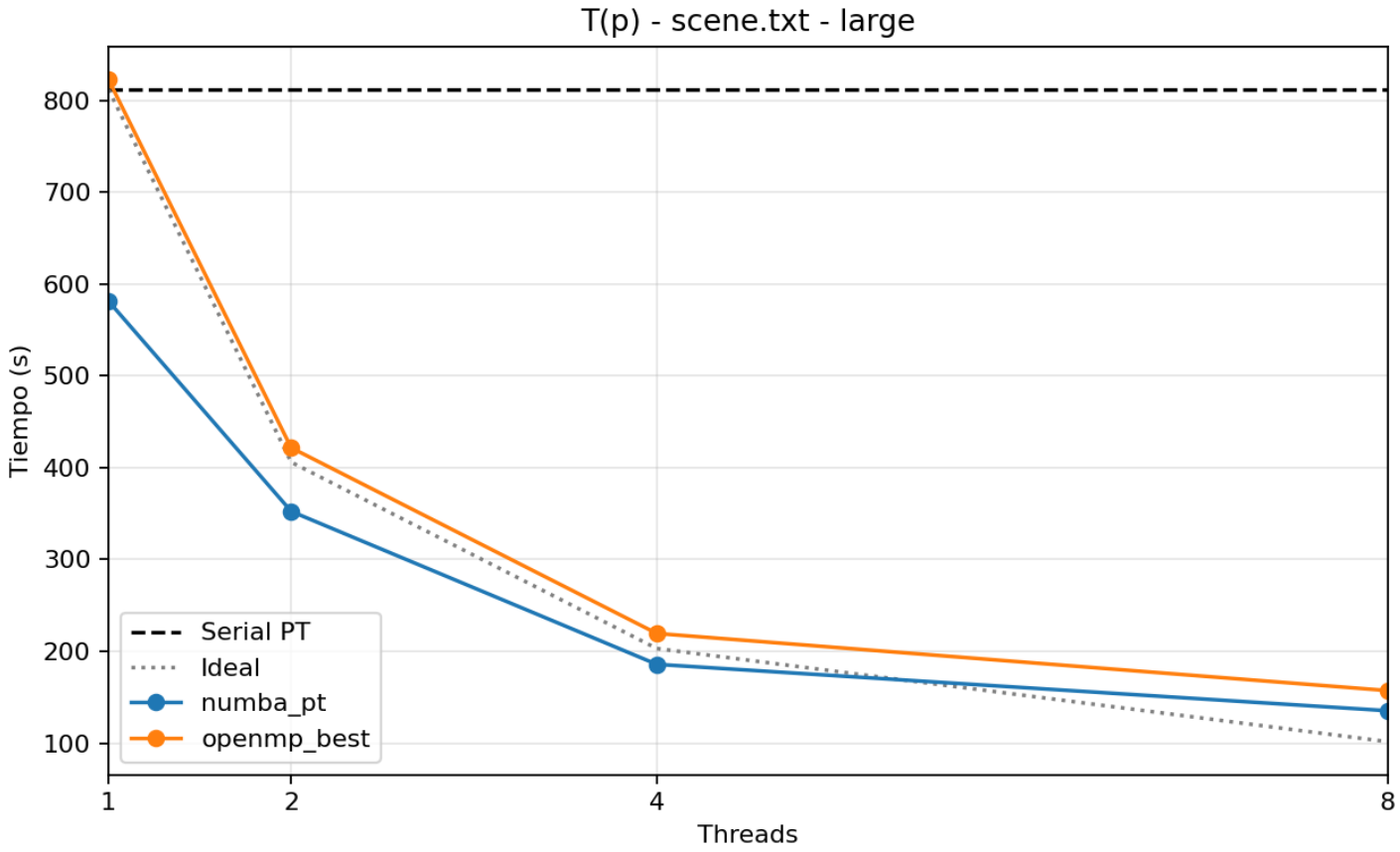

Figure 29: Cluster: Tiempo scene.txt large

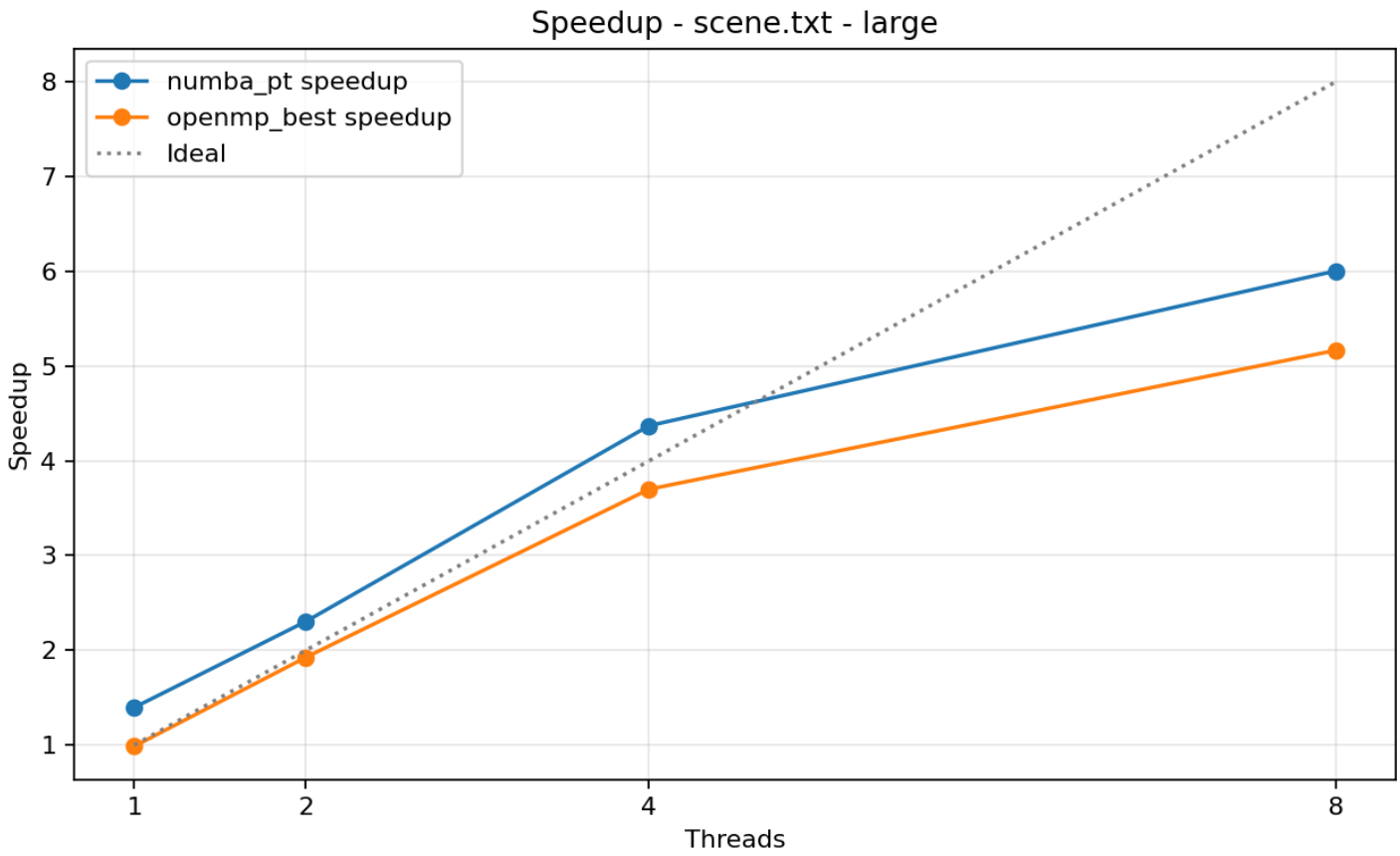

Figure 30: Cluster: Speedup scene.txt large

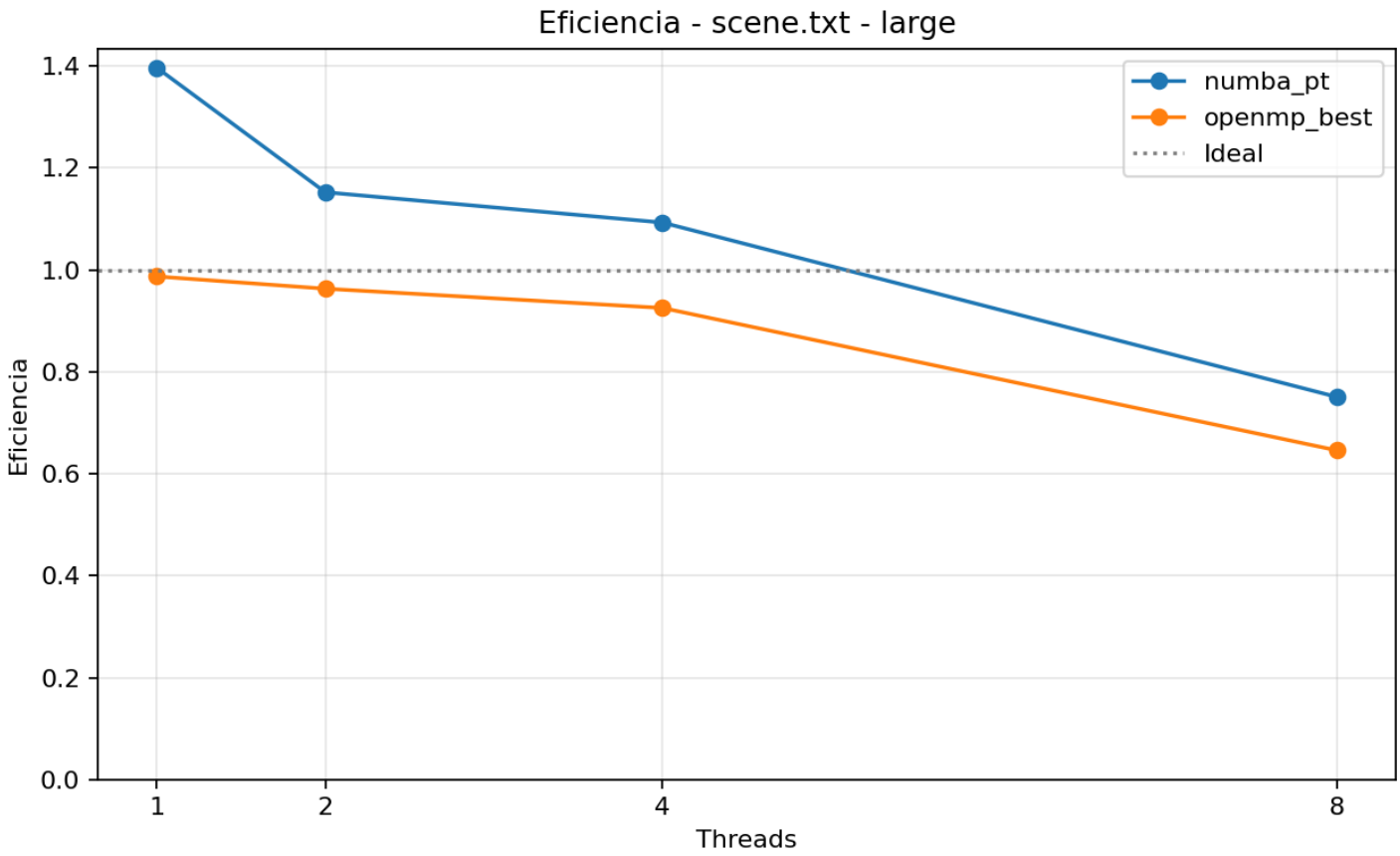

Figure 31: Cluster: Eficiencia scene.txt large

Cluster - scene_many.txt small

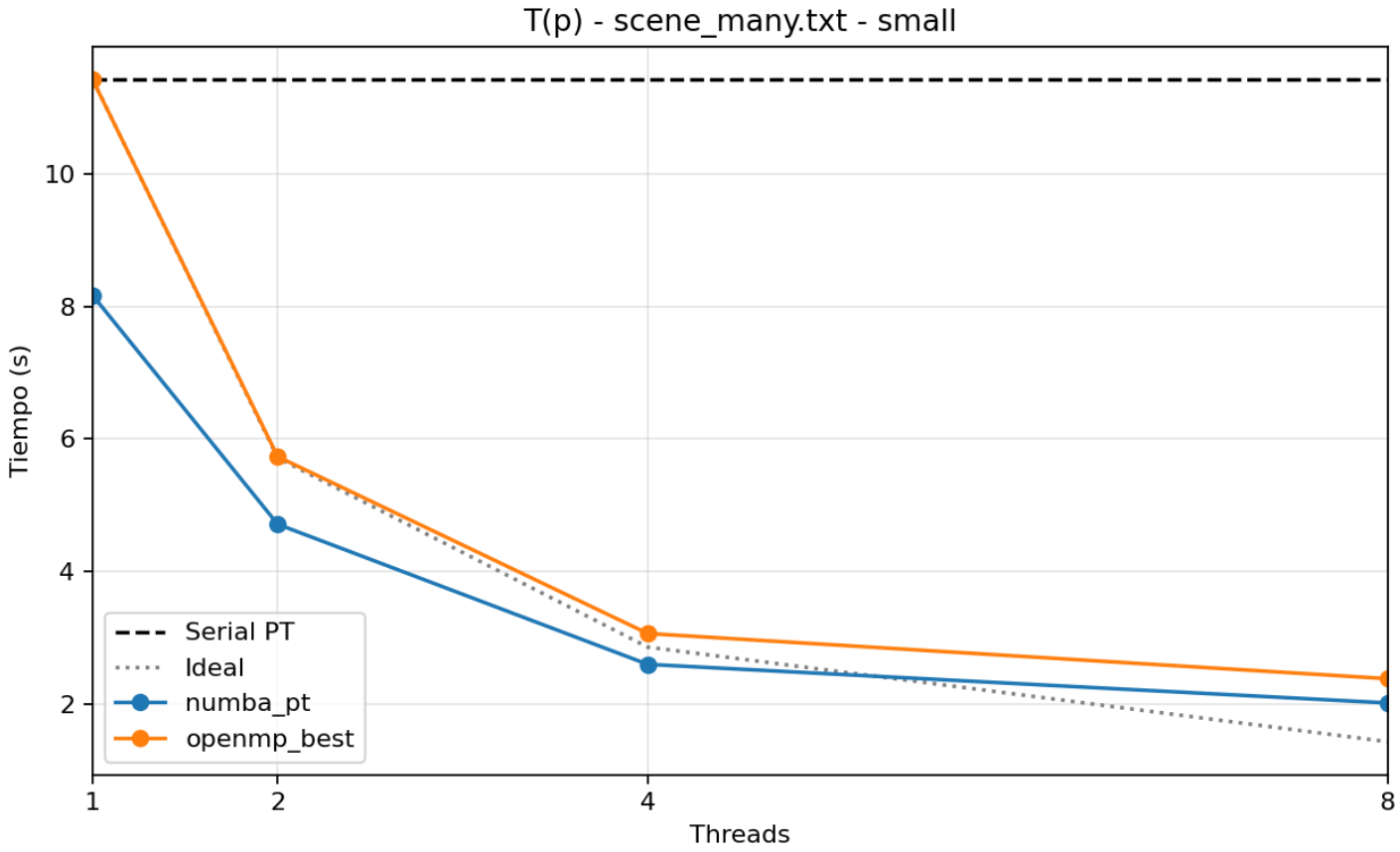

Figure 32: Cluster: Tiempo scene_many.txt small

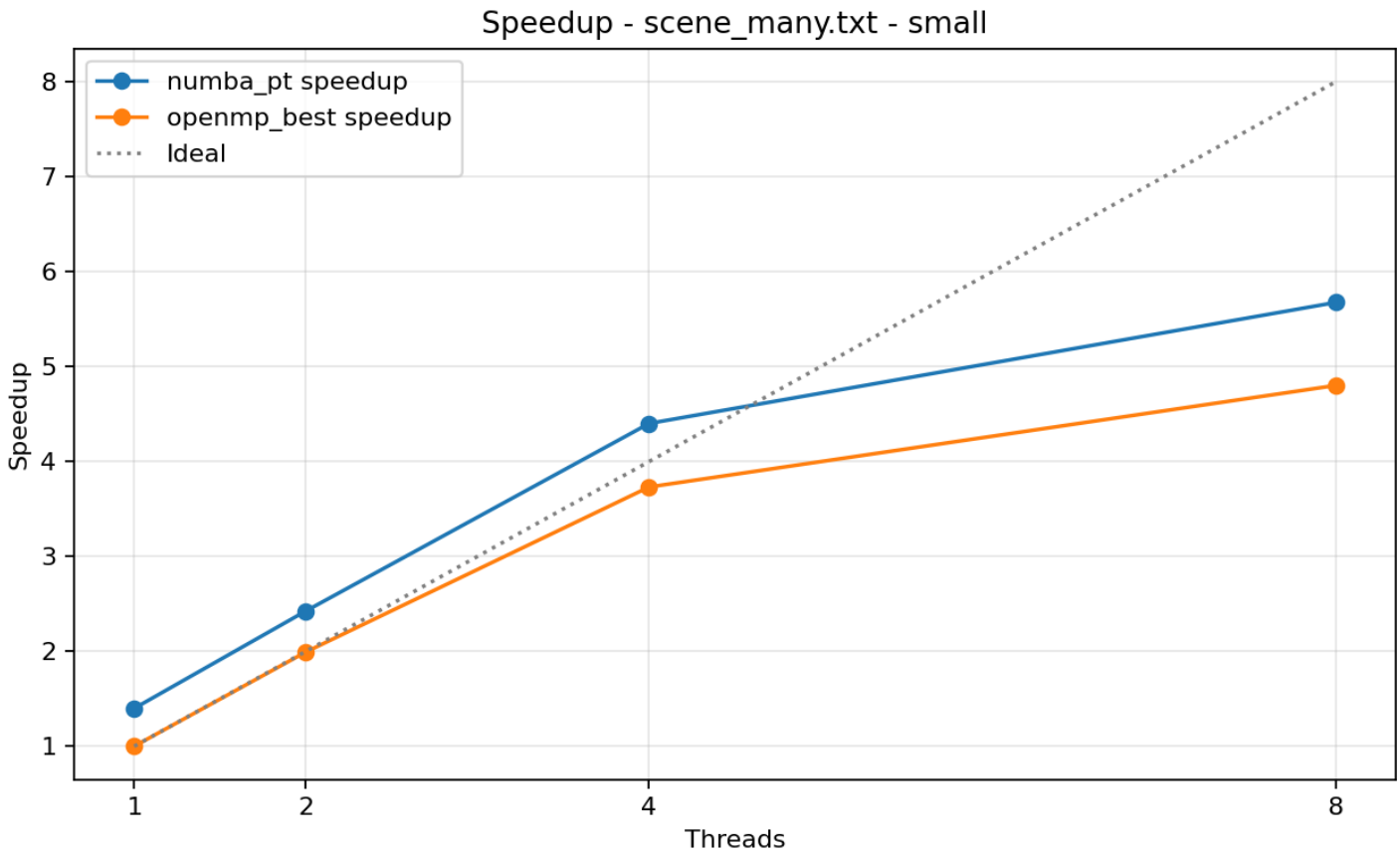

Figure 33: Cluster: Speedup scene_many.txt small

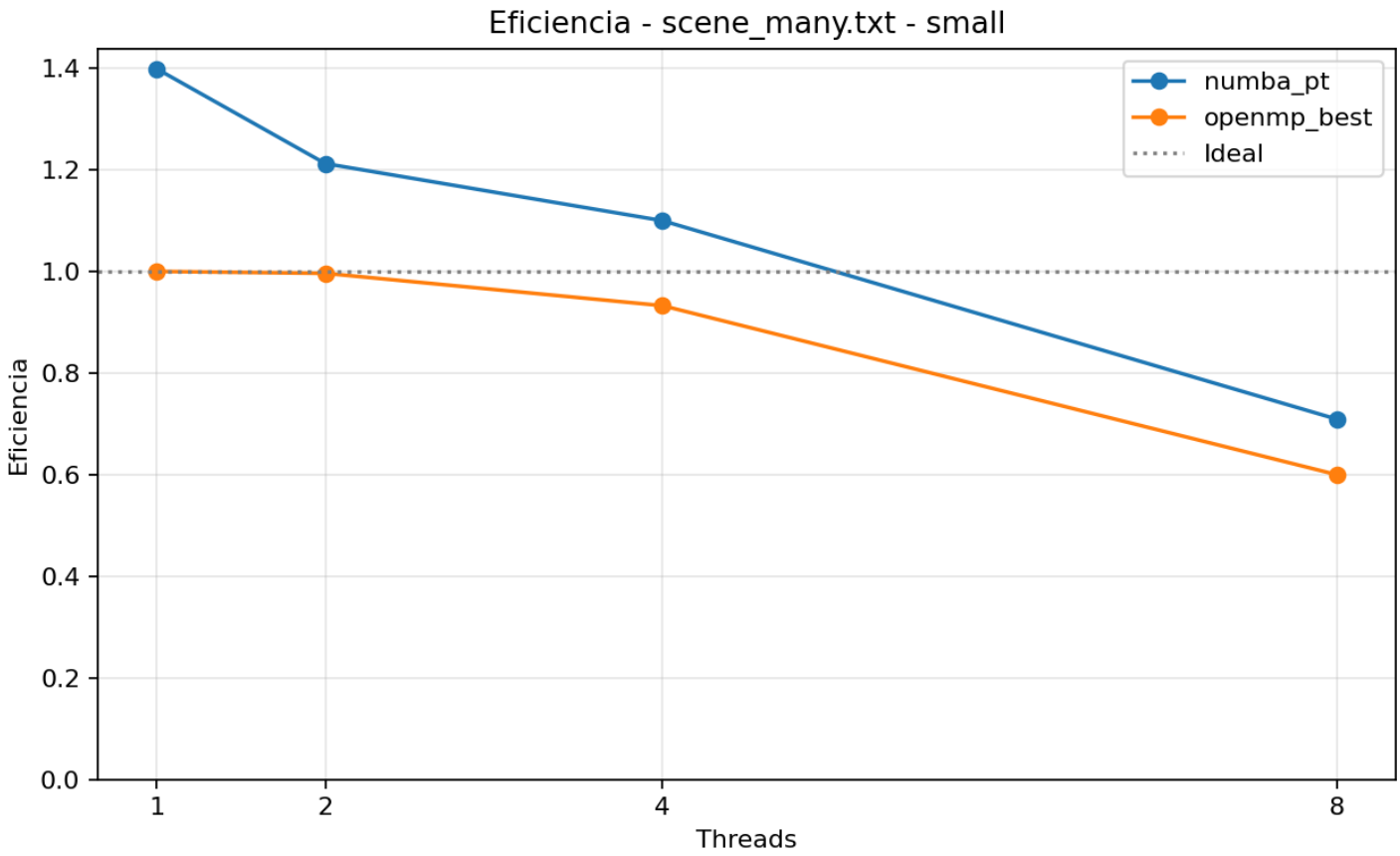

Figure 34: Cluster: Eficiencia scene_many.txt small

Cluster - scene_many.txt medium

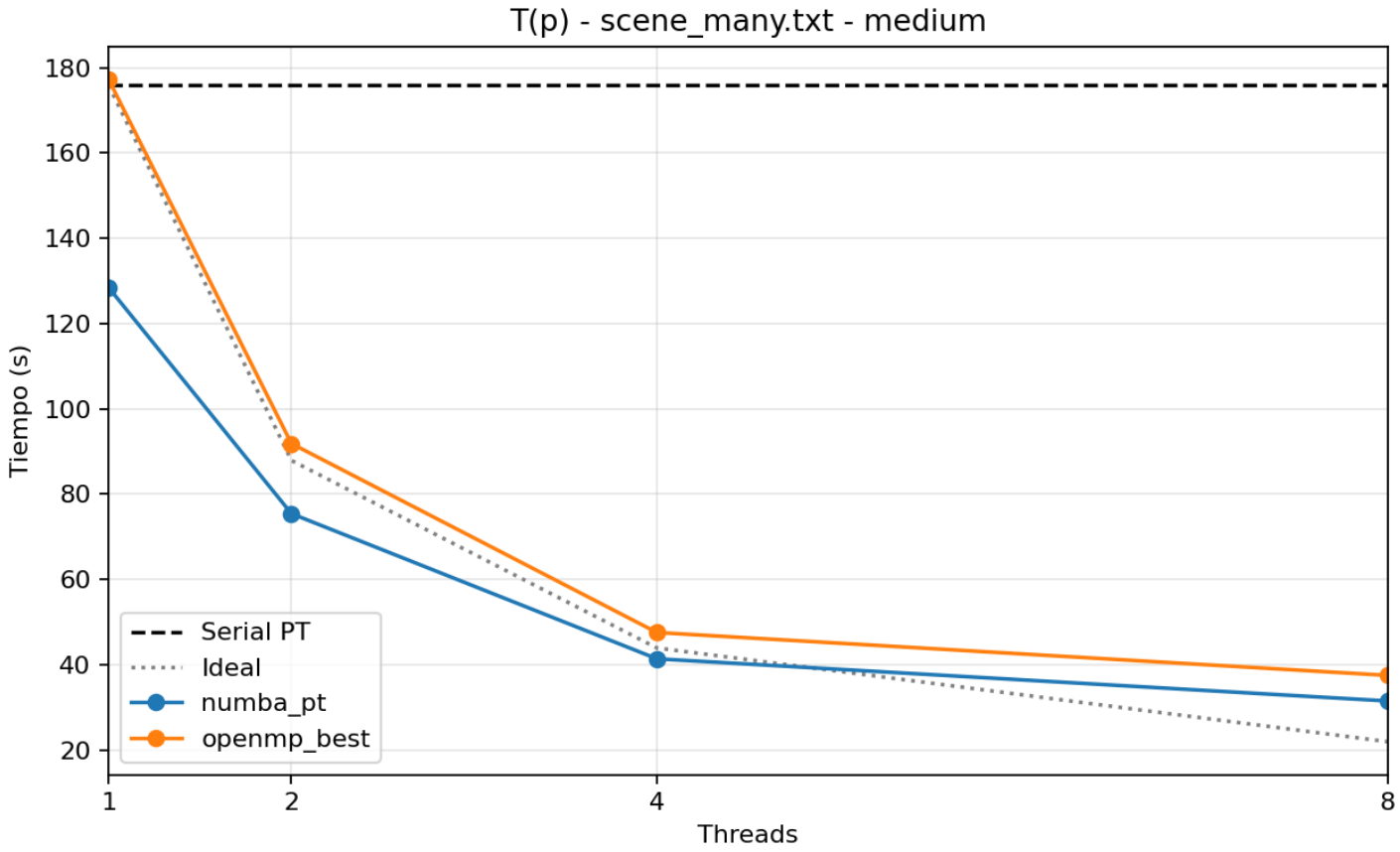

Figure 35: Cluster: Tiempo scene_many.txt medium

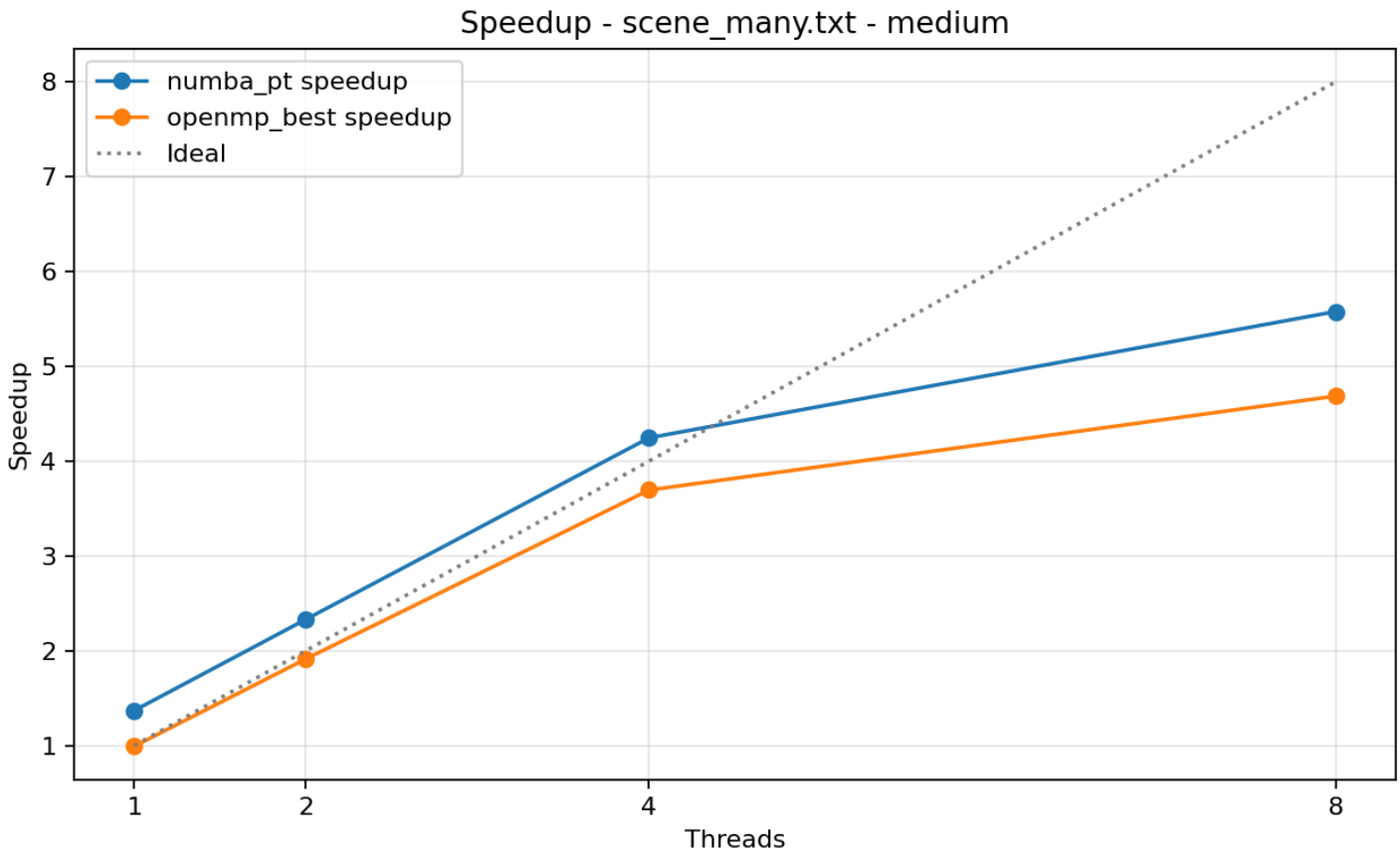

Figure 36: Cluster: Speedup scene_many.txt medium

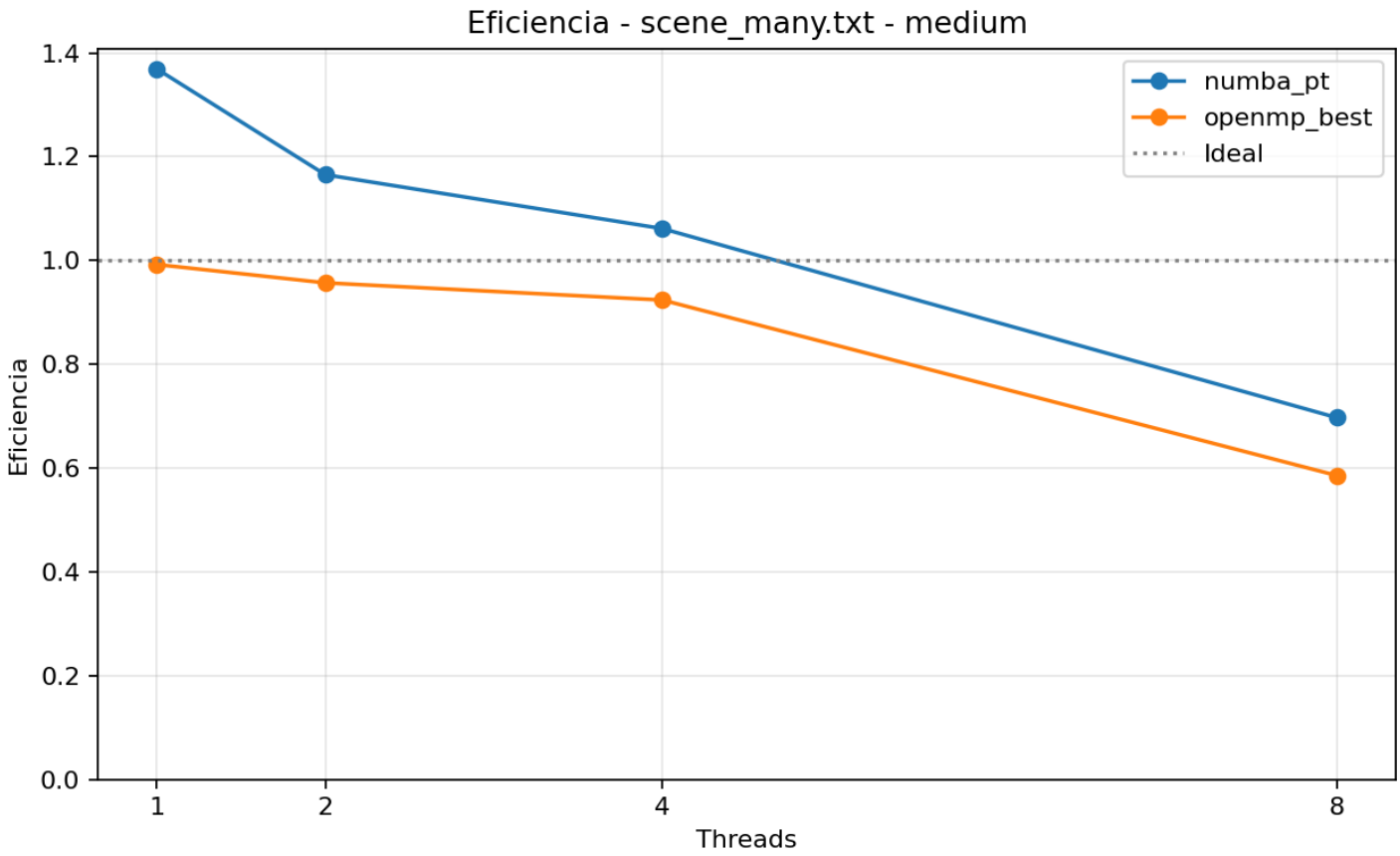

Figure 37: Cluster: Eficiencia scene_many.txt medium

Cluster - scene_many.txt large

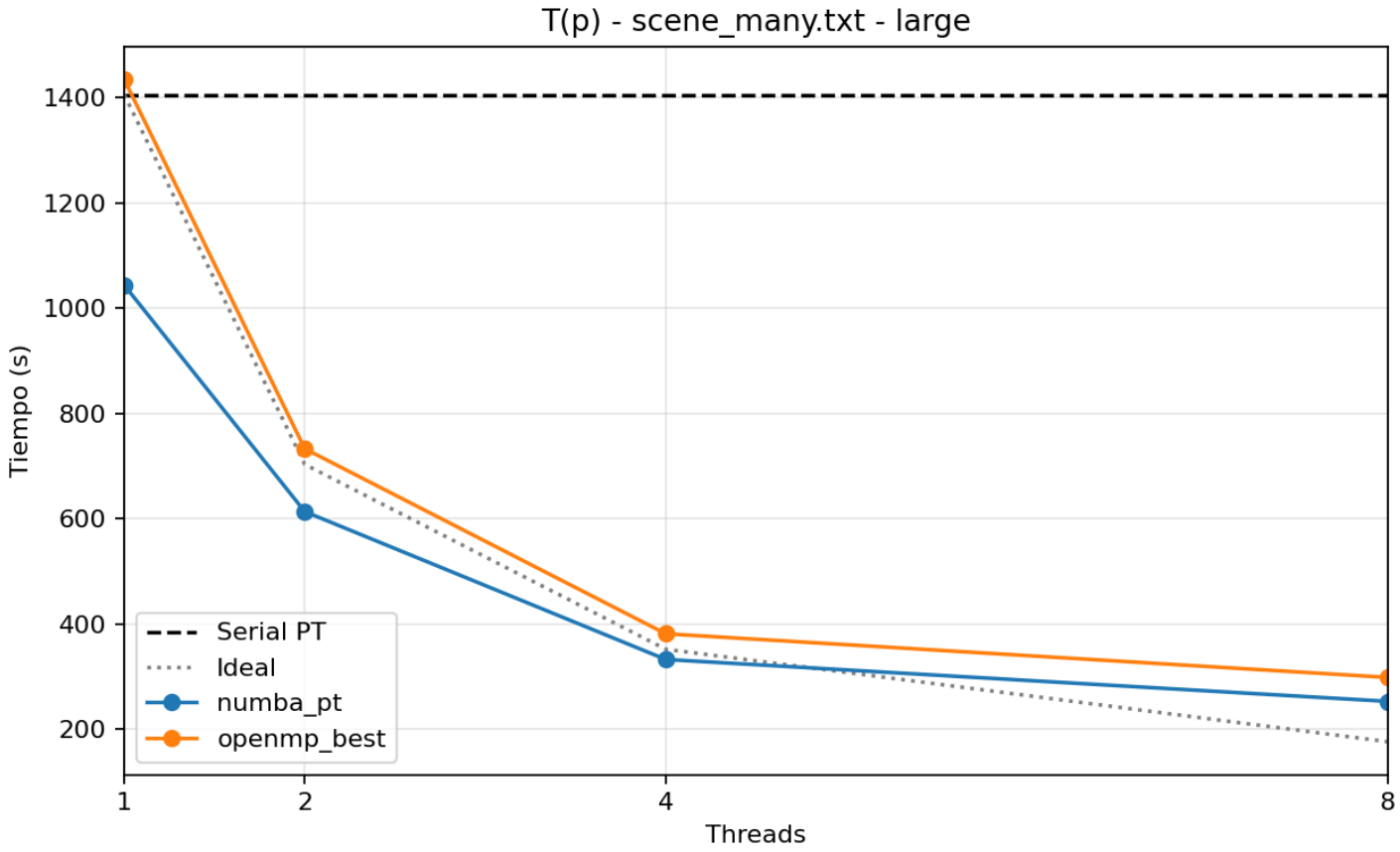

Figure 38: Cluster: Tiempo scene_many.txt large

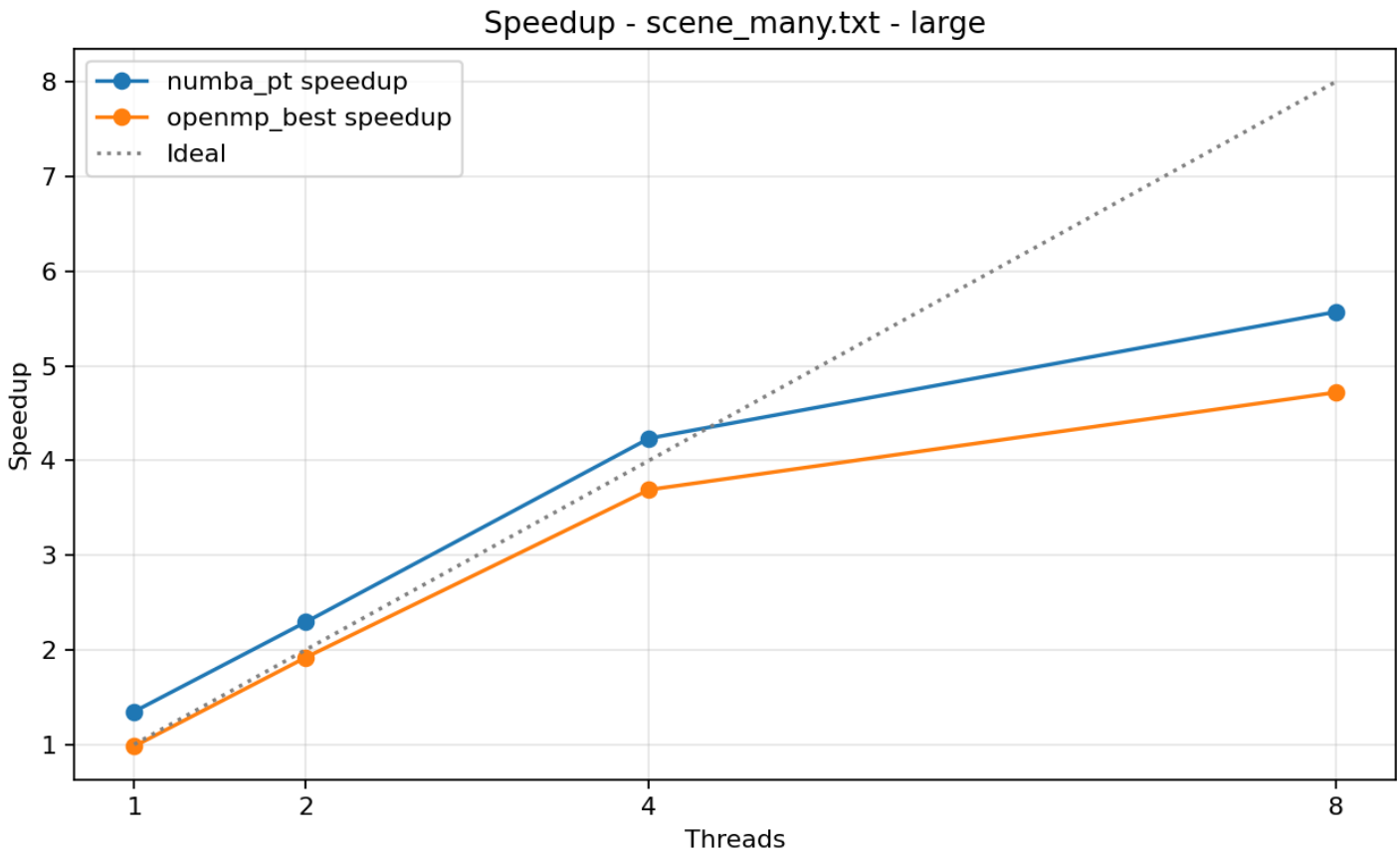

Figure 39: Cluster: Speedup scene_many.txt large

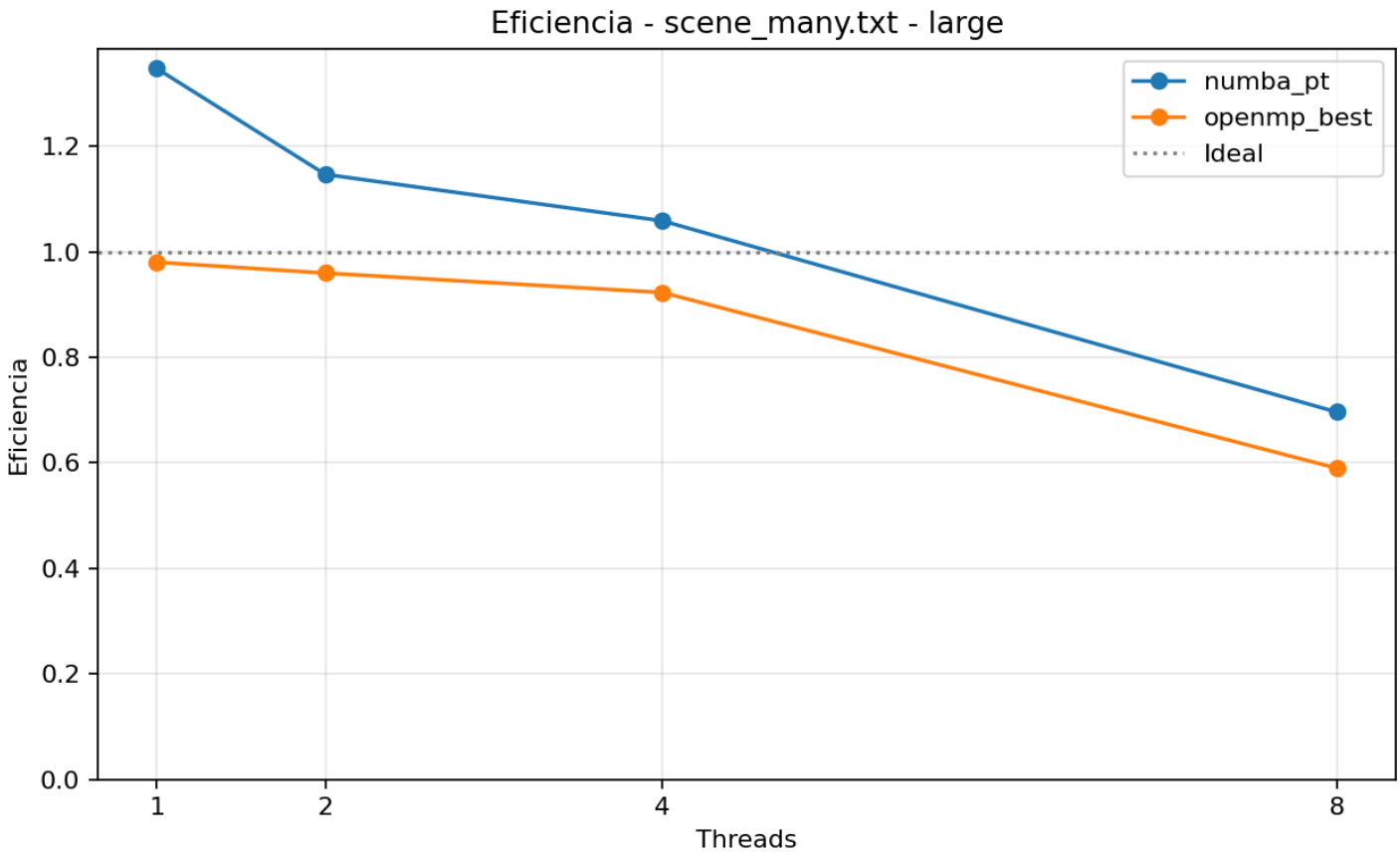

Figure 40: Cluster: Eficiencia scene_many.txt large

Grupo (i5-10600K)

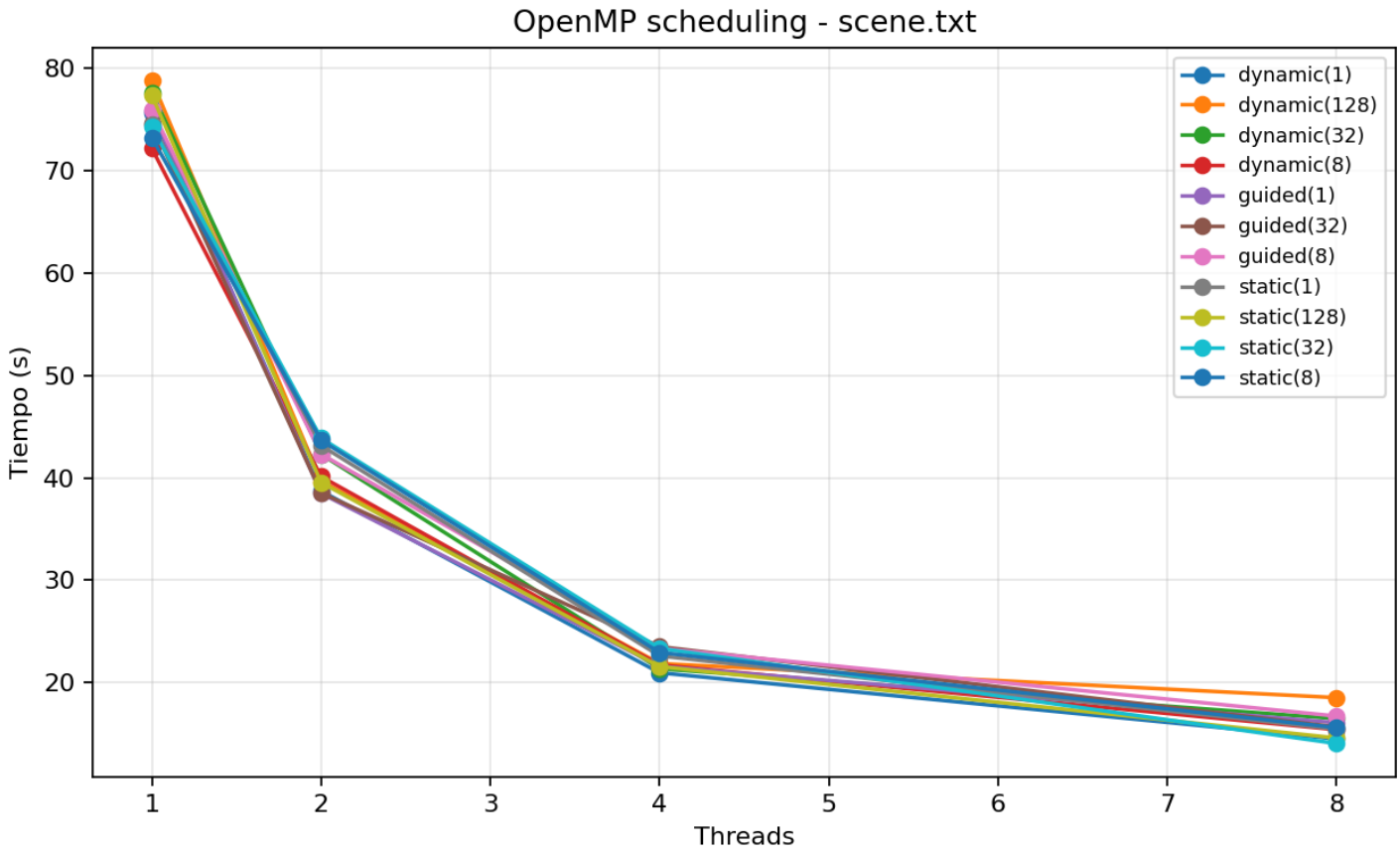

Figure 41: Grupo: Scheduling scene.txt

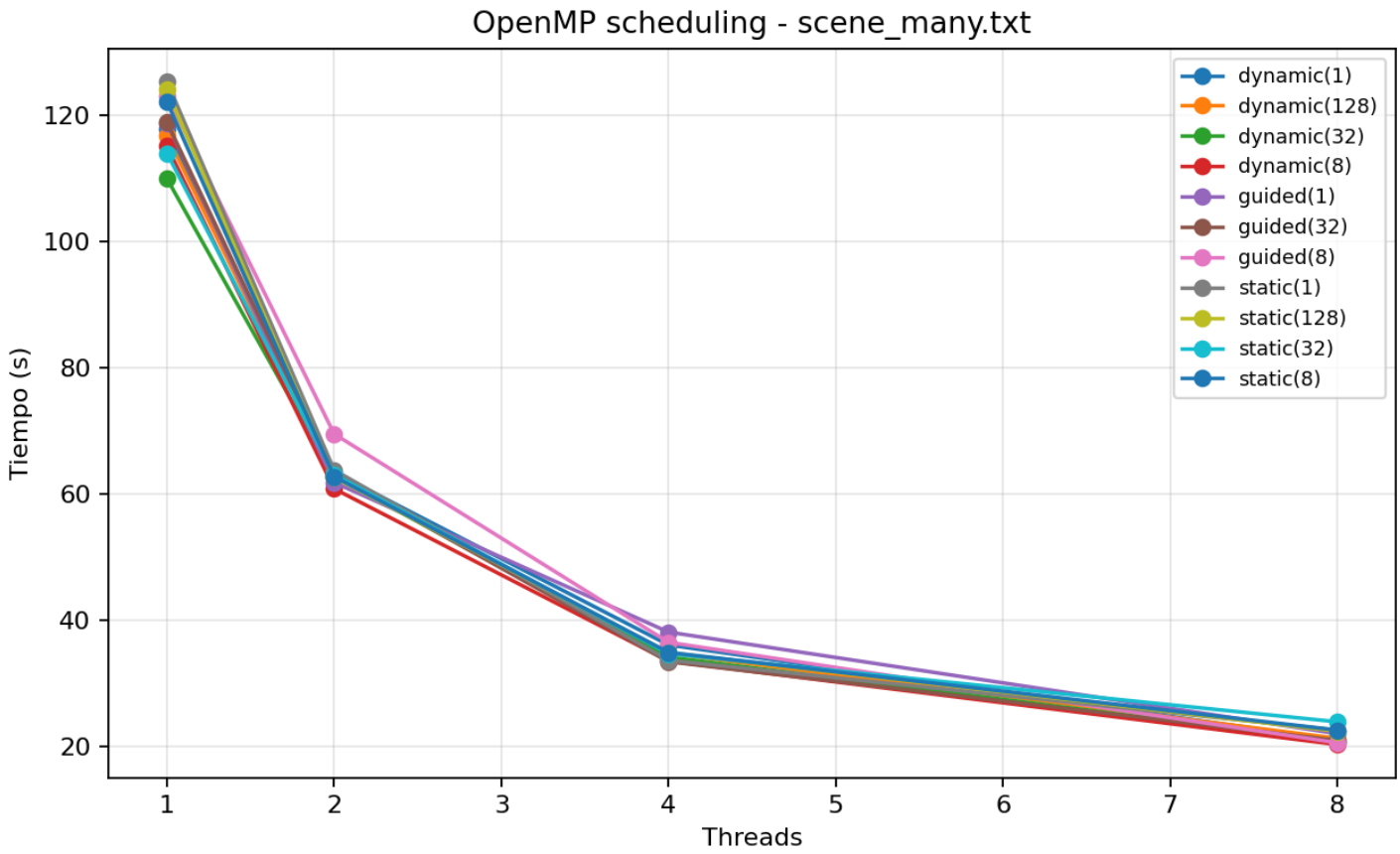

Figure 42: Grupo: Scheduling scene_many.txt

Grupo - scene.txt small

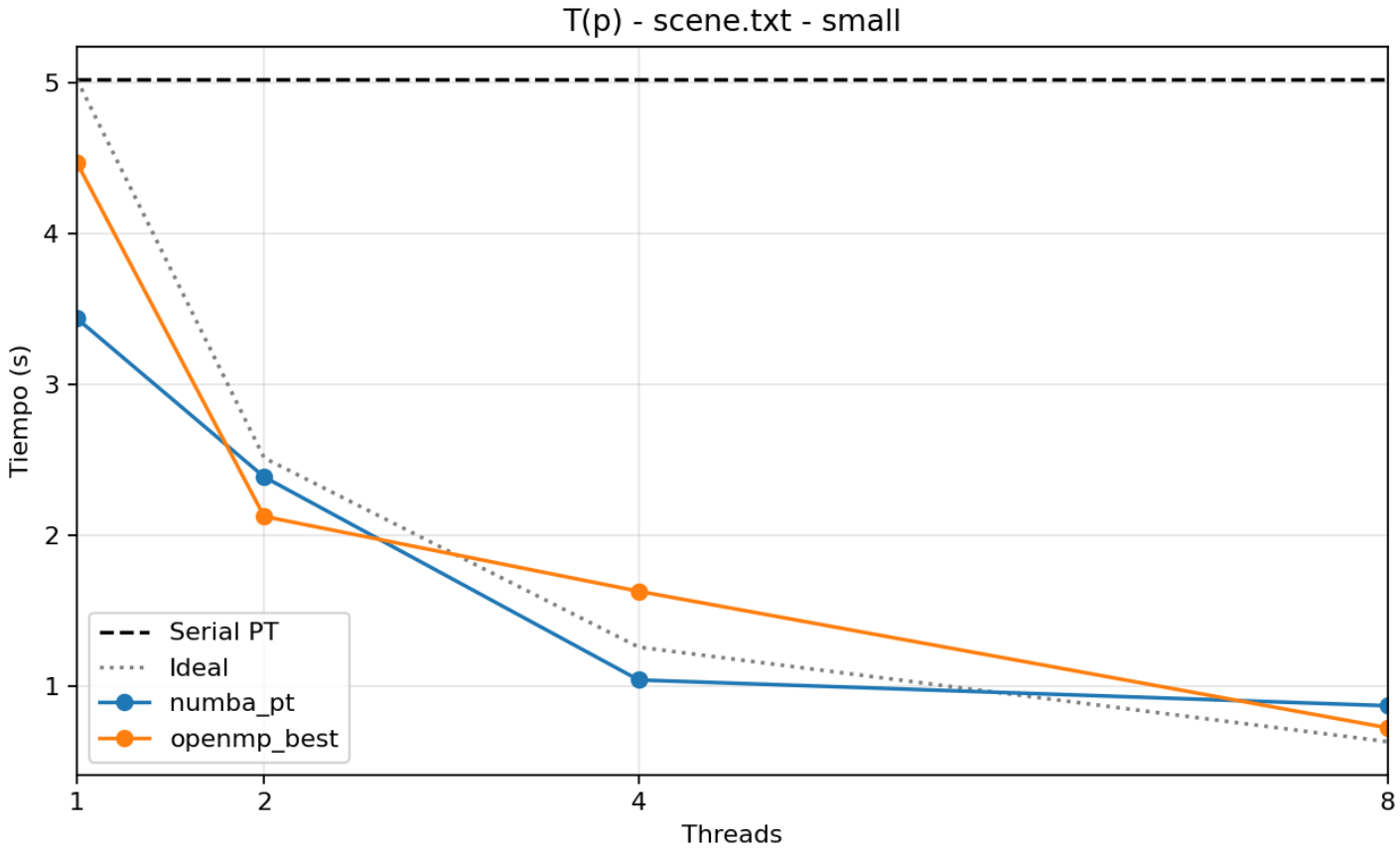

Figure 43: Grupo: Tiempo scene.txt small

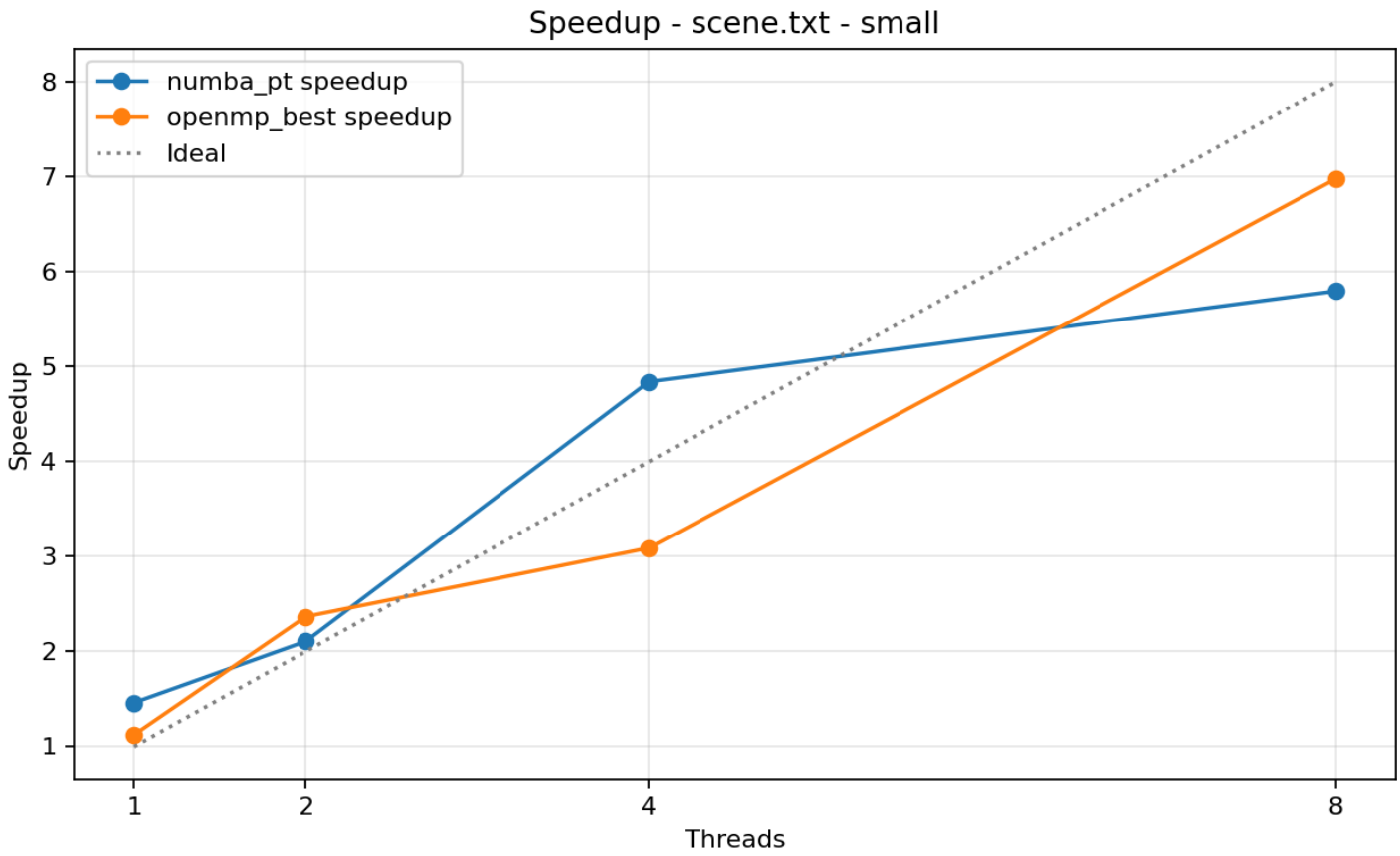

Figure 44: Grupo: Speedup scene.txt small

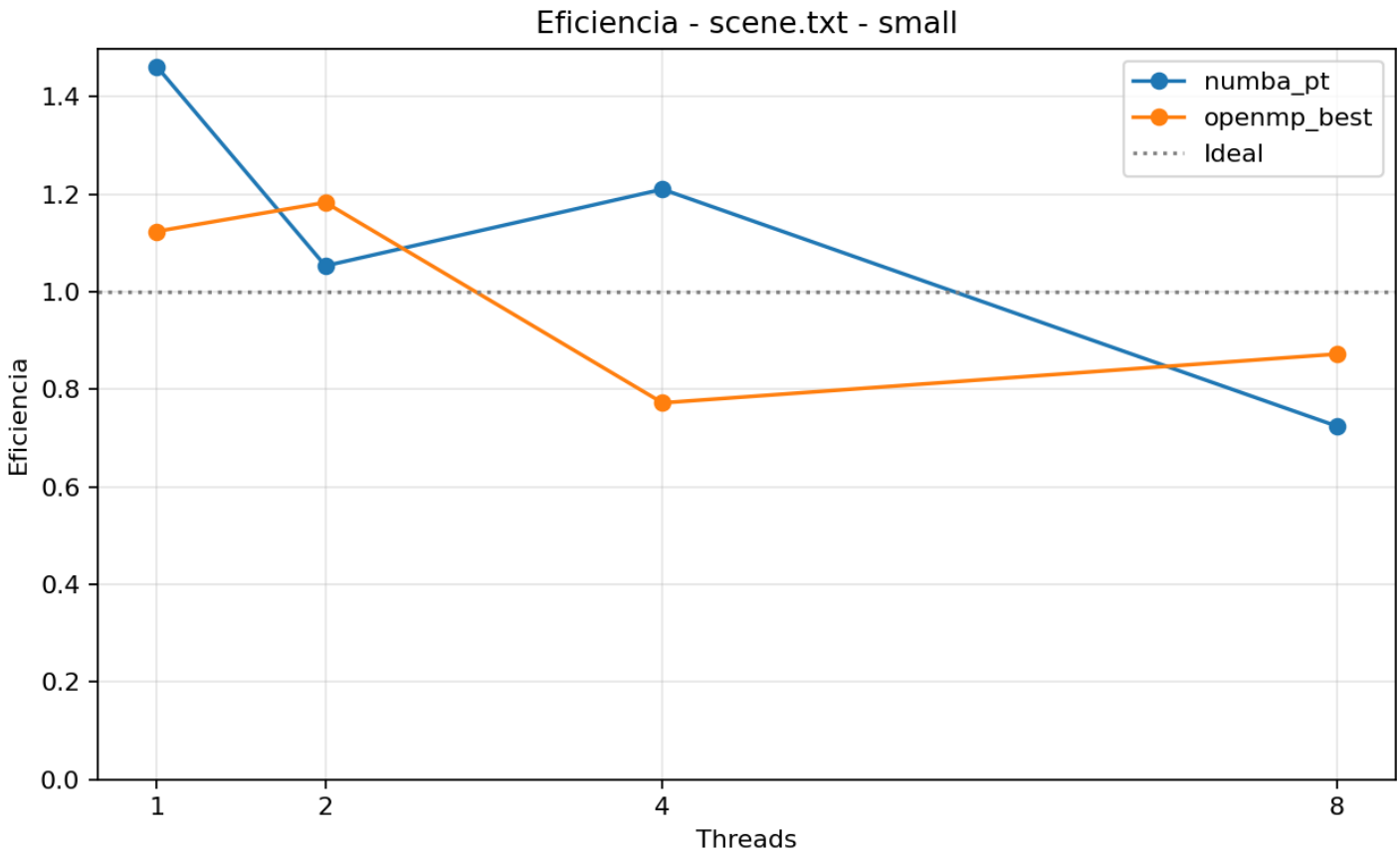

Figure 45: Grupo: Eficiencia scene.txt small

Grupo - scene.txt medium

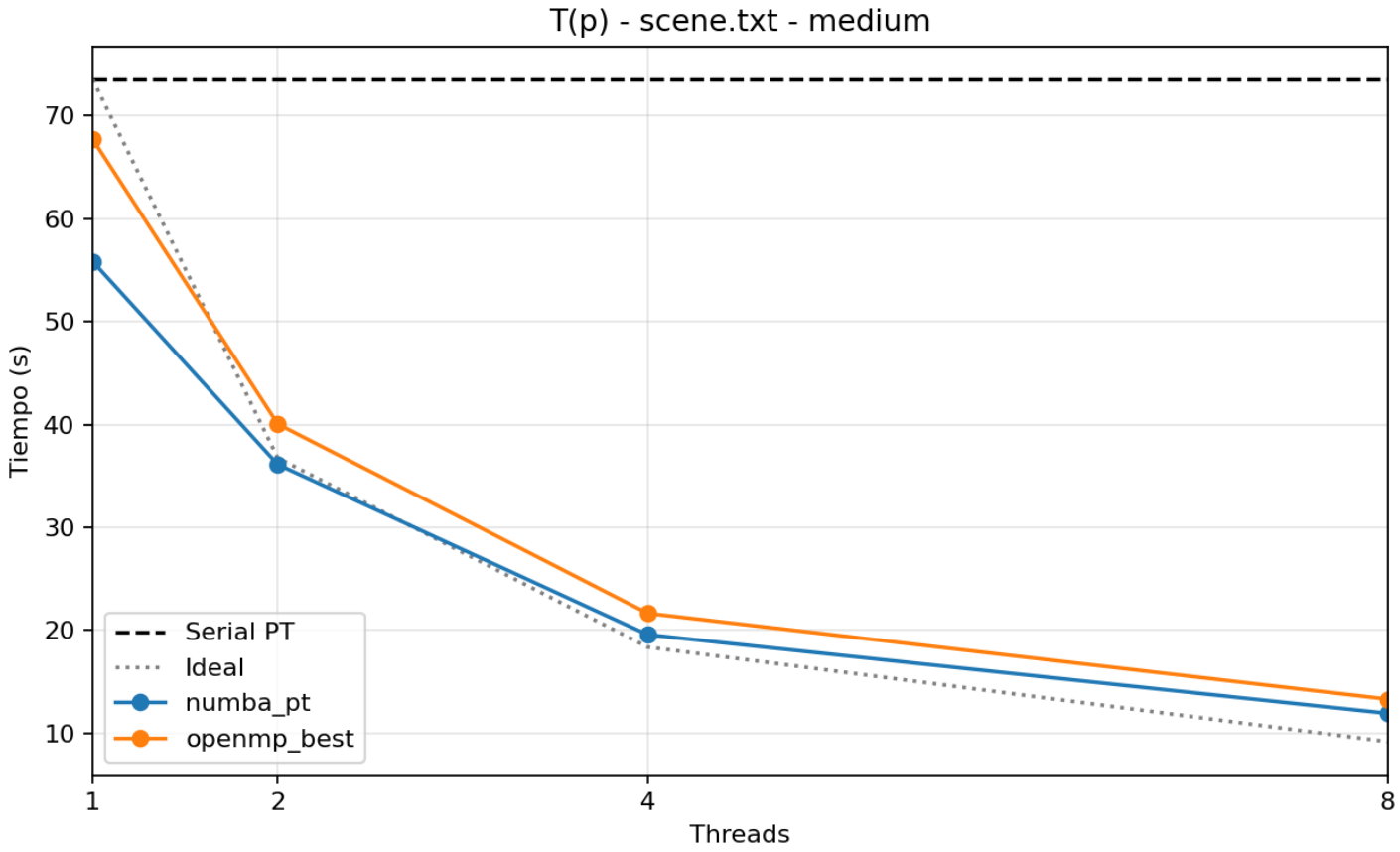

Figure 46: Grupo: Tiempo scene.txt medium

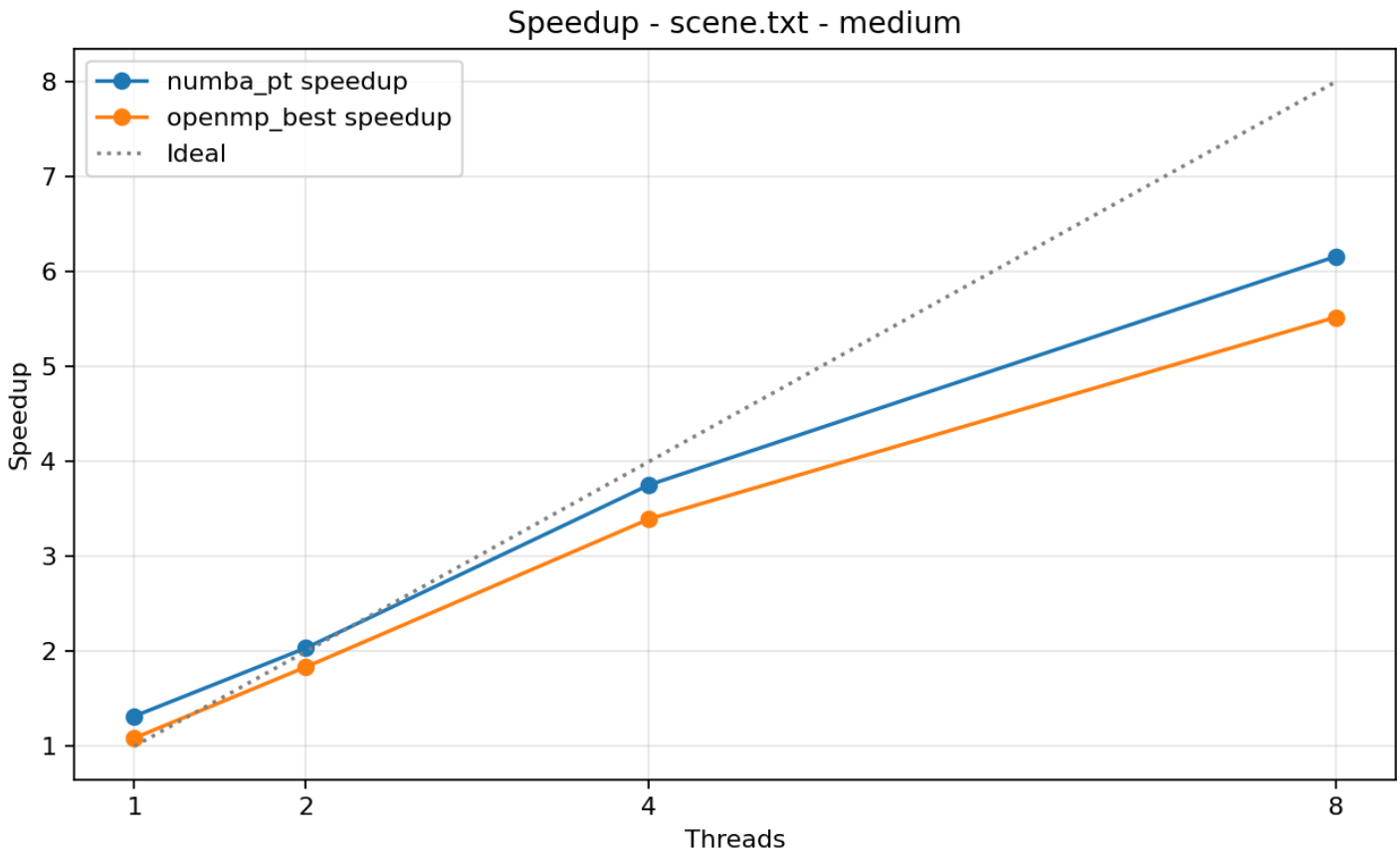

Figure 47: Grupo: Speedup scene.txt medium

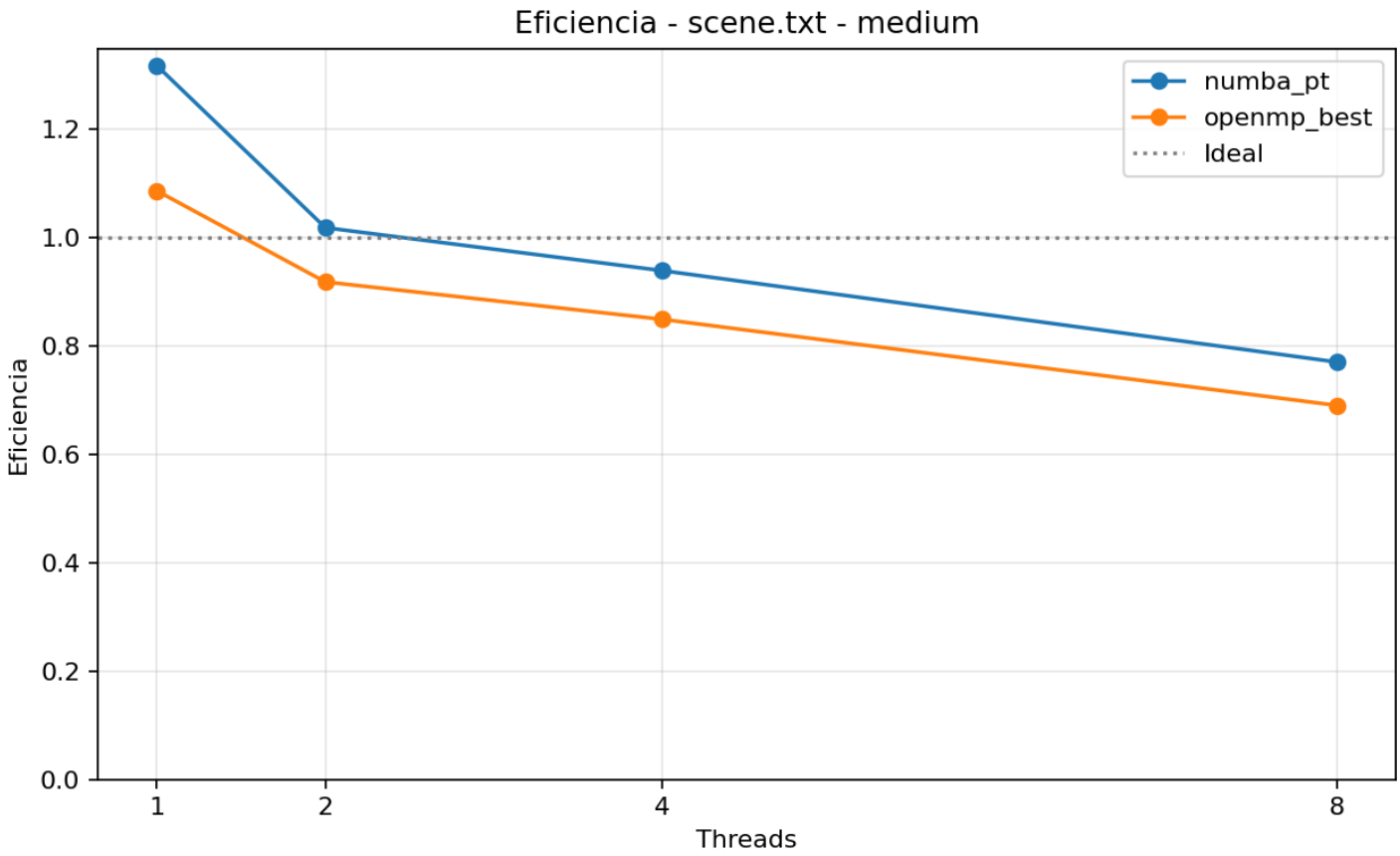

Figure 48: Grupo: Eficiencia scene.txt medium

Grupo - scene.txt large

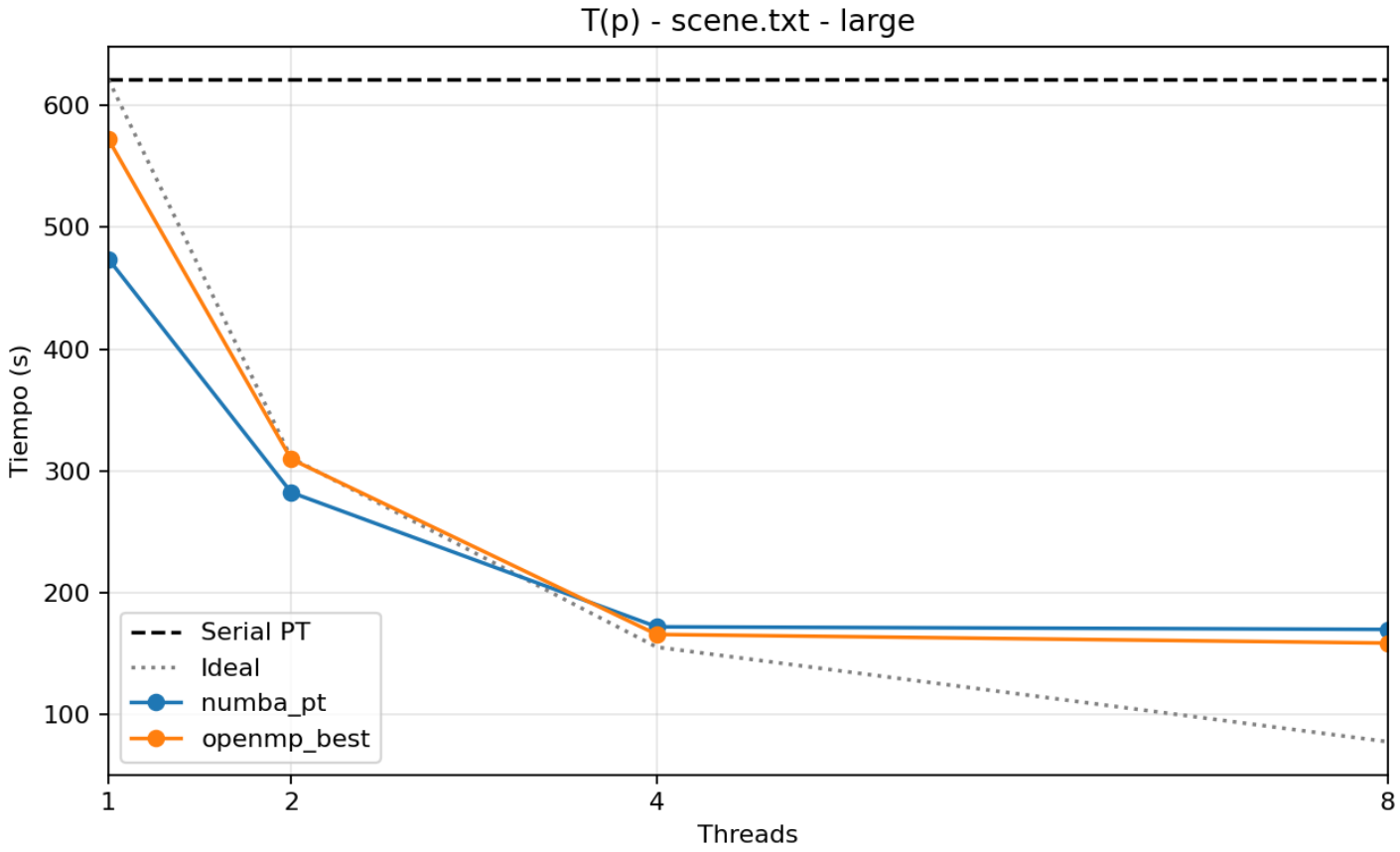

Figure 49: Grupo: Tiempo scene.txt large

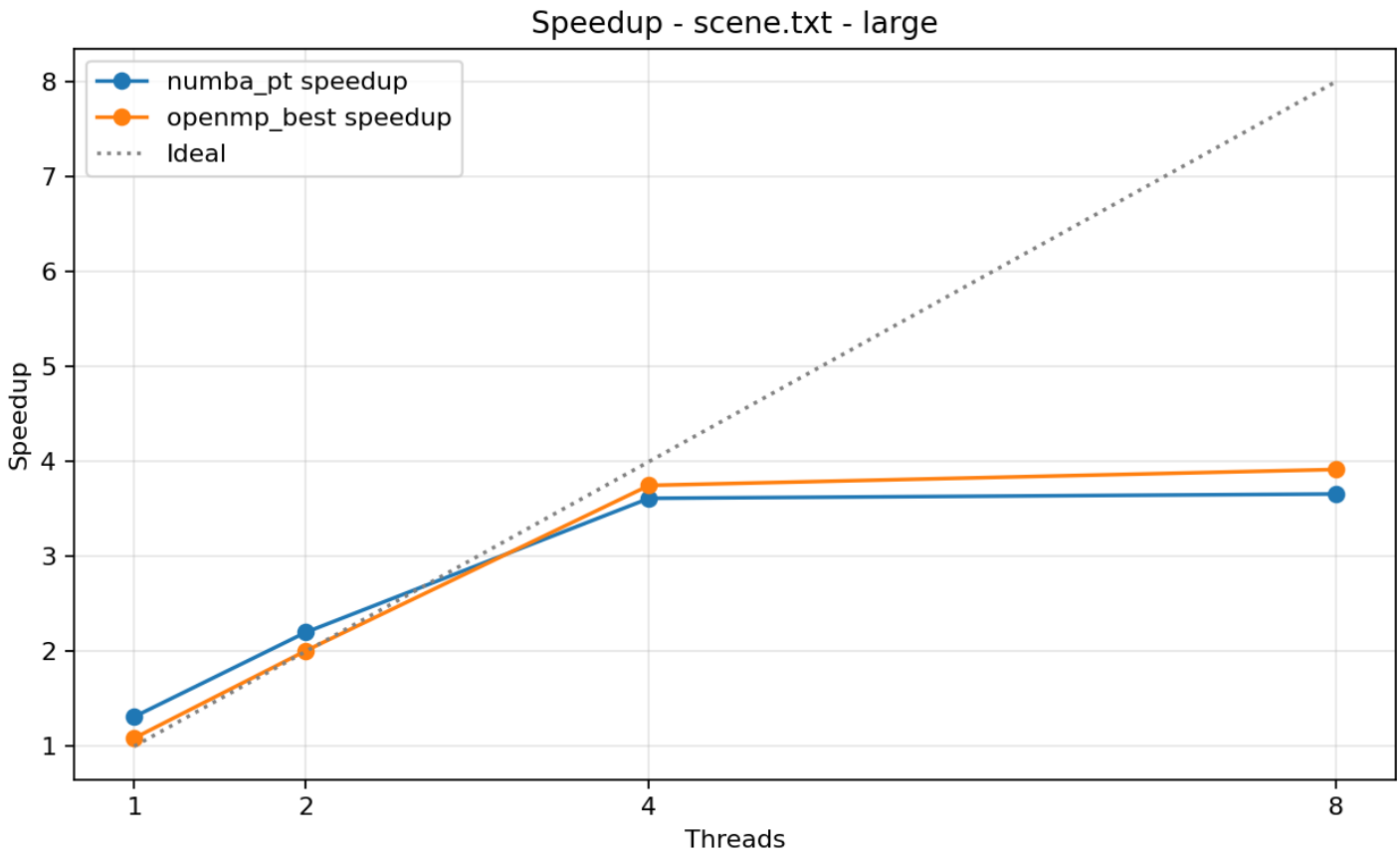

Figure 50: Grupo: Speedup scene.txt large

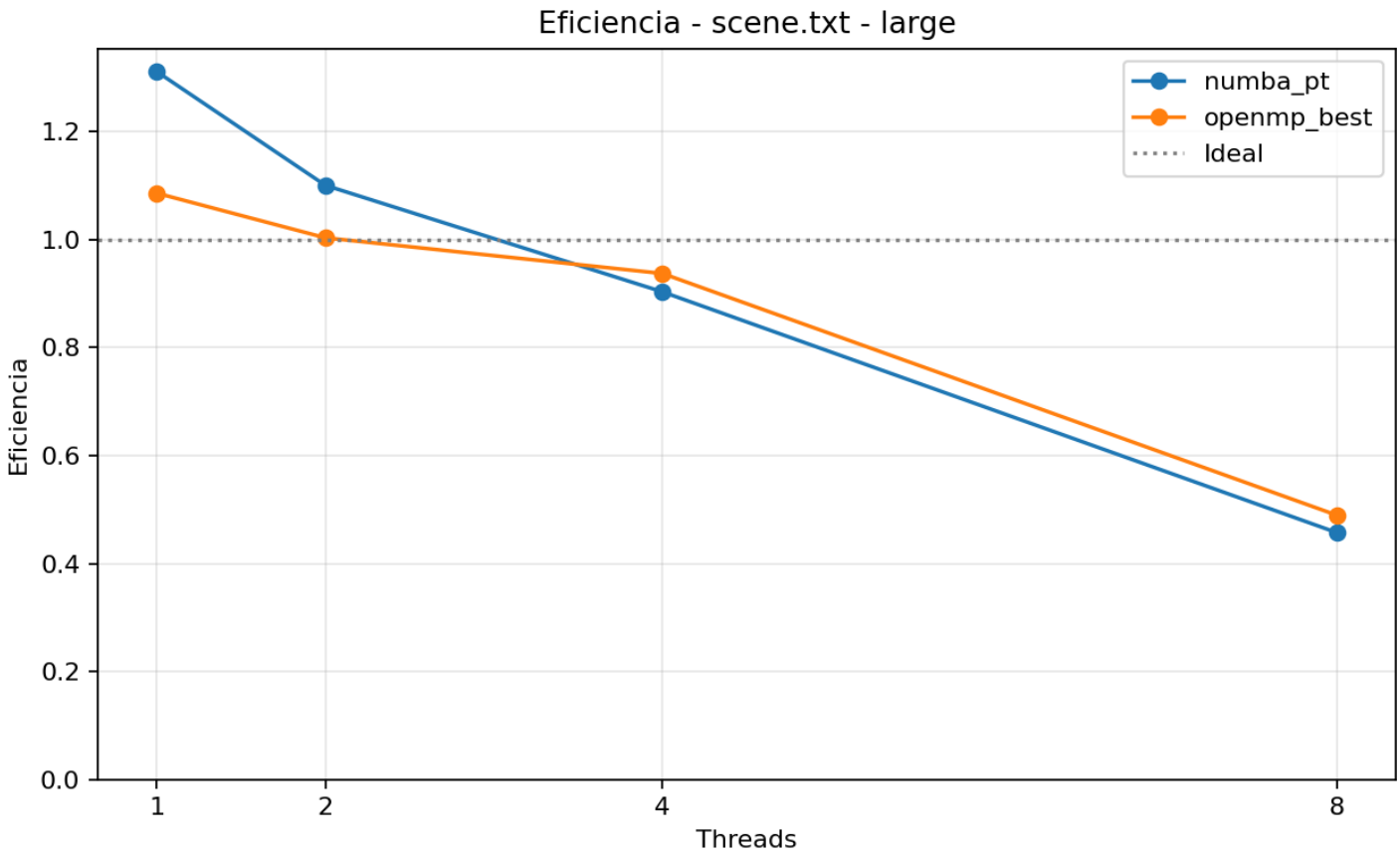

Figure 51: Grupo: Eficiencia scene.txt large

Grupo - scene_many.txt small

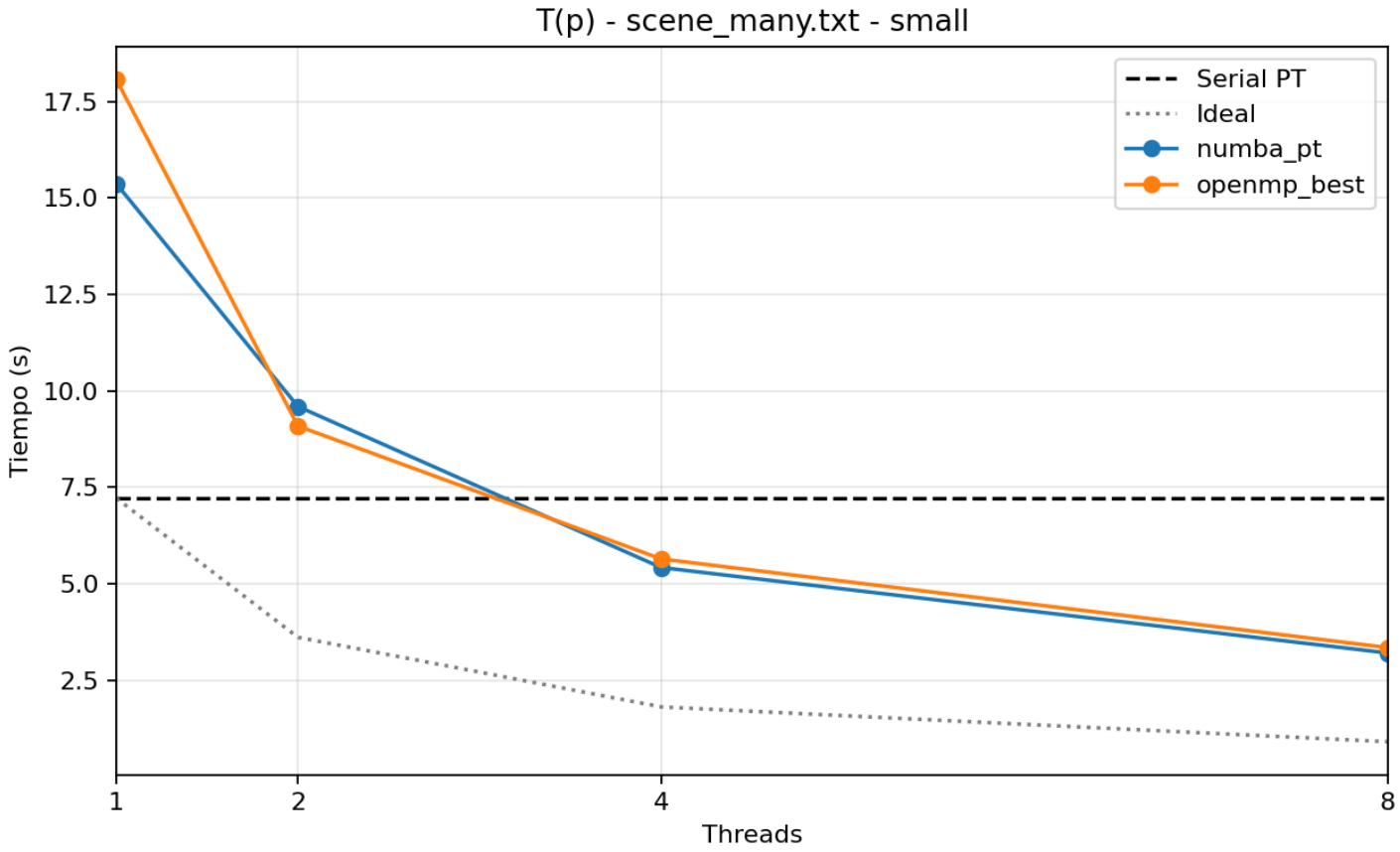

Figure 52: Grupo: Tiempo scene_many.txt small

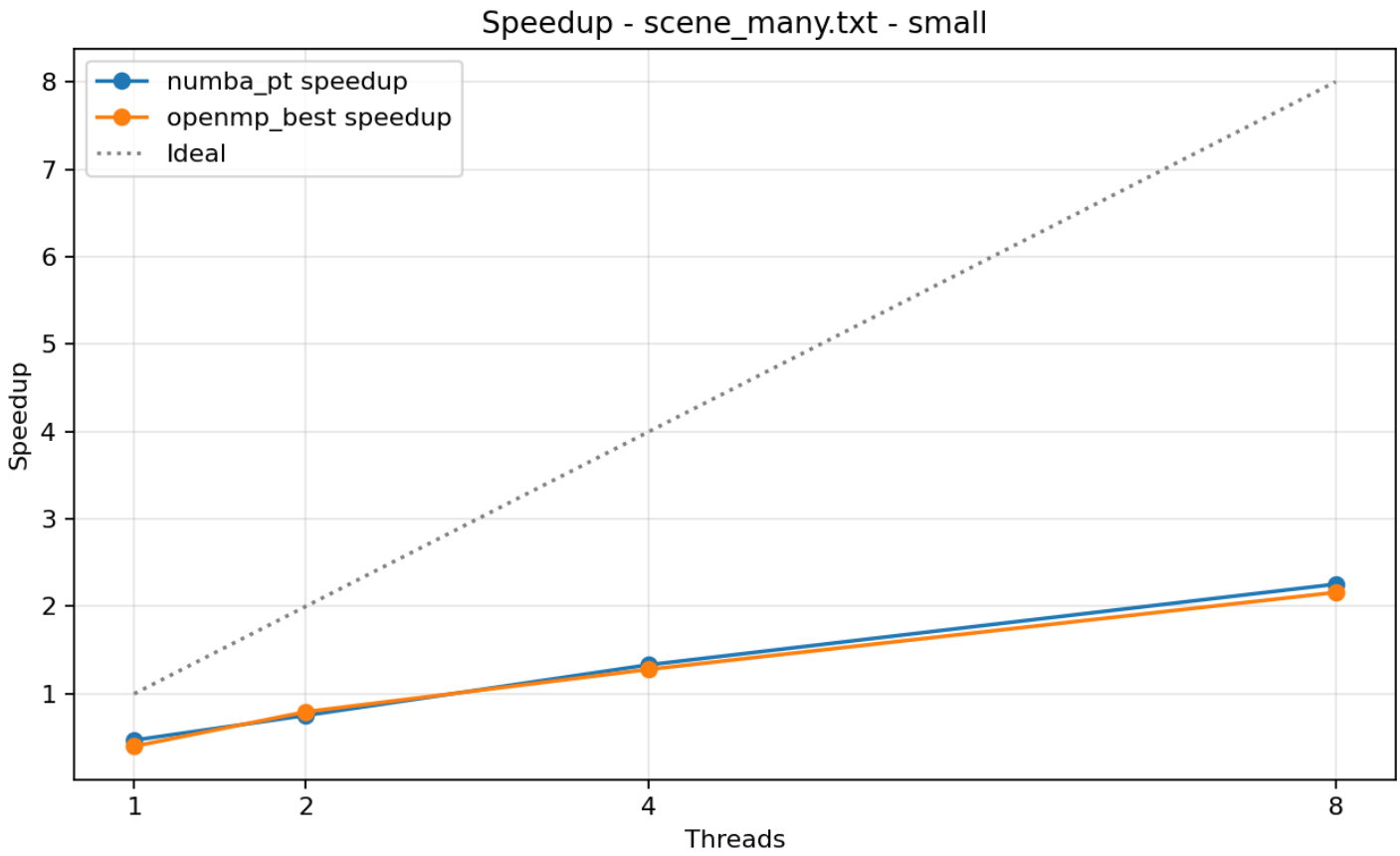

Figure 53: Grupo: Speedup scene_many.txt small

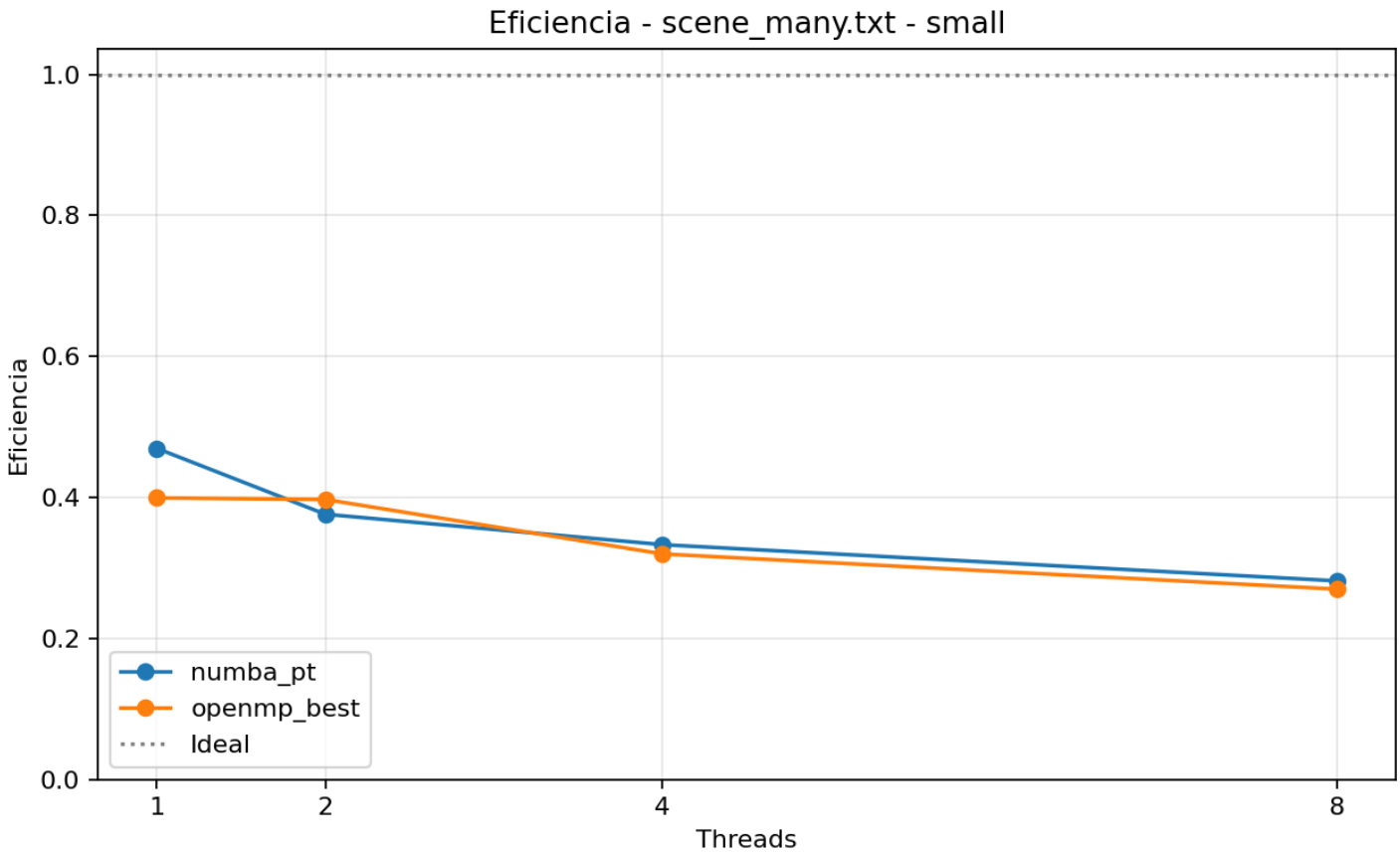

Figure 54: Grupo: Eficiencia scene_many.txt small

Grupo - scene_many.txt medium

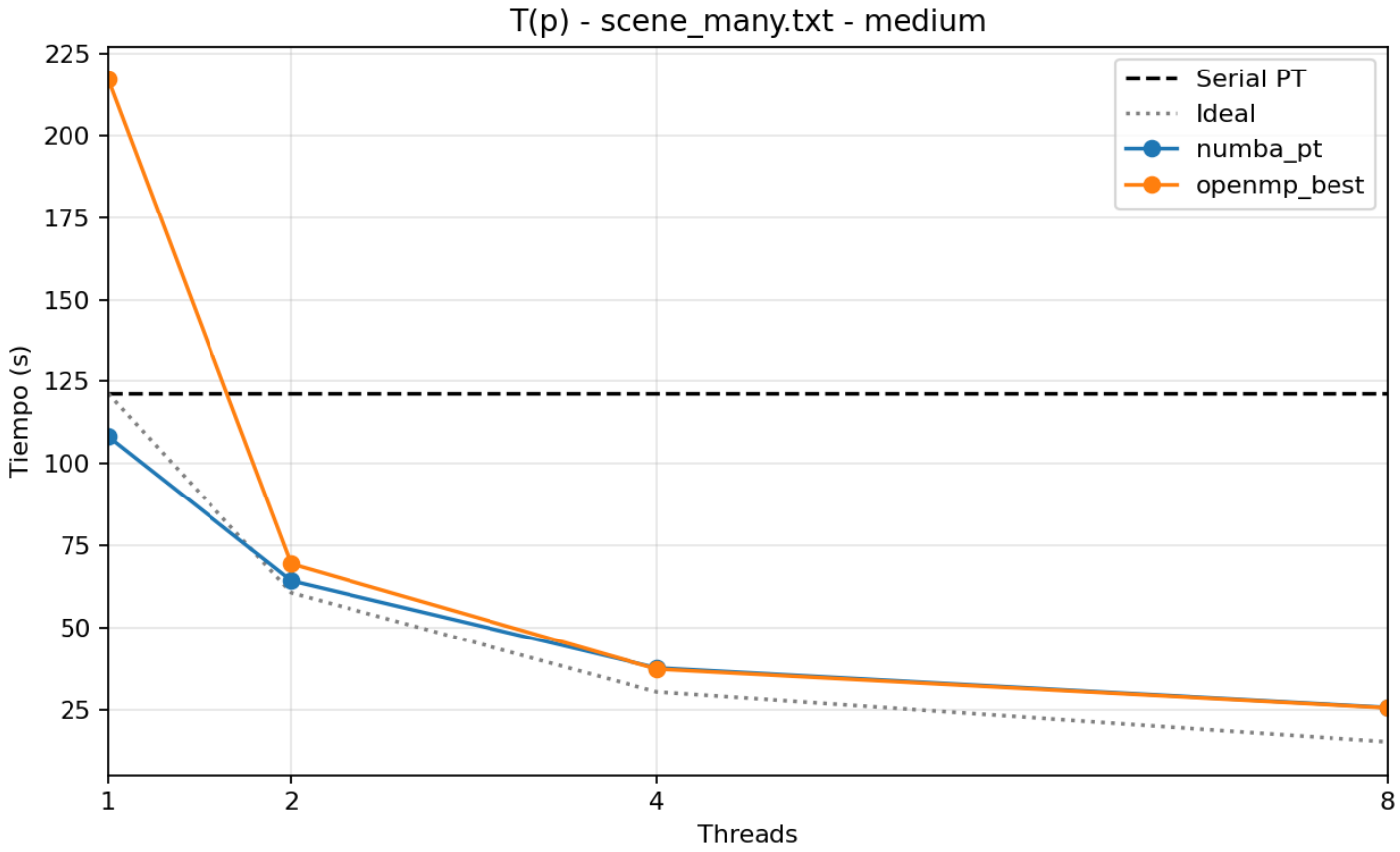

Figure 55: Grupo: Tiempo scene_many.txt medium

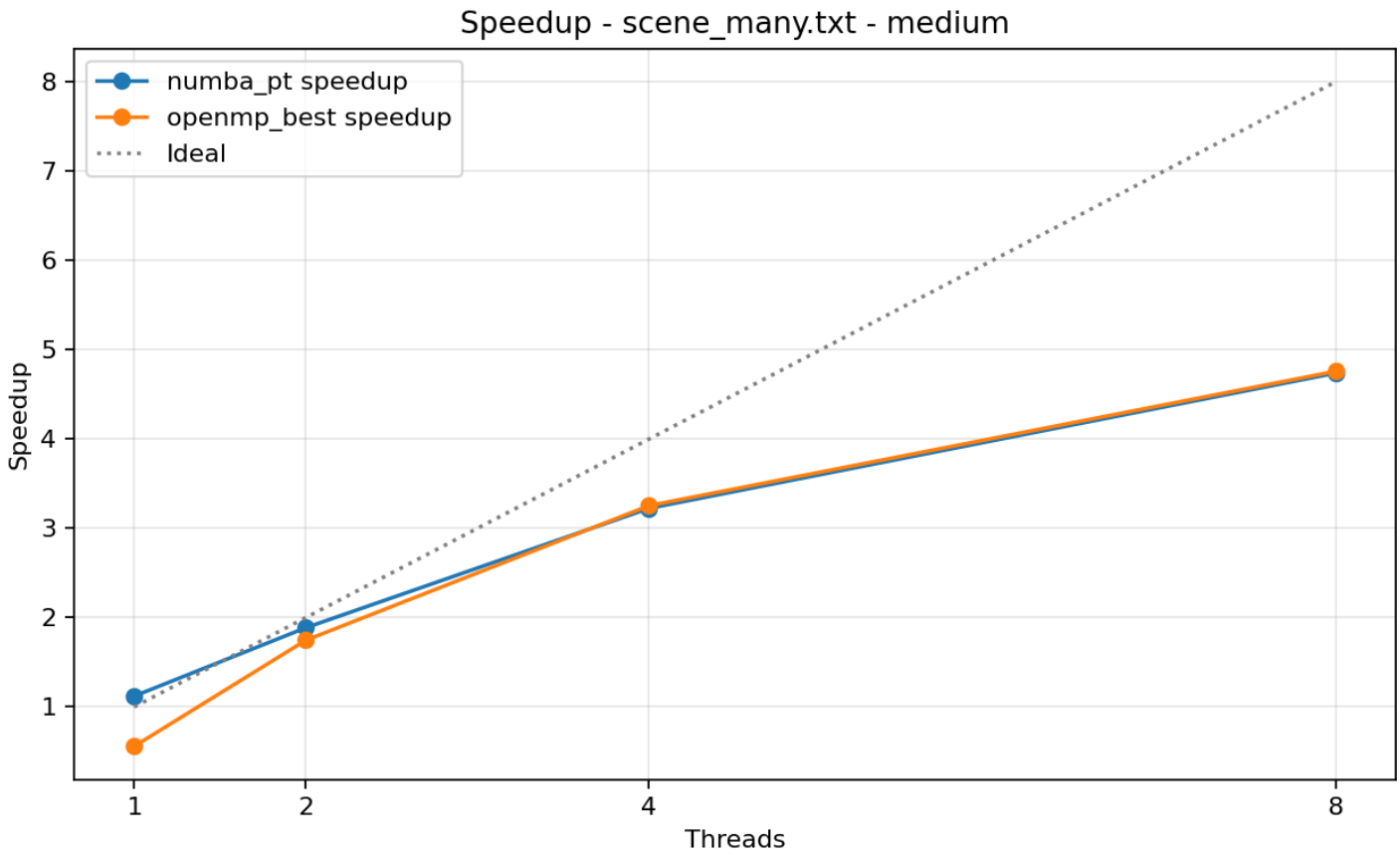

Figure 56: Grupo: Speedup scene_many.txt medium

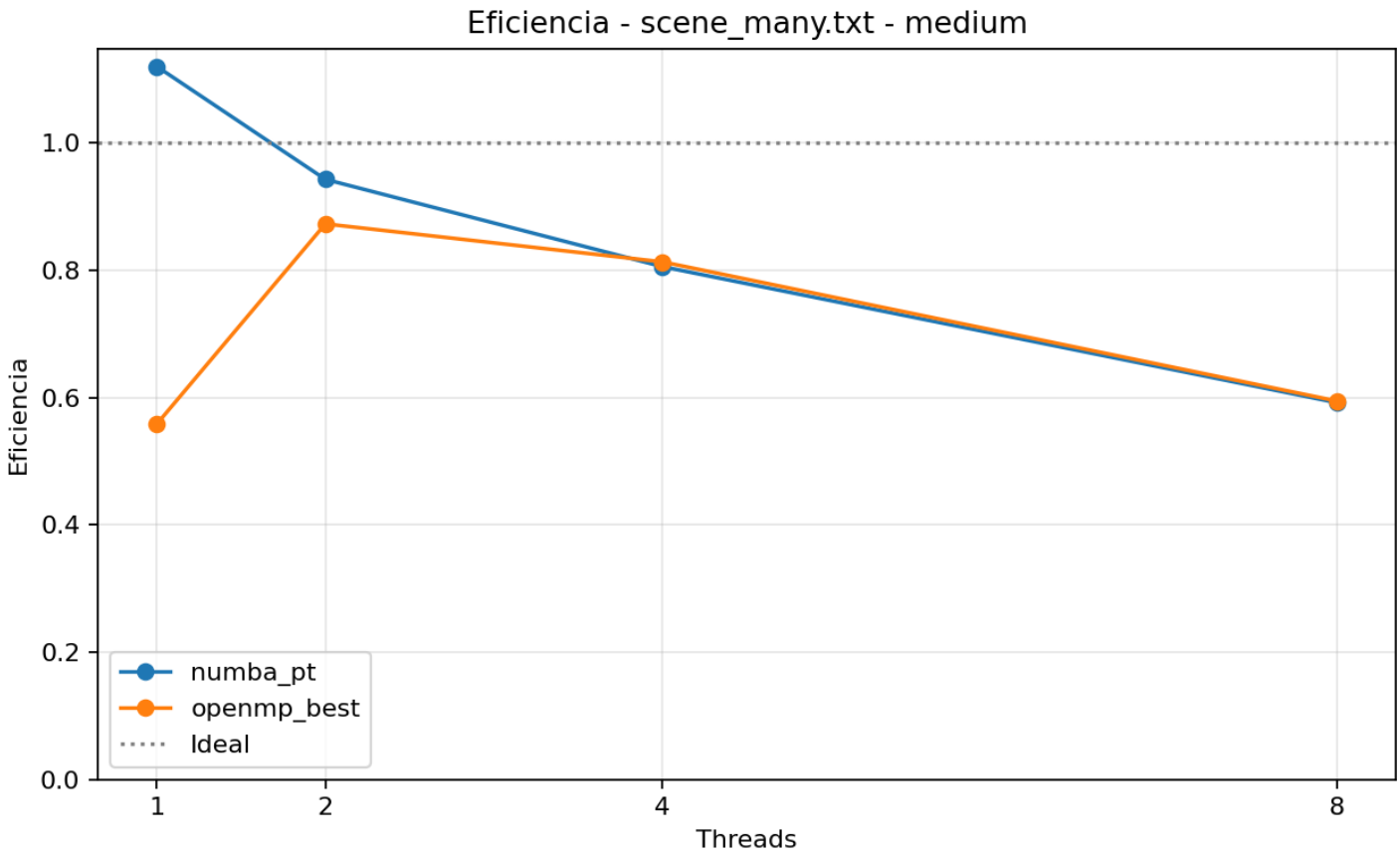

Figure 57: Grupo: Eficiencia scene_many.txt medium

Grupo - scene_many.txt large

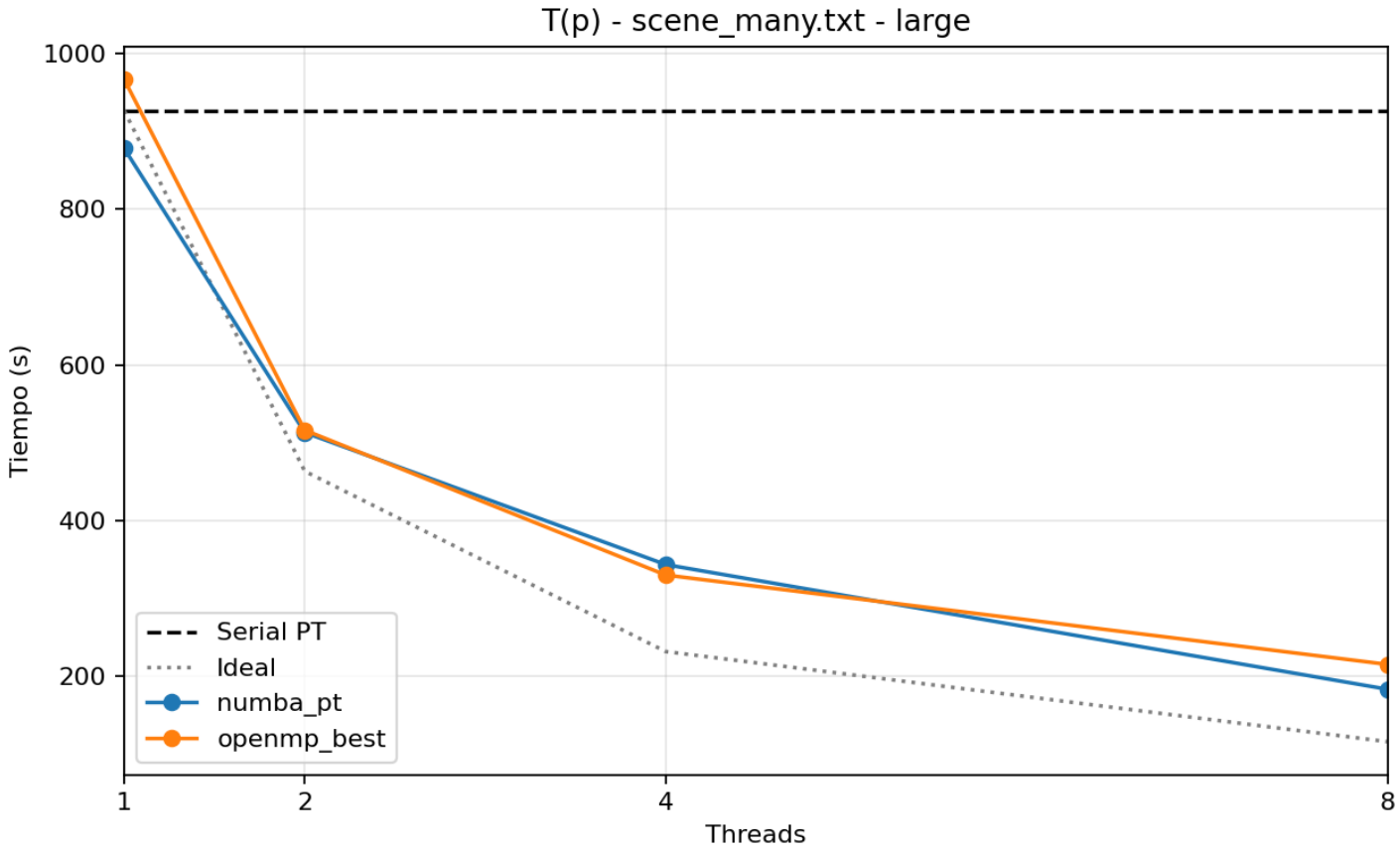

Figure 58: Grupo: Tiempo scene_many.txt large

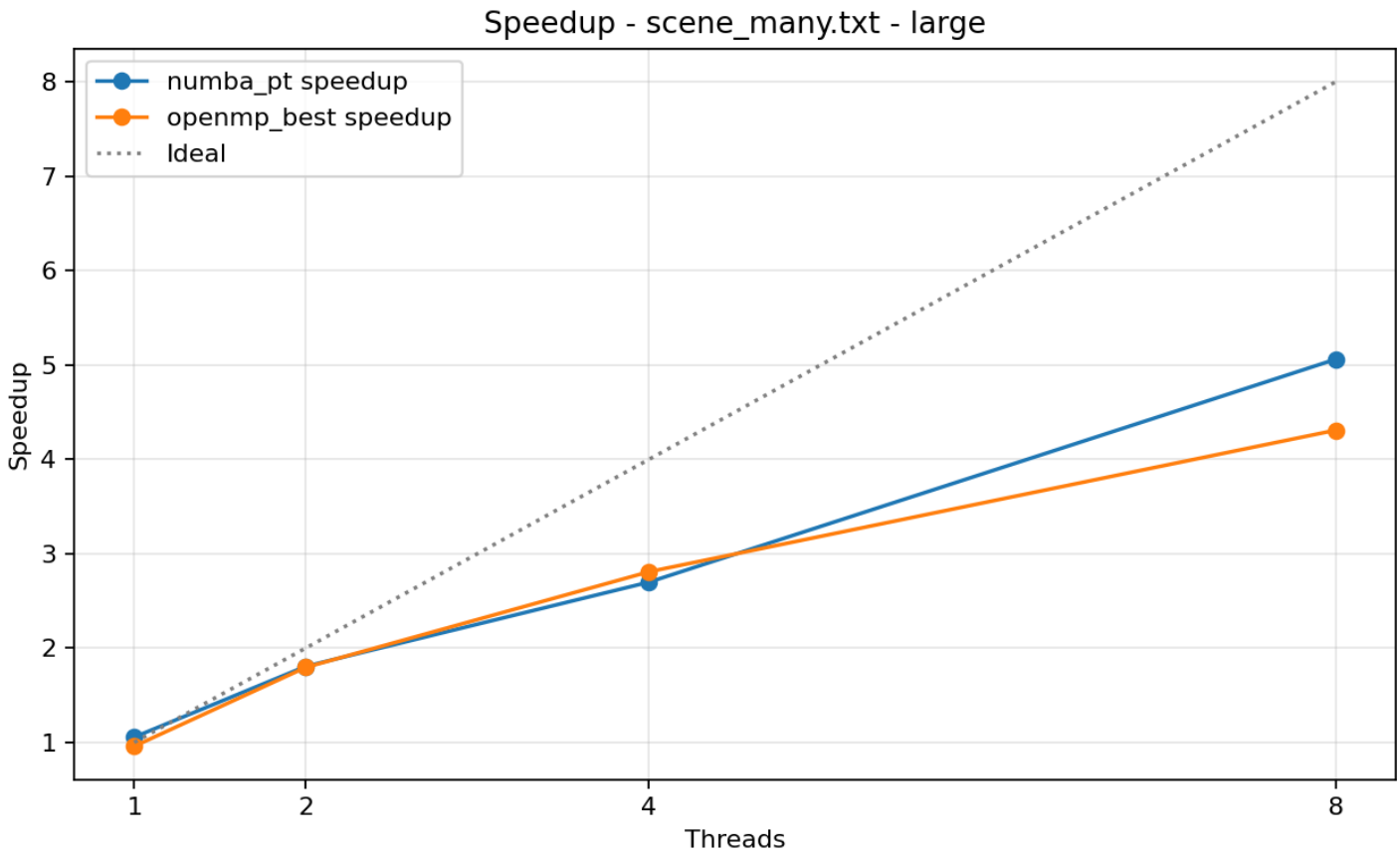

Figure 59: Grupo: Speedup scene_many.txt large

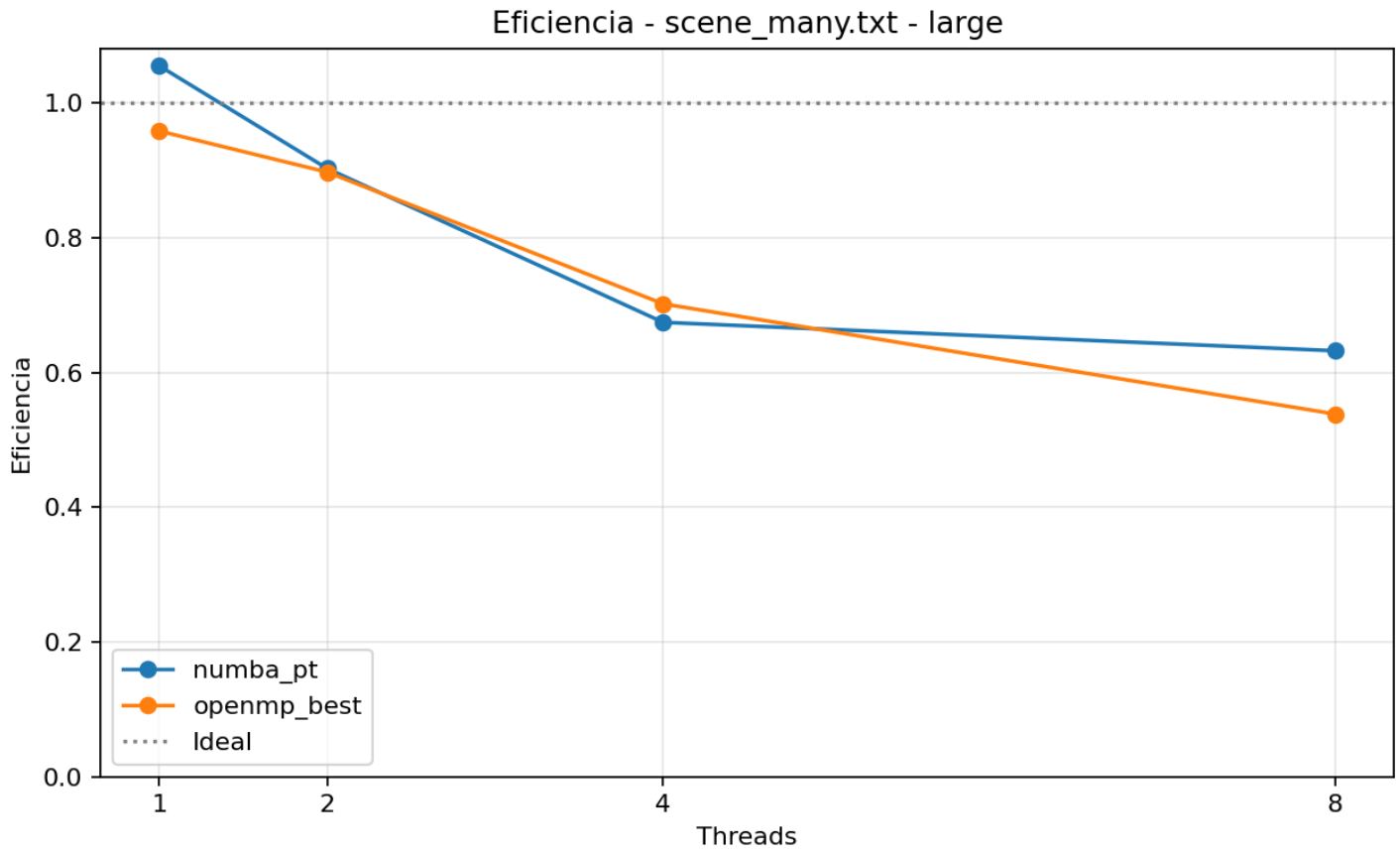

Figure 60: Grupo: Eficiencia scene_many.txt large

Archivos de respaldo

- Resultados locales completos: `results/final/local/results.csv`, `hardware.json`, `report.md`, `report.pdf` y `plots/`.
- Resultados cluster completos: `results/final/cluster/results.csv`, `hardware.json`, `report.md`, `report.pdf`, logs `slurm_2289.out` y `slurm_2289.err`, y `plots/`.
- Resultados grupo (i5-10600K) completos: `results/final/grupo/results.csv`, `hardware.json`, `report.md`, `report.pdf` y `plots/`. Corrida original en `results/full_grupo_jeanf_20260514_105212/`.
- Informe final del grupo: `results/final/informe_entrega_grupo.md` y `results/final/informe_entrega_grupo.pdf`.